

MANUALE COMPLETO DI CLAUDE CODE PER IL BUSINESS

Da Principiante a Esperto — Guida Professionale Esaustiva

INDICE GENERALE DEL MANUALE

Il presente manuale è suddiviso in parti consecutive. Di seguito l'architettura completa:

PARTE 1 — FONDAMENTA E PANORAMICA GENERALE

- Capitolo 1: Introduzione a Claude Code per il Business
- Capitolo 2: Piani di Abbonamento e Strategia di Pricing
- Capitolo 3: Metodi di Accesso e Interfacce Disponibili
- Capitolo 4: La Documentazione Ufficiale come Risorsa Primaria

PARTE 2 — INSTALLAZIONE E CONFIGURAZIONE

- Capitolo 5: Installazione di Claude Code via Terminal
- Capitolo 6: Gli IDE — VS Code e Antigravity
- Capitolo 7: Il Terminal come Interfaccia Avanzata
- Capitolo 8: Configurazione, Status Line e Comandi Fondamentali

PARTE 3 — IL SISTEMA CLAUDE.MD E L'ARCHITETTURA DEL PROGETTO

- Capitolo 9: CLAUDE.md — Il Cervello del Progetto
- Capitolo 10: Il Principio della Direzione (Analogia dell'Arco)
- Capitolo 11: La Cartella .claude e la Sua Struttura Interna
- Capitolo 12: I Tre Livelli — Local, Global, Enterprise

PARTE 4 — COSTRUIRE PROGETTI CON CLAUDE CODE

- Capitolo 13: Tre Metodi per Costruire un Sito Internet
- Capitolo 14: Il Ciclo Task-Do-Verify
- Capitolo 15: Il Metodo Screenshot Loop
- Capitolo 16: Costruire un'Applicazione Complessa (Clone di Trello)

PARTE 5 — MODALITÀ DI PERMESSO E PIANIFICAZIONE

- Capitolo 17: Le Quattro Modalità di Permesso
- Capitolo 18: Plan Mode — L'Approccio Strategico

- Capitolo 19: Bypass Permission — Autonomia Massima

PARTE 6 — CONTEXT MANAGEMENT

- Capitolo 20: Comprendere Contesto e Token
- Capitolo 21: Analisi e Monitoraggio del Contesto
- Capitolo 22: Autocompact e Densità Informativa
- Capitolo 23: Primacy Bias, Recency Bias e Lost in the Middle

PARTE 7 — SUB-AGENTI E AGENT TEAMS

- Capitolo 24: I Sub-agenti — Researcher, Reviewer, QA
- Capitolo 25: Agent Teams — Collaborazione Multi-Agente
- Capitolo 26: Costi, ROI e Utilizzo Strategico

PARTE 8 — IL SISTEMA DELLE SKILL

- Capitolo 27: Architettura delle Skill
- Capitolo 28: Creare Skill Personalizzate
- Capitolo 29: Il Marketplace delle Skill
- Capitolo 30: La Qualità dei Dati di Riferimento

PARTE 9 — MCP (MODEL CONTEXT PROTOCOL)

- Capitolo 31: Comprendere l'MCP
- Capitolo 32: Installare e Gestire gli MCP
- Capitolo 33: MCP vs Skill — Impatto sul Contesto
- Capitolo 34: Chrome Dev Tool MCP

PARTE 10 — FUNZIONALITÀ AVANZATE E DEPLOYMENT

- Capitolo 35: Hooks — Automazione Basata su Eventi
- Capitolo 36: Auto Memory e Persistenza tra Sessioni
- Capitolo 37: Git Worktrees e Version Control
- Capitolo 38: Deployment e Monetizzazione

PARTE 1 — FONDAMENTA E PANORAMICA GENERALE

CAPITOLO 1: INTRODUZIONE A CLAUDE CODE PER IL BUSINESS

1.1 — Definizione del Concetto

Claude Code è uno strumento di sviluppo e automazione basato sull'intelligenza artificiale, creato da Anthropic, che permette a qualsiasi persona — indipendentemente dal proprio background tecnico — di costruire applicazioni,

automatizzare processi, gestire workflow aziendali e creare sistemi complessi utilizzando il linguaggio naturale come interfaccia principale.

1.2 — Spiegazione Espansa

Claude Code non è un semplice chatbot. È un ambiente completo nel quale l'utente interagisce con un modello di linguaggio avanzato (LLM) che ha la capacità di:

- Leggere file e cartelle presenti nel computer dell'utente
- Scrivere, modificare e cancellare codice in modo autonomo
- Eseguire comandi nel terminal del computer
- Navigare nel web tramite strumenti integrati (MCP)
- Gestire sotto-agenti che lavorano in parallelo
- Memorizzare informazioni tra una sessione e l'altra
- Pianificare progetti complessi prima di eseguirli
- Auto-verificare i propri risultati e correggerli iterativamente

La differenza fondamentale tra Claude Code e un normale utilizzo di ChatGPT o di un chatbot è questa: Claude Code opera direttamente all'interno del vostro sistema operativo. Non è confinato in una finestra del browser. Ha accesso reale a file, cartelle, documenti, e può modificarli. Questo lo rende uno strumento di produttività radicalmente diverso da qualsiasi altro tool basato su AI.

1.3 — Perché Questo Concetto È Importante

La comprensione di cosa sia realmente Claude Code è il prerequisito per tutto il resto del manuale. Molte persone si avvicinano a Claude Code pensando che sia "un altro ChatGPT" e lo usano allo stesso modo: scrivono un prompt, leggono una risposta, scrivono un altro prompt. Questo è un utilizzo che sfrutta forse il 5-10% delle capacità reali dello strumento.

Claude Code è progettato per essere un collaboratore operativo. Può costruire interi siti web, creare applicazioni con autenticazione e pagamenti integrati, gestire social media, analizzare repository di codice, e fare deployment di servizi nel cloud — tutto attraverso istruzioni in linguaggio naturale.

1.4 — Interpretazione Pratica

A livello pratico, Claude Code viene utilizzato dall'autore della guida originale in questi contesti:

Contesto	Applicazione
Gestione aziendale personale	Automazione di processi interni, gestione documenti, workflow
Coaching business	Creazione di materiale, gestione clienti, automazioni

Implementazione in aziende terze	La più grande genera 70 milioni di euro di fatturato annuo
Formazione	Insegnamento a centinaia di persone per scopi personali e professionali

Questo significa che Claude Code è uno strumento che scala dal singolo individuo fino all'impresa di grandi dimensioni.

1.5 — Meccanismo Sottostante

Il funzionamento di Claude Code si basa su un modello di linguaggio (LLM) che ha accesso a una serie di "strumenti" (tools):

```
[Utente] → scrive un prompt in linguaggio naturale
↓
[Claude Code] → interpreta il prompt
↓
[Strumenti] → esegue azioni reali (bash commands, file editing, web navigation)
↓
[Output] → risultato visibile (codice modificato, file creato, sito costruito)
```

Ogni volta che l'utente scrive qualcosa, Claude Code non si limita a generare testo. Può decidere di:

- Leggere un file specifico nel progetto per capire il contesto
- Eseguire un comando nel terminal
- Modificare righe di codice
- Creare nuovi file
- Chiamare un sotto-agente per un compito specifico
- Navigare nel web per fare ricerche
- Fare uno screenshot per verificare il risultato visivo

1.6 — Errori Comuni e Frintendimenti

Errore 1: "Serve un background tecnico"

Falso. La guida originale specifica chiaramente: *"Non avrete bisogno di un background tecnico per seguire questo corso perché partirò dall'inizio e vi insegnerò tutti gli argomenti in maniera naturale e graduale."* Claude Code è progettato per essere usato da chiunque. Il terminal può sembrare intimidatorio all'inizio, ma le operazioni fondamentali sono estremamente semplici.

Errore 2: "È solo per software developer"

Falso. Claude Code viene usato per gestione aziendale, marketing, social media management, content creation, lead generation, contabilità, e molto altro. Lo sviluppo software è solo una delle applicazioni.

Errore 3: "Basta un prompt per ottenere un risultato perfetto"

Questo è uno dei frintendimenti più pericolosi. Claude Code richiede direzione, pianificazione e iterazione. Il concetto di "one shot" (ottenere tutto perfetto al primo tentativo) è fuorviante e spesso irrealistico. L'autore della guida specifica chiaramente: *"Quando trovate un video dove qualcuno vi dice 'ah one shot, poi lo provate' e non ci viene mai, il motivo è perché probabilmente l'hanno provato 25 volte."*

Errore 4: "Più strumenti installo, meglio è"
Falso. Come vedremo nel dettaglio nei capitoli dedicati agli MCP e al context management, ogni strumento installato consuma contesto. Un MCP come ClickUp può consumare il 27% del contesto totale disponibile prima ancora di iniziare a lavorare. La strategia è selettiva, non accumulativa.

1.7 — Insight Avanzato

La promessa fondamentale di Claude Code è questa: una volta padroneggiato, la vostra produttività aumenterà enormemente, a prescindere dalla ragione per cui lo userete. Questa affermazione non è un'iperbole di marketing. La ragione è strutturale: Claude Code equivale ad avere un software developer disponibile 24 ore su 24, che esegue in minuti ciò che normalmente richiede ore o giorni, e che può essere istruito con linguaggio naturale.

Il percorso di apprendimento delineato dalla guida è strutturato in quattro livelli progressivi:

PRINCIPIANTE → INTERMEDIO → AVANZATO → ESPERTO

[Basi]	[.claude]	[Skill]	[Chrome Dev Tool]
[Installazione]	[Sub-agenti]	[MCP]	[Agent Teams]
[IDE]	[Modalità]	[Marketplace]	[Git Worktrees]
[CLAUDE.md]	[Plan Mode]	[Sistemi Auto]	[Deployment]
[Web App]	[Context Mgmt]	[Plugin]	[Scaling]
	[Slash Commands]		

[Hooks]

Ogni livello si costruisce sopra il precedente. È un processo cumulativo dove ogni concetto apre la porta al successivo.

1.8 — Contesto Applicato

Per chiunque stia leggendo questo manuale, il punto di partenza è questo: non serve sapere programmare. Serve avere la volontà di imparare un processo strutturato. Claude Code è il ponte tra un'idea e la sua realizzazione tecnica, e questo manuale vi insegnerà a costruire quel ponte mattone dopo mattone.

L'investimento iniziale è di \$17 al mese (piano Pro), che l'autore definisce come *"il miglior \$17 della vostra vita"* perché il ROI (Return on Investment) in termini di produttività è definito "mostruoso".

CAPITOLO 2: PIANI DI ABBONAMENTO E STRATEGIA DI PRICING

2.1 — Definizione del Concetto

Anthropic offre diversi piani di accesso a Claude e a Claude Code, ognuno con caratteristiche, limiti e modalità di pagamento differenti. La scelta del piano corretto è una decisione strategica che impatta direttamente sulla capacità operativa e sui costi.

2.2 — Spiegazione Espansa

Esistono tre macro-categorie di piani:

A) Piano Individuale (Subscription-Based)

Si tratta di un abbonamento mensile a prezzo fisso. L'utente paga una cifra predefinita e riceve accesso allo strumento con determinati limiti di utilizzo.

All'interno del piano individuale esistono tre livelli (tier):

Tier	Prezzo	Claude Code Incluso	Note
Gratuito	\$0	✗ No	Solo chat base, niente Claude Code
Pro	\$17/mese	✓ Sì	Consigliato per chi inizia. Include Claude Code e Cowork
Max	\$100/mese+	✓ Sì	Per uso intensivo. L'autore usa questo piano + \$300 aggiuntivi

Punto critico: Claude Code è disponibile solo nei piani Pro e Max. Il piano gratuito non include Claude Code né Cowork. Questo è un dettaglio che molti trascurano e che porta a frustrazione quando si tenta di seguire tutorial senza avere il piano corretto.

B) Piano Enterprise/Team

Progettato per aziende e team di lavoro. Include funzionalità di gestione permessi, controllo accessi, e configurazioni condivise a livello organizzativo.

C) Piano API (Pay-per-Use)

In questa modalità non c'è un abbonamento fisso. Si paga esclusivamente per l'utilizzo effettivo del modello: ogni token processato ha un costo.

2.3 — Perché Questo Concetto È Importante

La scelta del piano ha implicazioni dirette su tre dimensioni:

1. Capacità operativa: Con il piano gratuito non potete usare Claude Code. Punto. Tutto ciò che è insegnato in questo manuale richiede almeno il piano Pro.
2. Controllo dei costi: Il piano API può sembrare attraente perché "paghi solo quello che usi", ma l'autore avverte esplicitamente: *"Vi consiglio di non usare il piano API a meno che non sappiate esattamente cosa state facendo, perché altrimenti il costo potrebbe esplodervi senza accorgervene."* Questo è particolarmente vero quando si usano Agent Teams o MCP che consumano grandi quantità di token.
3. ROI: \$17 al mese per avere accesso a uno strumento che può sostituire il lavoro di un software developer è un ROI che l'autore definisce "mostruoso". Per contestualizzare: un junior software developer costa almeno \$2.000-3.000 al mese, e Claude Code può eseguire molte delle stesse operazioni.

2.4 — Interpretazione Pratica

Se siete nuovi: Partite con il piano Pro a \$17/mese. Non c'è ragione di partire dal Max o dall'API. Il piano Pro vi dà accesso completo a Claude Code e a tutto ciò che questo manuale insegna.

Se siete utenti avanzati o aziendali: Il piano Max a \$100/mese o superiore è giustificato quando il volume di utilizzo è elevato. L'autore della guida spende circa €400/mese (piano Max + crediti aggiuntivi) perché lo utilizza tutto il giorno, ogni giorno, per gestire più business.

Se siete un'azienda: Il piano Enterprise/Team diventa necessario quando servono controlli di permesso granulari, gestione del team, e configurazioni condivise (concetti che saranno approfonditi nel Capitolo 12 sui livelli Local/Global/Enterprise).

2.5 — Meccanismo Sottostante

La ragione per cui il piano API è pericoloso per i non esperti è legata alla struttura dei costi:

```
Piano Subscription (Pro/Max):
→ Costo fisso mensile
→ Limite di utilizzo definito
→ Prevedibilità totale dei costi
→ Nessuna sorpresa
```

```
Piano API:
→ Nessun costo fisso
→ Pagamento per ogni token utilizzato
→ I token consumati crescono con:
  • Lunghezza del prompt
  • Numero di iterazioni
  • Utilizzo di sub-agenti
  • Agent Teams (3-5x il consumo normale)
  • MCP attivi
```

```
→ Possibilità di costi imprevisti molto elevati
```

Per dare un'idea concreta: durante la dimostrazione degli Agent Teams nella guida originale, l'autore ha speso circa €6-7 in meno di 5 minuti di analisi. In un piano API, questo costo sarebbe stato reale e addebitato. In un piano subscription, è incluso nell'abbonamento mensile.

2.6 — Errori Comuni

Errore 1: Iniziare con il piano API per "risparmiare"

Questo è controintuitivo ma vero: il piano API è più costoso per la maggior parte degli utenti, non meno costoso. La ragione è che quando si impara, si fanno molti tentativi, si sperimentano molte cose, e ogni tentativo consuma token. Con un piano subscription, tutti questi esperimenti sono inclusi nel prezzo fisso.

Errore 2: Restare sul piano gratuito e aspettarsi di usare Claude Code

Il piano gratuito non include Claude Code. Punto. Non è possibile seguire questo manuale con il piano gratuito.

Errore 3: Non monitorare il consumo nel piano API

Se per qualche ragione usate il piano API, il monitoraggio del consumo è obbligatorio. Le interfacce di Claude Code mostrano il costo stimato di ogni interazione (visibile nella barra di stato), ma questo può sfuggire facilmente durante sessioni intensive.

2.7 — Insight Avanzato

Nella barra di stato di Claude Code (che vedremo come configurare nel Capitolo 8), viene mostrato un costo per ogni interazione. Questo costo rappresenta quanto avreste pagato se foste sul piano API. È un dato puramente informativo per chi usa un piano subscription, ma diventa critico per chi usa il piano API.

L'autore mostra che una singola interazione ("Ciao, come stai?") ha un costo API di pochi centesimi, ma operazioni complesse come l'analisi di una repository con Agent Teams possono costare decine di euro in pochi minuti.

La formula decisionale per la scelta del piano è:

```
SE utilizzo_giornaliero > 2_ore → Piano Max
SE utilizzo_giornaliero tra 30_min e 2_ore → Piano Pro
SE utilizzo_occasionale < 30_min/giorno → Piano Pro (comunque, per il prezzo)
SE team > 3_persone → Piano Enterprise
```

```
SE esperienza_tecnica = alta E controllo_costi = necessario → Piano API (con cautela)
```

CAPITOLO 3: METODI DI ACCESSO E INTERFACCE DISPONIBILI

3.1 — Definizione del Concetto

Claude Code può essere utilizzato attraverso diverse interfacce, ognuna con caratteristiche specifiche. La scelta dell'interfaccia impatta sulle funzionalità disponibili e sulla modalità di lavoro.

3.2 — Spiegazione Espansa

Esistono cinque modalità principali per accedere a Claude:

Modalità 1: Web App

- URL: claude.ai
- Cos'è: Un'interfaccia grafica (GUI — Graphical User Interface) accessibile dal browser
- GUI significa: Un'interfaccia visiva con bottoni, campi di testo, e elementi cliccabili. "GUI" sta per "Graphical User Interface" ed è semplicemente ciò che vedete quando interagite con un'applicazione tramite bottoni e elementi visivi, anziché tramite comandi testuali
- Utilizzo: Conversazione diretta con Claude, senza accesso diretto ai file del computer

Modalità 2: Chrome Extension

- Cos'è: Un piccolo pulsante/estensione installabile nel browser Chrome
- Funzione: Permette di interfacciarsi con Claude direttamente durante la navigazione web
- Vantaggio: Claude può "vedere" e interagire con le pagine web che state visitando

Modalità 3: Desktop App

- Cos'è: Un'applicazione scaricabile e installabile sul computer
- Interfaccia: Quando la si apre, presenta tre sezioni: Chat, Cowork, e Code
- Vantaggio: Esperienza nativa, senza bisogno del browser

Modalità 4: IDE (Integrated Development Environment)

- Cos'è: Ambienti di sviluppo integrati come VS Code o Antigravity
- In italiano: "Ambienti di sviluppo integrati" — sono software progettati per scrivere e gestire codice, ma con Claude Code integrato diventano strumenti di produttività generali
- Vantaggio: Visualizzazione completa dei file del progetto + interazione con Claude Code nella stessa interfaccia
- I due IDE principali sono:
 - Antigravity (prodotto Google, lanciato circa novembre 2025)
 - VS Code (prodotto Microsoft, l'IDE storico/"OG")

Modalità 5: Terminal ★ (Raccomandata)

- Cos'è: L'interfaccia a linea di comando del computer
- Come si accede (Mac): Command + Barra spaziatrice → digitare "terminal" → Enter
- Perché è raccomandata: Sblocca funzionalità avanzate non disponibili nelle altre modalità

3.3 — Perché Questo Concetto È Importante

La scelta dell'interfaccia non è puramente estetica. Ha implicazioni funzionali concrete:

FUNZIONALITÀ DISPONIBILI PER INTERFACCIA:

	Web App	Chrome	Desktop	IDE	Terminal
Chat base:	✓	✓	✓	✓	✓
Accesso file locali:	✗	✗	✓	✓	✓
Claude Code completo:	✗	✗	✓	✓	✓
Bypass Permission:	✗	✗	✓	✓	✓
Agent Teams:	✗	✗	✗	✗	✓
Config avanzata:	✗	✗	✗	✗	✓
/context analysis:	✗	✗	✗	✗	✓
Status Line completa:	✗	✗	✗	✗	✓
Dangerously Skip (YOLO):	✗	✗	✓*	✓*	✓

* Nelle IDE richiede configurazione manuale nelle impostazioni

Come si vede dalla tabella, il Terminal è l'unica interfaccia che dà accesso a tutte le funzionalità. L'autore lo esprime così: *"Il terminal è importante perché vi permetterà di sbloccare delle funzionalità avanzate che non potete usare nelle altre modalità."*

3.4 — Interpretazione Pratica

Per chi inizia: L'IDE (Antigravity o VS Code) è il punto di partenza più accessibile. Ha un'interfaccia visiva, mostra i file del progetto, e integra Claude Code in modo intuitivo.

Per chi vuole progredire: Il Terminal dovrebbe essere appreso gradualmente. L'autore rassicura: *"Sembra difficile all'inizio ma veramente non lo è."* Il Terminal non è altro che una finestra dove si digitano comandi testuali, e i comandi fondamentali sono pochi e semplici.

Raccomandazione dell'autore: Prendere familiarità con il Terminal "piano alla volta" perché le funzionalità avanzate più importanti (Agent Teams, analisi del contesto, configurazione completa) sono accessibili solo da lì.

3.5 — Meccanismo Sottostante: IDE a Confronto

L'autore utilizza e raccomanda Antigravity (prodotto Google) rispetto a VS Code (prodotto Microsoft) per le seguenti ragioni:

Caratteristica	Antigravity	VS Code

Interfaccia	Più pulita, più moderna	Più rigida, meno pulita
Focus	Incentrato sulla parte di agenti	Incentrato sullo sviluppo tradizionale
Modelli disponibili	Modelli Google + Claude Code + GPT	Claude Code (tramite estensione)
Lancio	~Novembre 2025	Storico, consolidato
Preferenza autore	✅ Preferito	Utilizzato ma meno preferito

Un dettaglio importante: L'autore utilizza una strategia specifica per dividere il lavoro tra modelli:

- Backend (codice logico, server-side) → Claude Code
- Frontend (interfaccia visiva, design) → Modelli Google tramite Antigravity

Questa strategia può essere anche automatizzata tramite configurazioni nel CLAUDE.md (concetto che verrà approfondito nei capitoli dedicati).

3.6 — Errori Comuni

Errore 1: Usare solo la Web App e pensare di usare Claude Code

La Web App di claude.ai è Claude (il chatbot), non Claude Code (lo strumento di sviluppo). Sono due cose diverse. Claude Code richiede un piano Pro o Max e un'interfaccia che abbia accesso al sistema operativo (IDE, Desktop App, o Terminal).

Errore 2: Spaventarsi del Terminal e non provarlo mai

Il Terminal è intimidatorio per chi non ha background tecnico, ma le operazioni fondamentali sono:

- Digitare `claude` e premere Enter per avviare Claude Code
 - Digitare `clear` per pulire la schermata
 - Digitare comandi che Claude stesso vi suggerisce
- Non serve sapere altro per iniziare.

Errore 3: Non installare l'estensione Claude Code nell'IDE

Claude Code non è integrato di default in VS Code o Antigravity. Va installato come estensione. Il processo è:

1. Aprire l'IDE
2. Andare nella sezione Estensioni (icona a forma di blocchi)

3. Cercare "Claude Code"
4. Premere "Install"

Errore 4: Confondere il workspace/cartella con l'interfaccia

A prescindere dall'IDE che usate, ciò che conta è la cartella (workspace/folder) del progetto. Se aprite la stessa cartella in VS Code, in Antigravity, o la navigate dal Terminal, vedrete gli stessi file e le stesse modifiche. La cartella è il progetto. L'IDE è solo il modo in cui la visualizzate.

L'autore lo spiega così: *"A prescindere dall'interfaccia che usate, la cartella è quella che conta, perché la cartella è quella nella quale avremo tutti i cambiamenti, tutti gli update."*

3.7 — Insight Avanzato

Shortcut di accesso rapido a Claude Code:

- In Antigravity (Mac): Command + Shift + ESC → Apre direttamente Claude Code
- Icona stella: Sia in VS Code che in Antigravity, Claude Code è accessibile tramite un'icona a forma di stella nella barra laterale

Il concetto di file "in contesto":

Quando Claude Code è aperto in un IDE, nella parte superiore della chat viene mostrato quale file è attualmente "guardato" dal sistema. Questo è un indicatore visivo cruciale perché:

- Se chiedete di modificare "questo file", Claude modifica il file attualmente visualizzato
- Se siete nel file sbagliato, Claude potrebbe modificare il file errato
- È buona pratica assicurarsi di essere nel file corretto prima di dare istruzioni

L'autore spiega: *"Assicuriamoci di essere anche magari nel file giusto, di modo tale che non ci siano probabilità che lei sbagli."*

CAPITOLO 4: LA DOCUMENTAZIONE UFFICIALE COME RISORSA PRIMARIA

4.1 — Definizione del Concetto

La documentazione ufficiale di Claude e Claude Code, accessibile su docs.anthropic.com, è la fonte primaria e più aggiornata di informazioni sullo strumento.

4.2 — Spiegazione Espansa

L'autore ha studiato l'intera documentazione ufficiale e la considera la risorsa più importante per chiunque voglia padroneggiare Claude Code. Esistono due ragioni fondamentali per questa posizione:

Ragione 1: Aggiornamento costante

La documentazione viene aggiornata continuamente perché Claude Code riceve feedback da migliaia di software developer. Questi feedback hanno una "visibilità" tecnica che utenti normali non hanno, e permettono alla piattaforma di evolvere molto rapidamente.

Ragione 2: Autorevolezza

Non esiste una fonte più autorevole della documentazione ufficiale. Qualsiasi tutorial, video, o corso (incluso quello su cui si basa questo manuale) può diventare obsoleto quando la piattaforma si aggiorna. La documentazione ufficiale è sempre allineata con la versione corrente.

4.3 — Perché Questo Concetto È Importante

L'autore fa un'osservazione critica: *"Molte persone hanno tutti questi documenti anche per OpenAI eccetera che non leggono, e perché? Perché c'è un certo effort cognitivo da fare per leggersi e mettersi qui a leggere tutte queste cose. Però ha un ROI molto molto alto."*

Il costo cognitivo di leggere la documentazione è reale. È un documento tecnico, in inglese, denso di informazioni. Ma il ritorno sull'investimento di tempo è enorme perché:

1. Vi dà la comprensione fondamentale di come lo strumento funziona
2. Vi permette di trovare funzionalità che non sono coperte in nessun tutorial
3. Vi tiene aggiornati su funzionalità appena rilasciate
4. Vi dà la terminologia corretta per formulare prompt più efficaci

4.4 — Avvertenza sulla Volatilità dell'Interfaccia

L'autore avverte esplicitamente: *"È possibile che dal momento in cui io registro il video a quando voi lo vedrete cambierà enormemente. Quindi non soffermatevi a dove sono i bottoni o dove sono le cose. Capite che cosa stiamo cercando."*

Questo principio si applica a tutto ciò che riguarda interfacce e posizioni di pulsanti. Ciò che non cambia sono i concetti sottostanti: come funziona il contesto, come strutturare un CLAUDE.md, come usare i sub-agenti, come gestire le permission. Queste sono conoscenze durature.

4.5 — La Seconda Fonte: Twitter/X e Boris

Oltre alla documentazione ufficiale, l'autore utilizza una seconda fonte specifica: Twitter/X, e in particolare i post di Boris, il creatore di Claude Code.

La strategia è concreta:

1. Andare su X (Twitter)
2. Usare Grok (l'AI integrata in X) per chiedere: *"Raccogliami tutto quello che è successo nell'ultimo mese in termini di best practice di Claude Code e riassumilo"*
3. Grok analizza i post recenti dei power user di Claude Code e produce un riassunto
4. Copiare e incollare queste best practice nel proprio progetto

L'autore descrive questo come il suo metodo principale per restare aggiornato: *"Io uso solo per questo Twitter letteralmente, perché molte delle persone... è una delle poche piattaforme in cui si condivide qualcosa di valore e già una miniera d'oro."*

4.6 — Errori Comuni

Errore: Affidarsi esclusivamente a tutorial e corsi senza consultare la documentazione
I tutorial diventano obsoleti. La documentazione no. Un approccio equilibrato è:

- Usare tutorial e corsi per capire i concetti e vedere esempi pratici

- Usare la documentazione ufficiale per verificare, approfondire, e restare aggiornati

Errore: Ignorare i post di Boris su Twitter/X

Il creatore di Claude Code condivide regolarmente best practice, consigli, e annunci di nuove funzionalità. Seguirlo è un investimento di tempo minimo con un ritorno informativo elevato.

4.7 — Insight Avanzato

L'autore rivela un workflow specifico per integrare le best practice nel proprio progetto:

1. Ogni mese, usare Grok su X per raccogliere le ultime best practice
2. Copiare il riassunto prodotto da Grok
3. Incollarlo nel proprio CLAUDE.md o nelle proprie regole
4. Questo mantiene il progetto allineato con le ultime raccomandazioni della community

Questo è un esempio di come Claude Code, combinato con altri strumenti AI, crea un ciclo di miglioramento continuo.

PARTE 2 — INSTALLAZIONE E CONFIGURAZIONE

CAPITOLO 5: INSTALLAZIONE DI CLAUDE CODE VIA TERMINAL

5.1 — Definizione del Concetto

L'installazione di Claude Code è il processo attraverso il quale il software viene scaricato e integrato all'interno del sistema operativo del computer. Una volta installato, Claude Code ha accesso diretto a cartelle, file, documenti e a qualsiasi risorsa presente nel computer dell'utente.

5.2 — Spiegazione Espansa

Il processo di installazione è sorprendentemente semplice e si compone di pochi passaggi sequenziali. Nonostante il Terminal possa sembrare intimidatorio per chi non ha esperienza tecnica, l'operazione è meccanica: si copia un comando, si incolla, si preme Enter.

Passaggio 1: Trovare il comando di installazione

- Andare su docs.anthropic.com (la documentazione ufficiale)
- Cercare la sezione "Get Started"
- Nella pagina troverete una serie di comandi per diversi sistemi operativi (Mac, Linux, Windows)
- Il comando corretto per il vostro sistema operativo va copiato premendo il pulsante di copia accanto ad esso

Passaggio 2: Aprire il Terminal

- Su Mac: premere Command + Barra Spaziatrice (si apre Spotlight)
- Digitare "terminal"
- Premere Enter
- Si apre la finestra del Terminal

Passaggio 3: Incollare ed eseguire il comando

- Nel Terminal, premere Command + V per incollare il comando copiato
- Premere Enter
- Attendere che l'installazione si completi

5.3 — Perché Questo Concetto È Importante

L'installazione non è un semplice "download". Quando installate Claude Code nel vostro sistema, state dando al software accesso reale al vostro computer. Questo è un punto fondamentale da comprendere:

COSA SUCCEDDE DOPO L'INSTALLAZIONE:

Claude Code ottiene accesso a:

- Cartelle del computer
- Documenti
- File condivisi
- Terminal (per eseguire comandi)
- Capacità di leggere qualsiasi file
- Capacità di modificare qualsiasi file
- Capacità di creare nuovi file

- Capacità di cancellare file (in Bypass Permission)

Questo è esattamente ciò che rende Claude Code così potente — e anche il motivo per cui le modalità di permesso (che vedremo nel Capitolo 17) sono così importanti. Claude Code non è un'applicazione isolata in una sandbox: opera nel vostro ambiente reale.

5.4 — Interpretazione Pratica

Il comando non è immediato. L'autore avverte esplicitamente: *"Come vedete anche il comando non è immediato. Cosa vuol dire? Che ci metterà un po' prima di installarlo, e quindi non preoccupatevi se dopo avere premuto Enter magari avete 3-4 secondi, 10 di awkward silence, perché non ci interessa."*

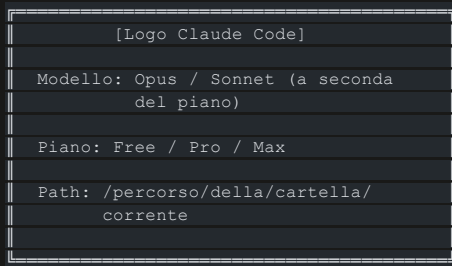
Questo è un dettaglio importante perché molte persone, soprattutto i principianti, premendo Enter e vedendo che non succede nulla immediatamente, pensano di aver sbagliato qualcosa. Non è così. L'installazione richiede tempo perché sta scaricando e configurando diversi componenti nel sistema.

Dopo l'installazione — prima esecuzione:

1. Pulire il Terminal digitando `clear` e premendo Enter
2. Digitare `claude` e premere Enter
3. Comparirà un messaggio di sicurezza: "Quick safety check" — rispondere Sì
4. A questo punto appare il logo di Claude Code (che l'autore descrive come un granchio o una medusa)
5. Vengono mostrate le informazioni di sessione

5.5 — Meccanismo Sottostante

Quando Claude Code viene avviato per la prima volta, vengono visualizzate diverse informazioni nella schermata iniziale:



> [barra del prompt - qui si scrive]

Modello: Indica quale modello di linguaggio è in uso (Opus è il più potente, Sonnet è più veloce ma meno capace per task complesse).

Piano: Conferma il tipo di abbonamento attivo.

Path: Mostra in quale cartella del computer vi trovate. Questo è critico perché Claude Code lavora nella cartella in cui viene avviato. Se lo avviate dalla cartella Desktop, avrà accesso ai file del Desktop. Se lo avviate dalla cartella di un progetto specifico, lavorerà su quel progetto.

5.6 — Errori Comuni

Errore 1: Avviare Claude Code dalla cartella sbagliata

Se avviate Claude Code dal Desktop ma il vostro progetto è in Documents/MioProgetto, Claude Code non vedrà automaticamente i file del progetto. Dovete prima navigare nella cartella corretta usando il comando `cd` (change directory) nel Terminal, oppure aprire il Terminal direttamente dalla cartella del progetto.

Errore 2: Non completare il safety check

Il messaggio "Quick safety check" richiede una conferma esplicita. Se lo ignorate o chiudete il Terminal a questo punto, l'inizializzazione non si completa.

Errore 3: Panico durante l'attesa dell'installazione

L'installazione richiede tempo. Non chiudete il Terminal, non premete Ctrl+C, non fate nulla. Aspettate che il processo si completi da solo.

Errore 4: Usare il comando di installazione sbagliato per il proprio sistema operativo

La documentazione fornisce comandi diversi per Mac, Linux e Windows. Assicuratevi di copiare quello corretto per il vostro sistema operativo.

5.7 — Insight Avanzato

Quando interagite con Claude Code nel Terminal, noterete diversi elementi visivi:

Indicatori di pensiero: Quando Claude sta elaborando una risposta, vedrete parole come "harmonizing", "thinking", "cogitating", "looping", "noodling". Queste sono etichette casuali (e personalizzabili) che indicano semplicemente che il modello sta processando la richiesta. Non hanno significato specifico — servono solo a comunicare "sto pensando, attendi".

Distinzione input/output: Il vostro input (ciò che scrivete) è preceduto da un rettangolino/indicatore visivo. La risposta di Claude non ha questo indicatore. Questo vi permette di distinguere immediatamente chi ha scritto cosa nella conversazione.

Informazioni di sessione nella barra inferiore (Status Line):

```
Context: 14% used | Cost: $0.03 | Tokens: 28K/200K | Session: 5m
```

- Context % used: La percentuale di contesto totale utilizzata. Questa è una delle metriche più importanti da monitorare (approfondita nel Capitolo 20-22)
- Cost: Il costo stimato dell'interazione se si fosse sul piano API. Per chi usa un piano subscription, è solo informativo
- Tokens: Il totale di token utilizzati rispetto al massimo disponibile
- Session duration: Da quanto tempo è attiva la sessione corrente

Nota importante dell'autore: Il numero di token totali e il contesto utilizzato non sono la stessa cosa. La ragione sarà spiegata nei capitoli dedicati al context management, ma per ora è sufficiente sapere che sono due metriche correlate ma distinte.

5.8 — Contesto Applicato: Cos'è un Token?

L'autore fornisce una definizione semplificata ma pratica: *"Per questo corso, dato che non voglio andare troppo nei tecnicismi, pensate ad un token come una parola. In realtà non lo è — sarebbero tre-quattro lettere — ma per semplicità potete pensarlo così."*

Per essere più precisi (senza eccedere nei tecnicismi):

APPROSSIMAZIONE PRATICA:

```
1 token ≈ 3-4 caratteri in inglese  
1 token ≈ 0.75 parole in inglese  
1 token ≈ 0.5-0.6 parole in italiano (l'italiano è più "costoso" in token)
```

Esempi:

```
"Ciao" = 1-2 token  
"Buongiorno, come stai oggi?" = ~8-10 token  
Un prompt di 200 parole ≈ 300-400 token
```

```
Un file di codice di 500 righe ≈ 5.000-15.000 token
```

I token sono l'unità di misura fondamentale perché:

- Il contesto disponibile è misurato in token (es. 200.000 token)
- Il costo del piano API è calcolato per token
- La qualità delle risposte degrada quando il contesto si riempie di token
- Ogni file letto, ogni prompt scritto, ogni risposta generata consuma token

CAPITOLO 6: GLI IDE — VS CODE E ANTIGRAVITY

6.1 — Definizione del Concetto

Un IDE (Integrated Development Environment), in italiano "Ambiente di Sviluppo Integrato", è un software che fornisce un'interfaccia visiva completa per lavorare su progetti. Un IDE tipicamente include: un esploratore di file (per navigare la

struttura del progetto), un editor di testo (per modificare i file), un terminal integrato, e — nel nostro caso — l'integrazione con Claude Code.

6.2 — Spiegazione Espansa

Per utilizzare Claude Code con un'interfaccia visiva (anziché solo dal Terminal), servono questi IDE. Il mercato attualmente si concentra su due opzioni principali.

6.2.1 — VS Code (Visual Studio Code)

Produttore: Microsoft

Download: Cercare "Visual Studio Code Download" su Google, primo risultato

Status: L'IDE storico, consolidato, utilizzatissimo. L'autore lo definisce "l'OG" (Original Gangster — il primo, quello storico)

Interfaccia: Più rigida e meno pulita rispetto ad Antigravity

Struttura dell'interfaccia VS Code:

[Barra superiore - Menu]	
EXPLORER	AREA EDITOR (visualizzazione/modifica file)
src	
public	
index	
cloud	
[Terminal integrato / Claude Code panel]	

L'Explorer (pannello sinistro) è semplicemente una rappresentazione visiva della cartella del progetto. Tutto ciò che vedete nell'Explorer corrisponde esattamente ai file e cartelle presenti fisicamente nel vostro computer. L'autore lo dimostra aprendo la stessa cartella nel Finder del Mac e mostrando che il contenuto è identico.

Come aprire un progetto in VS Code:

1. Avviare VS Code
2. Premere l'icona in alto a sinistra
3. Selezionare "Open Folder"
4. Navigare fino alla cartella del progetto
5. Premere "Open"

Come installare Claude Code in VS Code:

1. Premere l'icona delle estensioni (nella barra laterale sinistra — sembra un gruppo di blocchi/quadrati)
2. Nella barra di ricerca digitare "Claude Code"
3. Premere "Install"
4. Una volta installato, appare un'icona a stella nella barra laterale
5. Premere l'icona stella per aprire il pannello Claude Code

6.2.2 — Antigravity

Produttore: Google

Data di lancio: Circa novembre 2025

Download: Cercare "Antigravity Google Download" su Google

Interfaccia: Più pulita, moderna, con animazioni che seguono il mouse
Preferenza dell'autore: Preferito rispetto a VS Code

Struttura dell'interfaccia AntigraVity:

[Barra superiore - Menu + Impostazioni ⚙️]		
EXPLORER	AREA EDITOR	PANNELLO AGENTE
	(visualizzazione/	(Chat con AI)
📁 src	modifica file)	
📁 .cloud		Modelli disponibili:
📄 index		• Google (Gemini)
📄 CLAUDE		• Claude Code
		• GPT
[Terminal integrato]		

Differenza chiave: AntigraVity ha un pannello agente sulla destra che permette di chattare con diversi modelli AI — non solo Claude Code, ma anche i modelli di Google (Gemini) e GPT. Questo è perché Google, avendo sviluppato AntigraVity, spinge naturalmente i propri modelli rendendoli disponibili di default.

Come installare Claude Code in AntigraVity:

1. Premere l'icona delle estensioni
2. Cercare "Claude Code"
3. Premere "Install"
4. Accesso rapido: Command + Shift + ESC (apre Claude Code direttamente)
5. Alternativa: premere l'icona stella nella barra laterale

6.3 — Perché Questo Concetto È Importante

La scelta dell'IDE determina il vostro workflow quotidiano. Entrambi gli IDE funzionano allo stesso modo con Claude Code, ma ci sono sfumature pratiche:

AntigraVity è preferibile quando:

- Lavorate molto sul frontend (design, interfaccia visiva)
- Volete avere accesso a più modelli AI nella stessa interfaccia
- Preferite un'interfaccia più moderna e pulita
- Volete sfruttare i modelli Google per compiti specifici

VS Code è preferibile quando:

- Avete già familiarità con esso
- Avete estensioni specifiche installate che non esistono su AntigraVity
- Lavorate in un team che usa VS Code come standard

La strategia dell'autore:

L'autore utilizza una divisione specifica del lavoro tra i modelli disponibili:

- Codice backend (logica, server, database) → Claude Code
- Codice frontend (design, interfaccia utente) → Modelli Google in AntigraVity

Questa strategia può essere codificata nel CLAUDE.md in modo che il sistema scelga automaticamente il modello appropriato per il tipo di task.

6.4 — Meccanismo Sottostante: Il Concetto di Workspace/Cartella

Un punto fondamentale che l'autore sottolinea con forza è che la cartella è ciò che conta, non l'IDE:

STESSO PROGETTO, VISUALIZZATO IN MODI DIVERSI:

Cartella fisica sul computer:

```
company-website/
├── src/
│   └── public/
├── index.html
├── CLAUDE.md
├── .claude/
└── package.json
```

Questa STESSA cartella appare identica in:

- VS Code (Explorer panel a sinistra)
- Antigravity (Explorer panel a sinistra)
- Terminal (comando 'ls' per listare i file)

- Finder/File Manager del computer

Se modificate un file tramite Claude Code in VS Code, la modifica è visibile anche in Antigravity se aprite la stessa cartella, e viceversa. L'IDE è solo una "finestra" attraverso la quale guardate e lavorate sulla stessa cartella.

6.5 — Dettaglio Pratico: Il File in Contesto

Quando Claude Code è aperto in un IDE, nella parte superiore del pannello chat viene mostrato il nome del file attualmente visualizzato nell'editor. Per esempio, se state guardando il file `package.json`, vedrete un'indicazione come `package.json` accanto ai vari pulsanti.

L'autore spiega che questo è il file che Claude Code sta attualmente "guardando". Se chiedete *"Che tipo di file stai guardando in questo momento?"*, Claude risponderà con il nome di quel file.

Implicazione pratica: Quando scrivete un prompt che si riferisce a "questo file" o "il file corrente", Claude Code interpreta la richiesta nel contesto del file mostrato. Se volete modificare un file specifico, assicuratevi di averlo aperto nell'editor prima di dare l'istruzione.

6.6 — Errori Comuni

Errore 1: Non accettare le modifiche proposte da Claude Code

Quando Claude Code propone una modifica a un file, questa viene mostrata come una "diff" (differenza tra la versione originale e quella proposta). Se salvate il file senza accettare esplicitamente le modifiche, il file viene duplicato anziché modificato. L'autore ha mostrato questo accidentalmente durante il corso: *"Se poi io ora lo salvassi senza accettare questi cambiamenti, vedete che me l'ha semplicemente duplicato."*

La regola è: sempre accettare o rifiutare esplicitamente le modifiche proposte prima di procedere.

Errore 2: Non configurare il Dangerously Skip Permission

La modalità bypass permission (che verrà approfondita nel Capitolo 19) non è attiva di default. Per abilitarla:

In VS Code:

1. Premere l'icona ingranaggio (⚙️)
2. Selezionare "Settings"
3. Cercare "Claude"

- 4. Abilitare "Allow Dangerously Skip Permission"

In Antigravity:

- 1. Premere la rotellina/ingranaggio in alto a destra (posizione diversa da VS Code)
- 2. Selezionare "Settings"
- 3. Cercare "Claude Code"
- 4. Abilitare "Dangerously Skip Permission"

Nota: la posizione dell'ingranaggio è diversa nei due IDE. In VS Code è nella barra laterale, in Antigravity è in alto a destra.

Errore 3: Confondere le modalità di interazione
In entrambi gli IDE, Claude Code offre diverse modalità nella parte superiore del pannello:

Modalità	Comportamento
Ask before edits	Claude propone le modifiche e chiede approvazione prima di applicarle
Edit automatically	Claude applica le modifiche automaticamente (tranne creazione/cancellazione file)
Plan mode	Claude crea un piano strutturato e chiede approvazione prima di eseguirlo
Bypass permission	Claude fa tutto in autonomia: crea, modifica, cancella file senza chiedere

La confusione tra queste modalità è molto comune e può portare a risultati inattesi (Claude che modifica file senza il vostro consenso, o al contrario Claude che si ferma continuamente a chiedere permessi rallentando il workflow).

6.7 — Insight Avanzato

Indicatore "U" (Updated):
Quando un file viene modificato da Claude Code, nell'Explorer dell'IDE appare una lettera "U" accanto al nome del file. Questa indica che il file è stato aggiornato rispetto alla sua versione precedente. È un indicatore visivo utile per sapere immediatamente quali file sono stati toccati durante una sessione di lavoro.

La portabilità del progetto:
Poiché il progetto vive nella cartella e non nell'IDE, potete:

- Iniziare un progetto in VS Code e continuarlo in Antigravity

- Lavorare dal Terminal e poi visualizzare i risultati nell'IDE
- Condividere la cartella del progetto con un collaboratore che usa un IDE diverso
- Caricare la cartella su GitHub e scaricarla su un altro computer

Tutto funzionerà perché le configurazioni di Claude Code (CLAUDE.md, cartella .claude, regole, skill) sono tutte contenute nella cartella del progetto stessa.

CAPITOLO 7: IL TERMINAL COME INTERFACCIA AVANZATA

7.1 — Definizione del Concetto

Il Terminal è un'interfaccia a riga di comando che permette di interagire con il computer digitando comandi testuali anziché cliccando su bottoni e icone. Nel contesto di Claude Code, il Terminal è l'interfaccia più potente e completa, quella che sblocca tutte le funzionalità avanzate.

7.2 — Spiegazione Espansa

L'autore è molto chiaro su questo punto: nonostante il Terminal possa sembrare intimidatorio, è l'interfaccia raccomandata per chi vuole sfruttare Claude Code al massimo delle sue capacità. Le ragioni sono concrete:

Funzionalità esclusive del Terminal:

1. Agent Teams — La funzionalità multi-agente collaborativo è disponibile solo da Terminal
2. Analisi completa del contesto (/context) — Visione dettagliata di come il contesto è distribuito
3. Configurazione avanzata (/config) — Accesso a tutte le impostazioni
4. Gestione permessi granulare — Allow, Ask, Deny per tool specifici
5. Status Line completa — Barra informativa con contesto, costi, token, durata sessione
6. Navigazione tra modalità con shortcut — Shift+Tab per passare tra le modalità

Come avviare Claude Code dal Terminal:

```
Avvio standard:  
$ claude
```

```
Avvio con bypass permission (YOLO mode):  
$ claude --dangerously-skip-permissions
```

Nota: l'autore usa l'alias "YOLO" (You Only Live Once)

per abbreviare questo comando nel suo setup personale

7.3 — Perché Questo Concetto È Importante

Il Terminal è dove si effettua la transizione da "utente base" a "utente avanzato" di Claude Code. Molte delle operazioni più potenti e produttive — Agent Teams, analisi del contesto, gestione MCP — richiedono il Terminal.

L'autore lo esprime con una frase importante: *"Il motivo per cui vi dico, magari se ce la fate, prendete familiarità piano alla volta con il terminal: sembra difficile all'inizio ma veramente non lo è."*

La parola chiave è "piano alla volta". Non serve imparare tutto il Terminal in un giorno. I comandi fondamentali sono pochissimi:

text

COMANDI TERMINAL ESSENZIALI PER CLAUDE CODE:

```
claude          → Avvia Claude Code
claude --dangerously-skip-permissions → Avvia con bypass permission
clear           → Pulisce la schermata
cd /percorso/cartella → Cambia directory

ls              → Lista i file nella cartella corrente
```

Con questi cinque comandi potete fare il 90% di tutto ciò che serve nel Terminal per lavorare con Claude Code.

7.4 — Meccanismo Sottostante: Navigazione nel Terminal

Quando Claude Code è attivo nel Terminal, avete accesso a due livelli di navigazione:

Livello 1: Comandi Slash

Digitando / seguito da un comando, accedete a funzionalità specifiche di Claude Code:

text

```
/config → Visualizza e modifica la configurazione
/context → Analizza la distribuzione del contesto
/compact → Compatta il contesto per liberare spazio
/init → Inizializza un nuovo progetto (crea CLAUDE.md)
/mcp → Gestisce i Model Context Protocol

/status → Mostra lo stato della sessione corrente
```

Livello 2: Navigazione con tasti

- Shift + Freccia su + Tab: Visualizza le modalità di Claude Code (Accept edits, Plan mode, Bypass permission)
- Freccie su/giù: Navigano tra le opzioni nella vista configurazione
- Tab: Passa tra le sezioni (Config → Usage → Status)
- ESC: Esce dalla vista corrente e torna al prompt

7.5 — Interpretazione Pratica: Il Comando /config

Il comando /config è il primo comando avanzato che l'autore mostra. Quando lo eseguite, vedete tutte le impostazioni interne di Claude Code:

CONTENUTO DI /config:

```
Autocompact: [ON/OFF]
→ Compatta automaticamente il contesto quando si riempie
→ Fondamentale per il context management
```

```
Thinking Mode: [ON/OFF]
→ Permette a Claude di "pensare" in modo estensivo
→ Migliora la qualità delle risposte su task complesse
```

```
Rewind/Checkpoint: [ON/OFF]
→ Salva checkpoint automatici del progetto
→ Permette di tornare a versioni precedenti
→ Se OFF, non potete fare rollback
```

```
Theme: [personalizzabile]
  → Personalizzazione visiva del terminal

Status Line: [ON/OFF]
  → La barra informativa in basso al terminal
  → Mostra contesto, costi, token, durata
```

```
Teammate Mode: [ON/OFF]

  → Abilita la funzionalità Agent Teams
```

7.6 — Approfondimento: La Status Line

La Status Line è la barra informativa che appare nella parte inferiore del Terminal quando Claude Code è attivo. L'autore la considera essenziale e mostra come attivarla.

Come attivare la Status Line:

Metodo 1: Comando diretto

Nel prompt di Claude Code, digitare il comando specifico per la status line (disponibile nella configurazione) e premere Enter.

Metodo 2: Chiedere a Claude Code

Se non riuscite a configurarla manualmente, l'autore suggerisce un approccio pragmatico e potente: fare uno screenshot della Status Line come appare nel video/tutorial, incollarlo nel Terminal, e chiedere a Claude: *"Ehi, per favore fa sì che io abbia sotto al terminal queste cose qui."* Claude Code configurerà tutto automaticamente.

Questa è una perla filosofica dell'autore: *"D'ora in poi la vostra vita sarà sempre alla distanza di un buon prompt da risolvere gran parte dei vostri problemi."*

Cosa mostra la Status Line:

```
14% used | 💰 $0.03 | 🗣️ 28,000/200,000 tokens | ⌚ 5m 32s

📊 Contesto utilizzato (%)
  → La metrica più importante da monitorare
  → Quando si avvicina al 100%, le performance degradano
  → L'autocompact interviene automaticamente prima del 100%

💰 Costo stimato (API pricing)
  → Quanto costerebbe l'interazione nel piano API
  → Solo informativo per utenti con piano subscription
  → Utile per capire il "peso" di ogni operazione

🗣️ Token utilizzati / Token totali
  → Quanti token sono stati consumati nella sessione
  → Il totale dipende dal modello (es. 200K per alcuni, 1M per altri)

⌚ Durata sessione
  → Da quanto tempo la sessione è attiva

  → Utile per gestire il tempo e pianificare i compact
```

7.7 — Errori Comuni

Errore 1: Chiudere il Terminal durante un'operazione

Se Claude Code sta eseguendo un'operazione (installazione, modifica file, ricerca), chiudere il Terminal interrompe bruscamente il processo. Questo può lasciare file in uno stato inconsistente.

Errore 2: Non usare `/config` per verificare le impostazioni

Molti problemi (autocompact non attivo, checkpoint disabilitati, thinking mode spento) derivano da impostazioni non verificate. Eseguire `/config` come prima operazione in ogni nuova installazione è una best practice fondamentale.

Errore 3: Ignorare la Status Line

Senza la Status Line, non avete visibilità sul consumo di contesto. Questo è come guidare senza il cruscotto: non sapete a che velocità andate, quanto carburante avete, o da quanto tempo state guidando. La Status Line è il vostro cruscotto di Claude Code.

7.8 — Insight Avanzato

Il comando `/config` è navigabile:

Quando eseguite `/config`, potete:

- Usare le frecce su/giù per scorrere le opzioni
- Premere Tab per passare alla vista "Usage" (quanto avete utilizzato del modello, es. 28%)
- Premere Tab ancora per passare alla vista "Status"
- Premere ESC per uscire

Questo crea un flusso di navigazione:

```
/config → [Configurazione] --Tab--> [Usage] --Tab--> [Status] --ESC--> [Prompt]
```

Il concetto di Autocompact:

L'Autocompact è una delle funzionalità più importanti di Claude Code e merita una spiegazione dettagliata qui perché si configura tramite `/config`.

L'autore spiega il concetto con un esempio vivido. Immaginate di aver scritto un prompt come questo:

"Ciao, sono Giovanni, ho 30 anni, il mio compleanno è il 27 febbraio, quindi qualche giorno fa, mi piacciono le pentole, vivo a Lussemburgo..."

In questo prompt, "mi piacciono le pentole" è informazione irrilevante che occupa contesto inutilmente. L'Autocompact fa esattamente questo: prende il contesto e ne aumenta la densità informativa, eliminando ciò che non è rilevante.

Il risultato dopo Autocompact sarebbe qualcosa come:

"Giovanni, Lussemburgo, 30 anni, compleanno 27 febbraio"

Stesse informazioni utili, frazione dello spazio. Questo processo avviene automaticamente quando il contesto si avvicina alla soglia critica (circa 33.000 token nel buffer di Autocompact, come verrà spiegato nel Capitolo 22).

CAPITOLO 8: CONFIGURAZIONE, STATUS LINE E COMANDI FONDAMENTALI

8.1 — Definizione del Concetto

La configurazione di Claude Code comprende tutti i parametri, le impostazioni e i comandi che determinano come il sistema si comporta, cosa monitora, e quali funzionalità sono attive. Una configurazione corretta è il prerequisito per un utilizzo efficiente e produttivo.

8.2 — Spiegazione Espansa: Mappa Completa dei Comandi

L'autore introduce i comandi nel corso della guida in modo progressivo. Qui li raccogliamo tutti in una mappa organizzata per funzione:

Comandi di Inizializzazione e Gestione Progetto

```
/init
└─ Funzione: Inizializza un nuovo progetto Claude Code
└─ Cosa fa: Analizza i file presenti nella cartella,
    legge il codice esistente, comprende la struttura,
    e genera un CLAUDE.md strutturato secondo le best practice
└─ Quando usarlo: Sempre all'inizio di un nuovo progetto
└─ Quando è particolarmente utile:
    • Quando importate un progetto da qualcun altro
    • Quando il team vi manda script e risorse
    • Quando volete ricreare il CLAUDE.md dopo modifiche importanti
└─ Nota importante: Se avete impostazioni globali (~/.claude),
    il /init le incorpora automaticamente nel nuovo CLAUDE.md
```

L'autore sottolinea un vantaggio sottile ma potente di `/init`: se esiste un comando che rimanda a "come si crea un CLAUDE.md" per tutti i processi, quel file verrà strutturato in modo identico ogni volta. Questo crea standardizzazione: *"Tutti running quel comando abbiano lo stesso, e questo crea un'efficienza non da ridere."*

Comandi di Context Management

```
/context
└─ Funzione: Mostra un'analisi dettagliata della distribuzione del contesto
└─ Cosa mostra:
    • System prompt (injection di Anthropic) → ~10-12%
    • System tools → %
    • MCP tools → % (può essere molto alto!)
    • Memory files → %
    • Skills → %
    • Custom agents → %
    • Messages (la conversazione) → %
    • Spazio libero → %
└─ Perché è critico: Permette di capire dove il contesto viene consumato
    e identificare sprechi
```

```
/compact
└─ Funzione: Compatta manualmente il contesto della conversazione
└─ Cosa fa: Riscrive l'intera conversazione in formato ad alta densità
    (bullet point, eliminazione ridondanze, compressione)
└─ Esempio di compattazione:
    PRIMA: "Ho chiesto a Claude di importare i tre sub-agenti che sono
    il reviewer, il researcher e il QA. Poi ho chiesto di fare
    un review completo del codice..."
    DOPO: "• Importati 3 sub-agenti: reviewer, researcher, QA
    • Review codice completato"
└─ Quando usarlo: Quando il contesto supera il 60-70%
    o quando notate degradazione delle performance
└─ Nota: L'Autocompact fa la stessa cosa ma automaticamente
    quando si raggiunge la soglia del buffer (~33K token)
```

Comandi di Gestione Sessione

```
clear (nel prompt Claude)
└─ Funzione: Pulisce la conversazione corrente
└─ Nota: Non cancella la memoria o le configurazioni
```

```
clear context
└─ Funzione: Pulisce il contesto della conversazione
└─ Quando usarlo: Quando volete iniziare una nuova conversazione
```

```
mantenendo le configurazioni del progetto
```

Comandi di Gestione MCP e Permessi

```
/mcp
├─ Funzione: Gestisce i Model Context Protocol installati
├─ Cosa mostra: Lista degli MCP attivi e il loro stato
├─ Quando usarlo: Per verificare connessioni, autenticarsi,
  o risolvere problemi con MCP

permissions (nel prompt di configurazione)
├─ Funzione: Gestisce i permessi dei tool
├─ Opzioni:
  • Allow: Il tool è sempre permesso (nessuna richiesta)
  • Ask: Chiede sempre permesso prima di usare il tool
  • Deny: Il tool non viene mai usato
  • Workspace: Permessi specifici per il workspace corrente
├─ Nota critica: I permessi dell'agente principale si replicano
  a TUTTI i sotto-agenti (downstream)
```

8.3 — Perché Questo Concetto È Importante

La configurazione non è un'operazione "una tantum". È un processo continuo di ottimizzazione che impatta direttamente sulla qualità del lavoro e sull'efficienza del sistema. Una configurazione subottimale porta a:

- Spreco di contesto (meno spazio per le informazioni utili)
- Mancanza di visibilità sullo stato del sistema (senza Status Line)
- Impossibilità di tornare a versioni precedenti (senza checkpoint)
- Performance degradate (senza Autocompact o thinking mode)
- Rischi di sicurezza (senza gestione appropriata dei permessi)

8.4 — Interpretazione Pratica: Checklist di Configurazione Iniziale

Ogni volta che installate Claude Code o iniziate un nuovo progetto, questa è la sequenza di configurazione raccomandata:

CHECKLIST DI CONFIGURAZIONE INIZIALE:

- ☐ 1. Avviare Claude Code (`$ claude`)
- ☐ 2. Eseguire `/config` e verificare:
 - ☐ Autocompact: ON
 - ☐ Thinking Mode: ON
 - ☐ Rewind/Checkpoint: ON
- ☐ 3. Attivare la Status Line
- ☐ 4. Eseguire `/init` per generare il `CLAUDE.md`
- ☐ 5. Verificare i permessi con `/permissions`
- ☐ 6. Controllare gli MCP con `/mcp`
- ☐ 7. Eseguire `/context` per avere una baseline del contesto

8.5 — Meccanismo Sottostante: La Funzione Rewind/Checkpoint

Il Rewind/Checkpoint è una funzionalità che salva automaticamente "istantanee" del progetto durante il lavoro. L'autore la spiega così:

"Rewind checkpoint vuol dire che ipotizziamo che eravamo contenti con una roba che abbiamo fatto ma ci siamo fregati e abbiamo scritto una cosa che ha mandato a monte tutto il resto. Per tornare indietro semplicemente diciamo: 'Ehi torna alla versione precedente.' Se fosse OFF non potremmo farlo."

Questa funzionalità è particolarmente importante quando si lavora in `bypass permission`, dove Claude Code ha la libertà di creare e cancellare file autonomamente. Senza checkpoint, un errore potrebbe essere irreversibile (o almeno molto costoso da recuperare).

ESEMPIO DI FLUSSO CON CHECKPOINT:

```
Checkpoint 1: Progetto funzionante ✓
↓
Modifica A: Aggiunta feature X → tutto ok
↓
Checkpoint 2: Progetto con feature X ✓
↓
Modifica B: Tentativo feature Y → ERRORE! Progetto rotto ✨
↓
"Ehi, torna alla versione precedente"
↓
Ripristinato Checkpoint 2: Progetto con feature X ✓
```

8.6 — Errori Comuni

- Errore 1: Non abilitare l'Autocompact**
Senza Autocompact, il contesto si riempie progressivamente e non viene mai liberato. Questo porta a una degradazione graduale delle performance fino al punto in cui Claude Code non riesce più a operare efficacemente. L'Autocompact è la prima impostazione da verificare.
- Errore 2: Disabilitare il Thinking Mode per "velocizzare"**
Il Thinking Mode permette a Claude di ragionare in modo estensivo prima di rispondere. Disabilitarlo rende le risposte più rapide ma significativamente meno accurate, specialmente su task complesse. È come chiedere a qualcuno di risolvere un problema complesso vietandogli di pensare.
- Errore 3: Non monitorare `/context` regolarmente**
Molti utenti non sanno nemmeno che il comando `/context` esiste. Questo li rende "ciechi" rispetto a come il contesto viene distribuito. Come vedremo nei capitoli sugli MCP, un singolo MCP può consumare il 27% del contesto senza che l'utente se ne renda conto.
- Errore 4: Cambiare configurazioni senza capirne l'impatto**
Ogni impostazione in `/config` ha implicazioni a cascata. Per esempio, disabilitare i checkpoint in un progetto dove si usa `bypass permission` è una ricetta per il disastro. Ogni modifica di configurazione dovrebbe essere fatta con consapevolezza delle conseguenze.

8.7 — Insight Avanzato

Il Tab cycling in `/config`:
Il sistema di navigazione con Tab in `/config` rivela tre livelli di informazione che servono a scopi diversi:

```
Tab 1: CONFIG (Configurazione)
→ Per impostare il comportamento del sistema
→ Usare all'inizio di un progetto o quando serve cambiare qualcosa

Tab 2: USAGE (Utilizzo)
→ Per monitorare quanto del modello avete usato
→ Mostra la percentuale di utilizzo rispetto al limite del piano
→ Es: "Ho utilizzato il 28% del mio modello questo mese"
→ Utile per gestire i limiti del piano subscription

Tab 3: STATUS (Stato)
→ Per verificare lo stato corrente della sessione

→ Mostra informazioni tecniche sulla connessione e il funzionamento
```

Questi tre livelli corrispondono a tre domande diverse:

- Config: "Come è impostato il mio sistema?" (passato/futuro)
- Usage: "Quanto ho usato il mio sistema?" (storico)
- Status: "Come sta andando il mio sistema adesso?" (presente)

Personalizzazione della Status Line:

La Status Line non è un'entità fissa. L'autore menziona che contiene "informazioni aggiuntive che utilizzo per controllare meglio il mio terminal" e che è possibile personalizzarla. Se volete replicare la configurazione dell'autore, il metodo più efficace è fare uno screenshot della Status Line mostrata nel video, incollarlo in Claude Code, e chiedere di replicarla. Claude Code configurerà automaticamente il Terminal per mostrare le stesse informazioni.

PARTE 4 — COSTRUIRE PROGETTI CON CLAUDE CODE

CAPITOLO 13: TRE METODI PER COSTRUIRE UN SITO INTERNET

13.1 — Definizione del Concetto

Esistono tre modalità prevalenti con cui le persone oggi costruiscono siti internet utilizzando Claude Code. Ogni modalità ha vantaggi, svantaggi e contesti d'uso specifici. La comprensione di tutte e tre è necessaria per scegliere l'approccio ottimale in ogni situazione.

13.2 — Spiegazione Espansa

Prima di descrivere i tre metodi, l'autore introduce due risorse fondamentali per trovare ispirazione e riferimenti visivi per i siti da costruire:

Risorse per Riferimenti Visivi

Risorsa 1: Godly Website (godly.website)

- Contiene siti web che hanno vinto premi (award-winning websites)
- Eccellenti a livello di design
- L'autore nota: *"Senza dubbio a livello di design sono belli. Poi sulla conversion, quella è un'altra roba."* — Questo significa che un sito premiato per il design non è necessariamente ottimizzato per convertire visitatori in clienti
- Utile per copiare/replicare siti di alta qualità estetica

Risorsa 2: Dribbble (dribbble.com)

- Contiene design di ogni tipo: animazioni, branding, UI/UX
- Permette di trovare componenti specifici (bottoni, form, layout)
- Fornisce codice e specifiche di design
- Molto utile per trovare singoli elementi da integrare nei propri progetti

Metodo 1: Prompt Dettato a Voce + Editing Manuale

text

FLUSSO DEL METODO 1:

[Voce dell'utente] → [Speech-to-Text] → [Prompt scritto] → [Claude Code] → [Output]

↓

[Editing manuale]

↓

[Feedback vocale]

↓

[Iterazione]

Come funziona:

L'utente detta il prompt a voce utilizzando uno strumento di Speech-to-Text (STT). Il testo trascritto viene inviato a Claude Code, che genera l'output. L'utente poi fornisce feedback vocale o effettua editing manuale.

Perché è efficace:

L'autore spiega il principio di throughput: *"Se ci pensate, nel momento in cui una persona parla va molto più veloce a scrivere e a dettare parole a un throughput per hour molto maggiore di quando una persona scrive."*

Per quantificare:

text

VELOCITÀ DI OUTPUT PER MODALITÀ:

Scrittura a tastiera: ~40-60 parole/minuto (utente medio)

Dettatura vocale: ~130-180 parole/minuto (parlato naturale)

Rapporto: ~3x più veloce con la voce

In termini di prompt:

Un prompt di 200 parole:

- Scritto: ~4 minuti
- Dettato: ~1.5 minuti
- Risparmio: ~2.5 minuti per prompt

Su 20 prompt in una sessione:

- Risparmio totale: ~50 minuti

Quando è particolarmente utile:

- Prompt lunghi e descrittivi
- Sessioni di brainstorming con l'AI
- Quando si vuole "pensare ad alta voce" e lasciare che Claude interpreti
- Per utenti che scrivono lentamente a tastiera

Strumenti di STT utilizzabili:

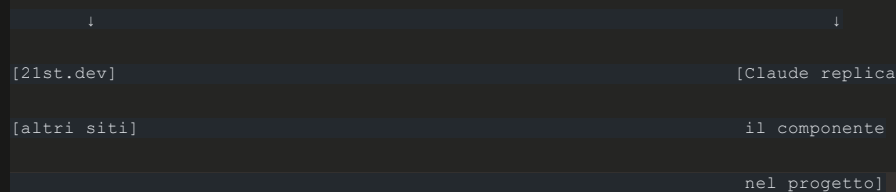
- Su Mac: tasto Function (Fn) attiva la dettatura nativa
- La trascrizione include punteggiatura, virgolette e formattazione
- L'autore lo utilizza regolarmente nel suo workflow quotidiano

Metodo 2: Utilizzo di Componenti Preconfezionati

text

FLUSSO DEL METODO 2:

[Sito di componenti] → [Seleziona componente] → [Copy Prompt] → [Incolla in Claude Code]



Come funziona:

Esistono siti web che offrono componenti UI (bottoni, navbar, sezioni, animazioni, interfacce 3D) già pronti. L'utente seleziona il componente desiderato, copia il prompt associato, e lo incolla in Claude Code che lo replica nel progetto.

Risorsa principale: 21st.dev

L'autore presenta questo sito come una risorsa preziosa che *"non molti conoscono"*. Caratteristiche:

- Libreria di componenti visivi con il pulsante "Copy Prompt"
- Include componenti interattivi e 3D (es. interfacce che seguono il mouse)
- Ogni componente è sostanzialmente un insieme di HTML, CSS, JavaScript
- Claude Code è *"estremamente bravo quando si tratta di codice"* — quindi la replica è fedele

Perché funziona bene:

I componenti web sono codice (HTML, CSS, JavaScript). Claude Code eccelle nel lavorare con il codice perché:

- Il codice è strutturato e non ambiguo
- Ogni componente ha una specifica tecnica chiara
- La replica di codice è deterministica (a differenza della generazione creativa di testo)

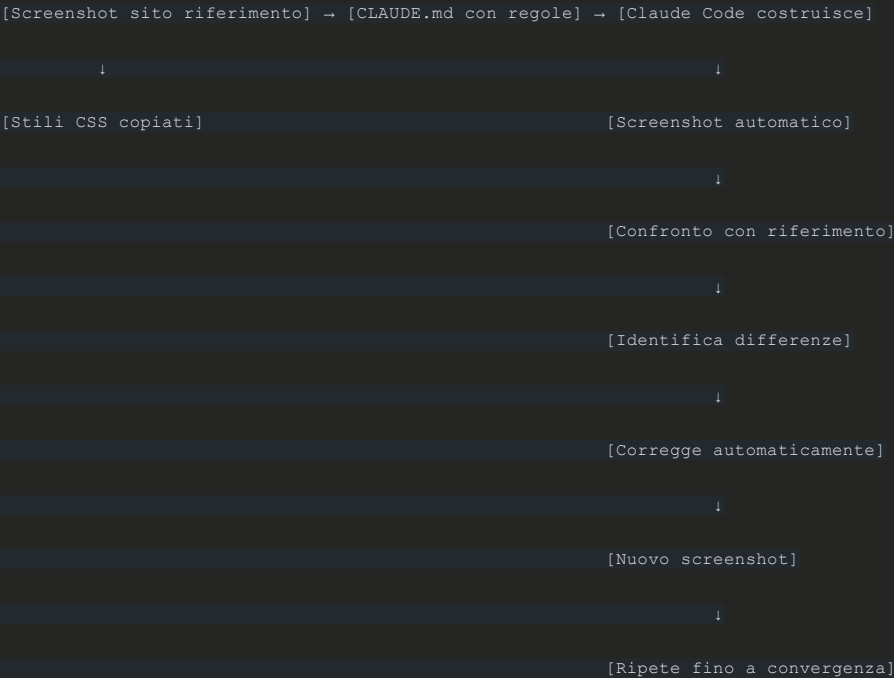
Quando è utile:

- Quando volete un elemento specifico (un'animazione, un layout, un effetto)
- Quando trovate un componente che vi piace e volete integrarlo nel vostro sito
- Per assemblare rapidamente un sito combinando componenti diversi
- Richiede registrazione sul sito dei componenti

Metodo 3: Design di Riferimento + Screenshot Loop ★ (Raccomandato)

text

FLUSSO DEL METODO 3:



Questo è il metodo che l'autore *"generalmente raccomanda"* ed è il più sofisticato dei tre. La ragione della raccomandazione è legata a un concetto fondamentale che merita un capitolo dedicato: il ciclo Task-Do-Verify (Capitolo 14).

13.3 — Perché Questo Concetto È Importante

La scelta del metodo non è arbitraria. Dipende da:

Fattore	Metodo 1 (Voce)	Metodo 2 (Componenti)	Metodo 3 (Screenshot)
---------	-----------------	-----------------------	-----------------------

Velocità iniziale	★★★	★★★	★★
Accuratezza finale	★★	★★	★★★
Fedeltà al design	★	★★	★★★
Autonomia di Claude	Bassa	Media	Alta
Iterazioni manuali	Molte	Moderate	Poche
Adatto per siti complessi	No	Parziale	Sì

Il Metodo 3 è raccomandato perché introduce il principio di auto-verifica, che è il concetto più importante per utilizzare Claude Code in modo professionale.

13.4 — Interpretazione Pratica: Come Eseguire il Metodo 3

L'autore mostra il processo completo passo dopo passo:

Step 1: Catturare lo screenshot full-page del sito di riferimento

Non si usa un normale screenshot (che cattura solo la parte visibile). Si usa una funzionalità specifica del browser:

text

PROCESSO PER FULL-PAGE SCREENSHOT:

1. Aprire il sito di riferimento nel browser
2. Tasto destro → "Inspect" (apre i DevTools)
3. Assicurarsi che la dimensione sia 1920x1080:
→ Nel pannello DevTools, cercare le dimensioni

→ Cambiare a 1920x1080 se necessario

4. Premere Command + Shift + P (apre il command menu dei DevTools)

5. Digitare "screenshot"

6. Selezionare "Capture full size screenshot"

7. L'immagine viene scaricata automaticamente

L'autore ha scoperto questo metodo dopo *"aver fatto 8 miliardi di siti"* — il che implica che molti developer non conoscono questa funzionalità e fanno screenshot manuali sezione per sezione, il che è enormemente meno efficiente.

Vantaggio aggiuntivo del full-page screenshot:

Oltre ad avere tutto il sito in un'unica immagine, permette di identificare le lazy images — immagini che non appaiono subito perché si caricano solo quando l'utente scorre fino alla loro posizione. Nel full-page screenshot queste appaiono come placeholder o spazi vuoti, il che è un'informazione diagnostica utile.

Step 2: Ridimensionare l'immagine

Il full-page screenshot di un sito complesso può pesare 10-15 MB. Inserire un'immagine di questa dimensione nel contesto di Claude Code causerebbe problemi. L'autore lo dice chiaramente: *"Questo sito è troppo grosso — se lo mettessimo nell'LLM, questa immagine è 12.5 mega, esploderebbe tutto."*

text

PROCESSO DI RIDIMENSIONAMENTO:

1. Andare su iloveimg.com (o servizio equivalente)

2. Caricare l'immagine

3. Selezionare "Resize" al 50%

(NON 75% — l'autore specifica: "non farei il 75% perché non ha molto senso")

4. Scaricare l'immagine ridimensionata

5. Rinominarla in qualcosa di significativo (es. "image-reference.png")

6. Posizionarla nella cartella del progetto

Step 3: Copiare gli stili CSS dal sito di riferimento

Questo passaggio riduce enormemente lo spazio di manovra per errori da parte di Claude Code:

text

PROCESSO PER COPIARE GLI STILI:

1. Sul sito di riferimento, tasto destro → Inspect

2. Selezionare l'elemento <body> nell'albero DOM

3. Tasto destro sull'elemento body

4. Selezionare "Copy" → "Copy styles"

5. Questi stili verranno incollati nel prompt a Claude Code

Perché copiare gli stili è cruciale:

L'autore spiega: *"Non vogliamo che l'LLM abbia molto spazio di manovra per sbagliarsi."* Gli stili CSS contengono informazioni precise su:

- Font utilizzati (famiglia, dimensione, peso)
- Colori esatti (codici esadecimali)
- Spaziature (margini, padding)
- Layout (flexbox, grid)
- Ombre, bordi, effetti

Senza questi stili, Claude Code dovrebbe "indovinare" tutti questi parametri dall'immagine, con un margine di errore molto alto. Con gli stili, ha informazioni precise per gran parte delle decisioni estetiche.

Step 4: Formulare il prompt e avviare

Il prompt combina tre elementi:

1. Il CLAUDE.md (già presente nel progetto con le regole del ciclo di verifica)
2. L'immagine di riferimento (posizionata nella cartella del progetto)
3. Gli stili CSS copiati dal sito originale

text

ESEMPIO DI PROMPT:

"Utilizzando il CLAUDE.md, per favore costruiscimi un sito.

Per farlo ti ho messo un'immagine di riferimento nella cartella.

Qua sotto ti incollerò lo stile che c'è all'interno del sito:

[INCOLLARE QUI GLI STILI CSS COPIATI]"

Step 5: Claude Code esegue il ciclo iterativo automaticamente

Grazie alle istruzioni nel CLAUDE.md (che contengono il workflow di verifica tramite screenshot), Claude Code:

1. Costruisce la prima versione del sito
2. Fa uno screenshot del risultato
3. Confronta con l'immagine di riferimento
4. Identifica le differenze (spaziature, dimensioni, colori)
5. Corregge le differenze
6. Fa un nuovo screenshot
7. Ripete fino a quando le differenze sono minime

L'autore osserva il processo in tempo reale: *"Vedete che ora sta facendo questo — prende gli screenshot, naviga, prende, capisce, fa briga. Io non sto toccando letteralmente nulla. L'AI sta già cercando di ripensare a cosa deve fare per replicare questo sito."*

13.5 — Meccanismo Sottostante: Perché il Metodo 3 è Superiore

La superiorità del Metodo 3 si basa su un principio che l'autore sviluppa nel capitolo successivo (il ciclo Task-Do-Verify), ma che anticipiamo qui nella sua essenza:

I Metodi 1 e 2 sono "open loop" (ciclo aperto):

text

Prompt → Esecuzione → Output → [L'UTENTE deve verificare] → Feedback → ...

L'utente è responsabile della verifica. Se l'utente si distrae o non nota un errore, l'errore persiste.

Il Metodo 3 è "closed loop" (ciclo chiuso):

text

Prompt → Esecuzione → Verifica AUTOMATICA → Correzione AUTOMATICA → ...

Claude Code è responsabile della verifica. Il ciclo continua autonomamente fino a convergenza.

13.6 — Errori Comuni

Errore 1: Non ridimensionare l'immagine di riferimento

Un'immagine di 12+ MB inserita nel contesto di Claude Code può causare problemi di performance o superare i limiti. Ridimensionare sempre al 50%.

Errore 2: Non copiare gli stili CSS

Dare solo l'immagine di riferimento senza gli stili CSS è come chiedere a qualcuno di replicare un dipinto dandogli solo una foto sfocata. Gli stili CSS sono le "misure esatte" che eliminano l'ambiguità.

Errore 3: Aspettarsi immagini identiche

Claude Code non può replicare le immagini del sito originale se non ha accesso agli asset originali. Il risultato avrà placeholder al posto delle immagini. L'autore spiega: *"Ovviamente qui abbiamo messo dei placeholder, perché le immagini non erano caricate all'interno del sito — sono le cosiddette lazy images."*

Errore 4: Usare il Metodo 1 o 2 per siti complessi

Per siti con più sezioni, animazioni complesse e design sofisticato, il Metodo 3 è significativamente superiore perché il ciclo di auto-verifica mantiene la qualità su tutta la lunghezza del sito.

Errore 5: Non avere il ciclo di verifica nel CLAUDE.md

Se il CLAUDE.md non contiene istruzioni per il confronto screenshot, Claude Code costruirà il sito ma non lo verificherà. Il ciclo di verifica deve essere esplicitamente codificato nelle regole del progetto.

13.7 — Insight Avanzato

Il Metodo 3 non è limitato ai siti web.

L'autore fa un'osservazione cruciale: *"Se noi zoomiamo al di fuori del sito internet, la capacità di avere Claude che valida il suo input è una capacità che utilizzeremo anche dopo, quando andremo a fare una web app."*

Il principio di auto-verifica si applica a qualsiasi progetto:

- Siti web: Screenshot loop con confronto visivo
- Applicazioni: Test automatici con verifica funzionale

- Codice backend: Test unitari e di integrazione
- API: Chiamate di test con verifica delle risposte
- Automazioni: Run di prova con verifica degli output

Il Metodo 3, nella sua essenza, non è "un modo per fare siti" — è un paradigma di lavoro che dice: dai a Claude Code la capacità di verificare i propri risultati e iterare autonomamente.

CAPITOLO 14: IL CICLO TASK-DO-VERIFY

14.1 — Definizione del Concetto

Il ciclo Task-Do-Verify è il framework operativo fondamentale per utilizzare Claude Code in modo efficace. Consiste nel dare a Claude Code non solo un'attività da eseguire (Task → Do), ma anche la capacità di verificare autonomamente i propri risultati (Verify) e correggere gli errori prima di presentare l'output finale all'utente.

14.2 — Spiegazione Espansa

L'autore introduce questo concetto contrapponendo due modalità di utilizzo di Claude Code:

Modalità Comune (Inefficace): Task → Do

text

FLUSSO SENZA VERIFICA:

Utente: "Fai X"

Claude: [fa X]

Utente: [verifica] → "Questo non va bene, correggi Y"

Claude: [corregge Y]

Utente: [verifica] → "Ora Z non funziona"

Claude: [corregge Z]

... loop lungo e frustrante ...

L'autore la descrive così: *"Molte persone utilizzano un'AI per dire 'fai qualcosa', danno una task, l'AI la fa, poi danno un feedback, e via dicendo. In realtà questo è utilizzare Claude Code non al meglio dei modi, perché manca una parte importante."*

Modalità Corretta (Efficace): Task → Do → Verify → Fix → Loop

text

FLUSSO CON VERIFICA:

Utente: "Fai X"

Claude: [fa X]

Claude: [verifica il risultato]

Claude: [identifica problemi]

Claude: [corregge autonomamente]

Claude: [verifica di nuovo]

Claude: [se ok → presenta risultato finale]

Claude: [se non ok → corregge di nuovo → loop]

...

Utente: [riceve un risultato già verificato e corretto]

La differenza fondamentale è che Claude Code diventa responsabile della verifica, non l'utente. L'utente riceve un output che è già stato iterato e migliorato internamente.

14.3 — Perché Questo Concetto È Importante

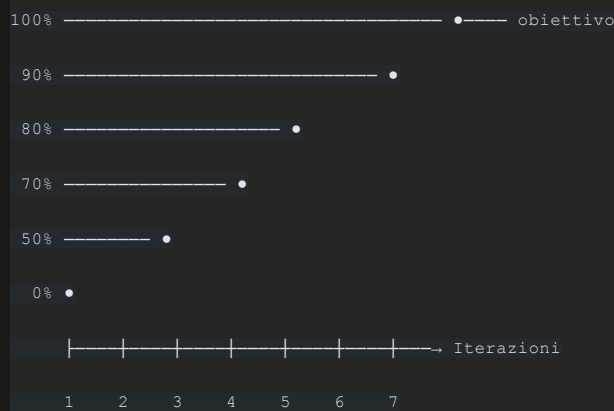
L'autore introduce una visualizzazione che è una delle intuizioni più preziose della guida: la curva di completamento delle task con l'AI.

text

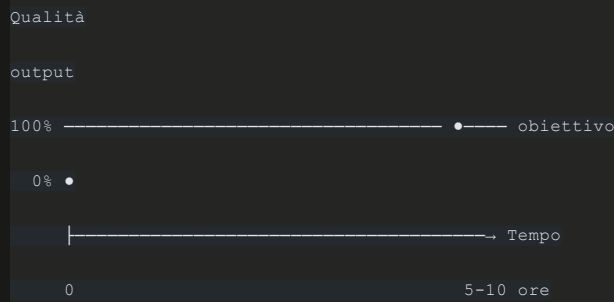
CURVA DI COMPLETAMENTO AI:

Qualità

output



CURVA DI COMPLETAMENTO UMANO:



CONFRONTO:

- L'AI fa 0-50% in una singola iterazione (3-10 minuti)
- L'AI fa 50-80% in 2-3 iterazioni aggiuntive
- L'AI fa 80-100% con qualche iterazione in più
- Tempo totale AI: 10-30 minuti
- L'umano fa 0-100% in un'unica operazione
- Tempo totale umano: 5-10 ore (per un web designer)

Il vantaggio dell'AI non è l'accuratezza — è la velocità.

L'autore lo dice esplicitamente: *"Il vantaggio dell'AI è speed, è la velocità alla quale questa intelligenza riesce a fare le cose. Non è tanto l'accuratezza, attenzione — è la velocità di esecuzione."*

Questo è un punto che molti fraintendono. L'AI non produce risultati perfetti al primo tentativo. Ma produce risultati "abbastanza buoni" in minuti anziché ore, e poi può iterare rapidamente per migliorarli.

14.4 — Interpretazione Pratica: Implementare il Ciclo nel CLAUDE.md

Per attivare il ciclo Task-Do-Verify, le istruzioni devono essere codificate nel CLAUDE.md o nelle rules del progetto. L'autore mostra che nel suo CLAUDE.md per siti web è presente:

text

ESEMPIO DI REGOLA DI VERIFICA NEL CLAUDE.MD:

Workflow di Verifica

Dopo ogni implementazione:

1. Confronta il tuo screenshot con l'immagine di riferimento
2. Confronta le differenze in spaziatura, dimensioni, colori

```
3. Correggi le differenze trovate
```

```
4. Rifai lo screenshot
```

```
5. Ripeti fino a quando le differenze sono minime
```

Per applicazioni (non siti web), il ciclo potrebbe essere:

```
text
```

```
ESEMPIO DI REGOLA DI VERIFICA PER APPLICAZIONI:
```

```
## Workflow di Verifica
```

```
Dopo ogni implementazione:
```

```
1. Esegui il codice per verificare che compili senza errori
```

```
2. Esegui i test automatici
```

```
3. Se un test fallisce, identifica la causa e correggi
```

```
4. Ri-esegui i test
```

```
5. Ripeti fino a quando tutti i test passano
```

```
6. Presenta il risultato finale con il riepilogo dei test
```

14.5 — Meccanismo Sottostante: Perché la Verifica Automatica è Superiore

La verifica automatica è superiore alla verifica manuale per tre ragioni strutturali:

Ragione 1: Velocità

Claude Code può fare uno screenshot, analizzarlo, identificare differenze e correggerle in secondi. Un umano impiegherebbe minuti per la stessa operazione (aprire il browser, confrontare, scrivere il feedback, inviarlo).

Ragione 2: Continuità

Tra un'iterazione e l'altra, Claude Code non ha bisogno di attendere l'input dell'utente. Il ciclo è continuo e ininterrotto. L'autore spiega: *"Non ha bisogno di noi che diamo un invio e quindi premiamo OK o facciamo mille altre cose, ma può semplicemente iterare in maniera successiva, che è molto più velocemente di quello che potremmo fare noi."*

Ragione 3: Consistenza

Claude Code applica gli stessi criteri di verifica ogni volta. Un umano potrebbe notare un problema alla prima iterazione ma non alla seconda (perché è stanco, distratto, o ha dimenticato il criterio).

14.6 — Errori Comuni

Errore 1: Non codificare il ciclo di verifica nelle istruzioni

Se non scrivete esplicitamente nel CLAUDE.md che Claude deve verificare i propri risultati, non lo farà. Claude Code segue le istruzioni — se la verifica non è nelle istruzioni, non avverrà.

Errore 2: Aspettarsi perfezione alla prima iterazione

La curva di completamento mostra che la prima iterazione porta tipicamente al 50-70%. Le iterazioni successive avvicinano al 100%. Pretendere perfezione al primo tentativo è irrealistico e causa frustrazione inutile.

Errore 3: Interrompere il ciclo troppo presto

Se Claude Code sta iterando e l'utente lo interrompe al secondo ciclo, il risultato sarà al 70-80%. Lasciare che il ciclo completi tutte le iterazioni necessarie è la strategia corretta, anche se significa attendere qualche minuto in più.

Errore 4: Non fornire criteri di verifica specifici

"Verifica che sia bello" non è un criterio verificabile. "Confronta con l'immagine di riferimento e correggi le differenze in spaziatura superiori a 5px" è un criterio verificabile.

14.7 — Insight Avanzato

Il ciclo di verifica come principio universale:

L'autore anticipa che questo principio si estende ben oltre i siti web. Nella costruzione dell'app Trello (Capitolo 16), aggiunge un prompt on-the-go durante la costruzione:

"Per favore assicurati che ogni volta che tu implementi qualcosa, tu faccia dei check in modo tale da verificare che quello che stai implementando funzioni, e nel momento in cui io veda il risultato finale, tutto questo sia funzionante."

Nota che l'autore commenta onestamente la qualità di questo prompt: *"Prompt alto livello — non abbiamo specificato come dovrebbe fare i check, che è la cosa che aggiungerà valore."* Questo suggerisce che un prompt più specifico (che definisca esattamente come fare i check) produrrebbe risultati ancora migliori.

La lezione è: il ciclo di verifica è potente, ma la qualità delle istruzioni di verifica determina quanto potente sarà il ciclo.

CAPITOLO 15: IL METODO SCREENSHOT LOOP IN DETTAGLIO

15.1 — Definizione del Concetto

Il metodo Screenshot Loop è l'implementazione pratica del ciclo Task-Do-Verify applicato alla costruzione di siti web e interfacce grafiche. Claude Code costruisce una versione del sito, fa uno screenshot, lo confronta con un'immagine di riferimento, identifica le differenze e le corregge, ripetendo il ciclo fino a raggiungere un livello di fedeltà accettabile.

15.2 — Spiegazione Espansa: Il Processo Completo End-to-End

L'autore dimostra l'intero processo dal vivo, replicando il sito "Hyper" trovato su Godly Website. Qui documentiamo ogni passaggio in dettaglio.

Fase di Preparazione

Step 1: Selezionare il sito di riferimento

- L'autore sceglie un sito chiamato "Hyper" da Godly Website
- L'obiettivo è ricostruirlo usando Claude Code

Step 2: Catturare il full-page screenshot

text

1. Aprire il sito in Chrome
2. Tasto destro → Inspect
3. Impostare dimensioni: 1920x1080
4. Command + Shift + P → "screenshot" → "Capture full size screenshot"
5. Il file viene scaricato automaticamente

Step 3: Ridimensionare l'immagine

text

1. Andare su iloveimg.com
2. Caricare lo screenshot
3. Resize al 50% (NON 75%)
4. Scaricare
5. Rinominare in "image-reference.png"
6. Posizionare nella cartella del progetto

Step 4: Copiare gli stili CSS

text

1. Sul sito originale: tasto destro → Inspect
2. Selezionare <body> nell'albero DOM
3. Tasto destro → Copy → Copy styles
4. Tenere negli appunti per il prompt

Fase di Esecuzione

Step 5: Formulare il prompt

Combinare tre elementi:

- Riferimento al CLAUDE.md (che contiene le regole del screenshot loop)
- Riferimento all'immagine di riferimento nella cartella
- Gli stili CSS copiati incollati direttamente nel prompt

Step 6: Scegliere la modalità

L'autore sceglie Bypass Permission anziché Plan Mode. La ragione:

"In questo caso, siccome il piano è già altamente dettagliato — perché ho un image reference e ho tutti i dettagli di stile — se anche salto [il planning] va benissimo."

La logica è: se le informazioni nel prompt sono sufficientemente dettagliate (immagine + stili + CLAUDE.md), il piano è implicito nei materiali forniti. La pianificazione formale è meno necessaria.

Avvertenza dell'autore: Questa scelta è basata sull'esperienza. Per chi inizia, il Plan Mode è sempre raccomandato.

Step 7: Claude Code esegue il ciclo autonomamente

text

CICLO OSSERVATO DALL'AUTORE:

Iterazione 1 ("Round 1"):

- └─ Claude costruisce la prima versione del sito
- └─ Fa il primo screenshot
- └─ Confronta con l'immagine di riferimento
- └─ Identifica le differenze
- └─ Corregge (font, dimensioni, spaziature, colori)

Iterazione 2 ("Round 2"):

- └─ Fa un nuovo screenshot
- └─ Confronta di nuovo
- └─ Identifica differenze residue
- └─ Corregge

[Continua fino a convergenza o fino a quando Claude

dichiara di aver raggiunto il massimo livello di fedeltà possibile]

L'autore osserva: *"Io non sto toccando letteralmente nulla. L'AI sta già cercando di ripensare a cosa deve fare per replicare questo sito."*

Fase di Risultato

Step 8: Valutazione del risultato

Il risultato prodotto dopo il ciclo di screenshot loop presenta:

-  Struttura generale corretta
-  Bottoni corretti ("Solution Personalized Aviation" con i due bottoni)
-  Sezione con quattro componenti correttamente replicata
-  Sezione "Flexible Services" con tre screenshot
-  Immagine principale in un rettangolo anziché a schermo intero
-  Placeholder al posto delle immagini originali (lazy images)
-  Font potenzialmente diverso (se non installato nel sistema)

Perché mancano le immagini:

Claude Code ha identificato che le immagini del sito originale sono "lazy" (si caricano on-demand) e non erano incluse nello screenshot. Ha quindi inserito placeholder. Per risolvere questo serve accesso agli asset originali del sito.

Perché il font potrebbe essere diverso:

Claude Code segnala: *"Il font vertical now display potrebbe non essere installato sul sistema."* I font custom devono essere disponibili nel sistema per essere replicati correttamente.

15.3 — Perché Questo Concetto È Importante

Il Screenshot Loop è la dimostrazione pratica più tangibile di cosa significhi "dare a Claude Code la capacità di auto-verificarsi". L'autore lo presenta come il *"primo passo per utilizzare Cloud come si deve e per ampliare la vostra produttività"*.

La tempistica reale osservata dall'autore è significativa. L'intero processo — dalla cattura dello screenshot al risultato finale con cicli iterativi — è avvenuto nell'ordine di minuti (l'autore verifica l'ora: "sono le 15:24" e il processo si completa in pochi minuti).

15.4 — Aspettative Realistiche

L'autore dedica un passaggio importante alle aspettative, che è una delle parti più oneste e preziose della guida:

"Ci aspettavamo questa cosa — non è una sorpresa che il sito non venga one shot. Come non è una sorpresa che quando vi trovate un video dove qualcuno vi dice 'ah one shot, poi lo provate' e non ci viene mai."

Il motivo principale per cui il sito non è perfetto:

- Non sono state date direttive specifiche di styling e design
- Non sono stati forniti gli asset originali (immagini, font)
- L'integrazione end-to-end (backend, autenticazione) era specifica nel prompt → ha funzionato molto bene
- Lo styling visivo era generico nel prompt → ha funzionato in modo accettabile ma non perfetto

La lezione fondamentale: L'accuratezza dell'output è direttamente proporzionale alla specificità dell'input. Le aree dove siete specifici saranno accurate. Le aree dove siete vaghi saranno approssimative.

15.5 — Errori Comuni

Errore 1: Screenshot singoli anziché full-page

Fare screenshot sezione per sezione e poi assemblarli è enormemente meno efficiente. Il metodo Command + Shift + P → full size screenshot cattura tutto in un'unica operazione.

Errore 2: Immagini troppo pesanti

Mai inserire immagini superiori a 5-6 MB nel contesto. Ridimensionare sempre al 50%. L'autore ha specificato che l'immagine originale di 12.5 MB avrebbe fatto "esplodere tutto".

Errore 3: Non copiare gli stili CSS

Dare solo l'immagine è insufficiente. Gli stili CSS danno a Claude Code le "coordinate esatte" del design: colori, font, spaziature. Senza di essi, Claude deve indovinare — e indovinare significa errare.

Errore 4: Aspettarsi le stesse immagini del sito originale

Claude Code non ha accesso alle immagini del sito originale. Può solo replicare la struttura e inserire placeholder. Le immagini devono essere sostituite manualmente o tramite un prompt successivo che indica dove trovare gli asset.

CAPITOLO 16: COSTRUIRE UN'APPLICAZIONE COMPLESSA — IL CLONE DI TRELLO

16.1 — Definizione del Concetto

La costruzione di un'applicazione complessa con Claude Code è il processo di creazione di un software funzionante end-to-end — con autenticazione utenti, database, interfaccia grafica, e integrazione con servizi di pagamento — utilizzando Claude Code come strumento principale. Questo capitolo documenta la costruzione pratica di un clone di Trello (project management board) con login utenti, backend Supabase e pagamenti Stripe.

16.2 — Spiegazione Espansa: La Struttura del Progetto

L'applicazione costruita dall'autore ha queste componenti:

text

ARCHITETTURA DELL'APPLICAZIONE TRELLO CLONE:



	• Campo email	
	• Campo nome	
	• Login diretto (senza email confirmation)	
	BACKEND	
	Supabase	
	• Tabella utenti (nome, email)	
	• Tabella board/progetti	
	• Tabella card (titolo, posizione, labels)	
	• Utenti attivi/inattivi	
	• Storico pagamenti	
	PAGAMENTI	
	Stripe	
	• Piano Free + Pro	
	• Abbonamento mensile (\$9/mese)	
	• Form di pagamento integrato	
	• "Pay and Subscribe"	

16.3 — Il Processo di Costruzione in Due Stadi

L'autore adotta una strategia deliberata a due stadi:

Stadio 1: App funzionante SENZA Stripe

text

Obiettivo: Verificare che login, database e interfaccia funzionino

Perché: Isolare i potenziali problemi

Se funziona → procedi allo Stadio 2

Se non funziona → debugga senza la complessità aggiuntiva di Stripe

Stadio 2: Integrazione Stripe

text

Obiettivo: Aggiungere i pagamenti all'app già funzionante

Perché: Aggiungere complessità su una base solida

Se funziona → app completa ✓

Se non funziona → il problema è sicuramente nell'integrazione Stripe

Questa strategia di *divide et impera* è una best practice fondamentale: non cercate di costruire tutto simultaneamente. Costruite a strati, verificando ogni strato prima di aggiungere il successivo.

16.4 — Il Prompt Iniziale e la Plan Mode

L'autore costruisce un prompt strutturato che copre tutti gli aspetti dell'applicazione. Analizzandolo nel dettaglio:

text

STRUTTURA DEL PROMPT:

1. CONTESTO VISIVO:

"Interfaccia simile a Trello come nell'immagine.png"

→ Immagine di riferimento scaricata da Google

2. FUNZIONALITÀ RICHIESTE:

- Card draggabili con cambio di status
- Ricerca online su cos'è Trello (per le funzionalità)

3. AUTENTICAZIONE:

- Form di login con email e nome
- Dati salvati su Supabase

4. BACKEND:

- Supabase come database
- Tracking utenti attivi/inattivi
- Storico pagamenti

5. PAGAMENTI:

- Integrazione Stripe (da predisporre ma non implementare subito)
- Database predisposto per tracking pagamenti

6. SICUREZZA:

- File .env per le chiavi API
- Chiavi salvate in locale
- Non condividere le chiavi quando si condivide il progetto

7. API KEYS:

- Chiave Stripe (fornita direttamente nel prompt)
- Chiave Supabase (fornita direttamente nel prompt)

Osservazione dell'autore sulla qualità del prompt:

"A livello teorico, uno dovrebbe mettersi lì e fare un prompt un po' migliore. Ma già con questo riusciremo ad ottenere un prodotto che è sufficiente per essere venduto ad un'azienda."

Questa osservazione è importante: il prompt non è perfetto, ma è sufficiente per ottenere un risultato funzionante. La perfezione del prompt non è necessaria quando si ha una buona architettura di progetto (CLAUDE.md, rules, ciclo di verifica).

16.5 — Come Ottenere le API Key

L'autore mostra il processo pratico per ottenere le chiavi API necessarie:

API Key di Stripe

text

PROCESSO:

1. Andare su [Stripe.com](https://stripe.com)
2. Navigare a: Developer → API Keys
3. Selezionare: Standard API key → "Building your own integration"
4. Premere: Create
5. Verificare l'identità (email + codice telefono)
6. Dare un nome alla chiave (es. "Giovanni YouTube Italia")
7. Copiare la chiave generata

 NOTA DI SICUREZZA:

L'autore usa una chiave di produzione nel video per semplicità.

In uno scenario reale, usare SEMPRE chiavi di TEST durante lo sviluppo.

Le chiavi test iniziano con "sk_test_"

Le chiavi live iniziano con "sk_live_"

API Key di Supabase

text

PROCESSO:

1. Andare su [Supabase.com](https://supabase.com)

2. Aprire il progetto

3. Navigare a: Settings → API

4. Copiare:

- La URL del progetto (es. <https://xxx.supabase.co>)
- La chiave Anon/Public

5. Per le chiavi segrete: Settings → API → Service Role Key

⚠ NOTA SUI NOMI:

I nomi delle chiavi in Supabase possono contenere solo

lettere minuscole e cifre (no maiuscole, no spazi, no caratteri speciali)

16.6 — La Plan Mode in Azione

Dopo aver inviato il prompt, Claude Code in Plan Mode produce un piano strutturato:

text

PIANO GENERATO DA CLAUDE CODE:

Fase 1: Tech Stack Setup

└─ Inizializzare il progetto (Next.js o simile)

└─ Installare le dipendenze

Fase 2: Database

└─ Creare lo schema SQL per Supabase

└─ Tabelle: utenti, board, card, pagamenti

Fase 3: Autenticazione

- └─ Form di login con email + nome
- └─ Login diretto senza verifica email

Fase 4: Pagine e Routing

- └─ Pagina di login
- └─ Pagina principale (board)
- └─ Pagina di pagamento

Fase 5: Componenti UI

- └─ Board Trello-like
- └─ Colonne draggabili
- └─ Card con labels
- └─ Commenti

Fase 6: Server & Autenticazione

- └─ Middleware di autenticazione
- └─ Protezione delle rotte

Fase 7: Stripe

- └─ Integrazione pagamenti
- └─ Webhook
- └─ Gestione abbonamenti

Ordine di implementazione: Sequenziale (1-7)

Verifica: Test alla fine di ogni fase

Interazione Intelligente del Piano

Durante il planning, Claude Code pone domande strategiche all'utente:

Domanda 1: "Quale metodo di autenticazione preferisci? Magic Link (l'utente riceve un'email e si autentica da lì) o Login diretto?"

→ L'autore sceglie: Login diretto

Domanda 2: "La chiave Stripe che hai condiviso è una chiave live di produzione. Vuoi usare chiavi test per lo sviluppo?"

→ L'autore sceglie: Usare le chiavi fornite, consapevole del rischio

Queste domande dimostrano che Claude Code in Plan Mode non è un esecutore cieco: è un collaboratore che identifica potenziali problemi e chiede chiarimenti prima di procedere.

16.7 — Dalla Plan Mode al Bypass Permission

Una volta approvato il piano, l'autore fa il passaggio chiave:

text

TRANSIZIONE:

Plan Mode (pianificazione)

↓

[Piano approvato dall'utente]

↓

Bypass Permission (esecuzione)

↓

Claude Code costruisce in autonomia

L'autore spiega la logica: *"Una volta che il piano ha senso ed è fatto in maniera corretta, allora Claude Code può fare il cosiddetto one-shot."*

Il principio è: investire tempo nella pianificazione per poi lasciare che l'esecuzione sia completamente autonoma. Il piano di qualità è la preconditione per un'esecuzione autonoma di qualità.

16.8 — Configurazione Post-Build

Dopo che Claude Code ha costruito l'applicazione, c'è una fase di configurazione manuale necessaria:

text

PASSI DI CONFIGURAZIONE MANUALE:

1. Aggiornare il file .env con le chiavi reali:

- NEXT_PUBLIC_SUPABASE_URL = [URL dal dashboard Supabase]
- NEXT_PUBLIC_SUPABASE_ANON_KEY = [chiave Anon da Supabase]

2. Eseguire lo schema SQL in Supabase:

- Dashboard Supabase → SQL Editor
- Copiare il contenuto del file schema (generato da Claude)

- Premere "Run"

- Verificare: "Success, no rows returned"

3. Disabilitare email confirmation in Supabase:

- Settings → Authentication → Sign in providers
- Disattivare "Enable email confirmations"
- Salvare

4. Avviare il server di sviluppo:

- Chiedere a Claude: "Per favore run il comando `npm run dev` e dimmi cos'altro devo avere per testare l'app.
- Se è tutto a posto, dammi il localhost."

16.9 — Il Risultato Funzionante

L'autore testa l'applicazione dal vivo e verifica:

text

FUNZIONALITÀ VERIFICATE:

- ✔ Login funzionante (email: `job@...`, nome: Giovanni)
- ✔ Board Trello-like con colonne
- ✔ Card creabili con titolo
- ✔ Card spostabili tra colonne (drag & drop)
- ✔ Labels colorati (rosso, blu, arancione)
- ✔ Sistema di commenti sulle card
- ✔ Preferenze utente
- ✔ Dark mode (funzionalità extra non richiesta — Claude l'ha aggiunta)
- ✔ Dati salvati su Supabase (verificato nel dashboard)
- ✔ Multi-utente (testato con secondo account: Pini/Paolo)
- ✔ Sign out funzionante
- ⚠ Dark mode non perfettamente funzionante (Claude l'ha inventata autonomamente senza istruzioni specifiche → risultato imperfetto)

NOTA DELL'AUTORE SUL DARK MODE:

"Ma chi l'aveva promptato il dark mode? Se l'è inventato lui.

Quindi che non funzionasse me lo aspettavo."

16.10 — Integrazione Stripe (Stadio 2)

Per l'integrazione Stripe, l'autore utilizza il meccanismo di memoria per trasferire il contesto alla nuova sessione:

text

PROCESSO DI TRANSIZIONE SESSIONE:

1. Nella sessione corrente (contesto al 66%):

"Sei al 66% del contesto, il che significa che comincerai a perdermi a livello di performance. Salvare le cose in memoria in modo che nel nuovo contesto possa continuare."

2. Claude salva in memoria:

- Struttura del progetto
- File modificati
- Configurazioni
- Stato attuale
- Cosa manca (integrazione Stripe)

3. Nuova sessione:

"Continua con integrazione Stripe"

4. Claude legge la memoria e riprende il lavoro

L'integrazione Stripe richiede passaggi specifici:

text

CONFIGURAZIONE STRIPE:

1. Creare il prodotto su Stripe:

- Dashboard Stripe → Product Catalog → Add Product

- Nome: "Canboard Pro" (o il nome dell'app)
- Pricing: Standard, Recurring, \$9/mese, Monthly
- Save Product

2. Copiare il Price ID:

- Nella pagina del prodotto, sotto Pricing
- Copiare il codice che inizia con "price_"

3. Fornire il Price ID a Claude Code

4. Claude Code integra Stripe nell'app:

- Pagina di pagamento
- Form "Subscribe Now"
- Bottone "Pay and Subscribe"

16.11 — Gestione delle Aspettative

L'autore dedica un passaggio importante alla gestione delle aspettative, che è una lezione di valore inestimabile:

"Ci aspettavamo che l'integrazione end-to-end funzionasse molto bene, perché siamo stati iper specifici. Abbiamo dato indicazioni per quanto riguardava come questa integrazione doveva andare, abbiamo dato API key, abbiamo dato dei collegamenti, abbiamo adressato un paio di step che ci ha chiesto. Quello che non abbiamo dato sono state le direttive di styling e design — che ovviamente, se volesse, uno andrebbe in serenità ad averle."

REGOLA FONDAMENTALE:

Specificità dell'input → Qualità dell'output

SPECIFICO nel prompt → ACCURATO nell'output

- └─ API keys fornite →  Integrazione funzionante
- └─ Screenshot di riferimento →  Struttura replicata
- └─ Schema database definito →  Backend corretto
- └─ Flusso autenticazione definito →  Login funzionante

VAGO nel prompt → APPROSSIMATIVO nell'output

- └─ "Fai un design carino" →  Design mediocre
- └─ Nessuna guida di stile →  Font/colori casuali

16.12 — Errori Comuni

Errore 1: Cercare di costruire tutto in un colpo solo

La strategia a due stadi (prima senza Stripe, poi con Stripe) è intenzionale. Costruire tutto simultaneamente aumenta la probabilità di errori difficili da debuggare.

Errore 2: Non salvare in memoria prima di cambiare sessione

Se il contesto è al 60-70% e si continua senza salvare, le performance degradano. L'autore chiede esplicitamente a Claude di salvare in memoria e poi apre una nuova sessione.

Errore 3: Non eseguire lo schema SQL manualmente

Claude Code genera lo schema SQL ma non può sempre eseguirlo direttamente su Supabase. L'utente deve copiare lo schema e eseguirlo manualmente nel SQL Editor di Supabase.

Errore 4: Usare chiavi API di produzione durante lo sviluppo

L'autore lo fa per semplicità nel video ma avverte che è una pratica rischiosa. Le chiavi test sono sempre preferibili durante lo sviluppo.

Errore 5: Non verificare che Supabase abbia salvato i dati

Dopo la prima registrazione, l'autore va nel dashboard Supabase per verificare che i dati siano effettivamente stati salvati nelle tabelle. Questo è un passaggio di verifica che non dovrebbe essere saltato.

16.13 — Insight Avanzato

La nota sulle risorse esterne per la sicurezza:

L'autore fa un'osservazione pratica molto preziosa per chi vuole trasformare progetti Claude Code in prodotti vendibili:

"Per favore non dimenticatevi mai che c'è tutta la parte di security. Un buon software developer è sempre necessario. In Upwork, che è una piattaforma di terzi, trovate persone estremamente competenti che per €200-300 vi fanno un mega lavoro. Sono ovviamente indiani o pakistani, quindi dovete parlare un minimo di inglese, ma sono persone estremamente competenti."

Questo significa che il workflow ottimale per creare un prodotto vendibile è:

WORKFLOW PRODOTTO VENDIBILE:

1. Claude Code → Costruisce l'applicazione (ore)
2. Review → Voi verificate le funzionalità (minuti-ore)
3. Security Audit → Developer esterno su Upwork (€200-300)
4. Deployment → Claude Code + Vercel/Modal (minuti)
5. Prodotto live e vendibile

Costo totale: \$17 (Claude Pro) + €200-300 (security audit) = ~€220-320

Tempo totale: 1-2 giorni

Alternativa tradizionale: Team di sviluppo, settimane, migliaia di euro

Aggiungere la verifica automatica on-the-go:

L'autore dimostra che è possibile aggiungere istruzioni a Claude Code mentre sta lavorando:

"Possiamo darglielo on the go, quindi nel mentre che sta andando, perché comunque capirà. Quello che succede è che una volta che avrà finito di lavorare nella task a cui sta lavorando, poi leggerà il mio prompt."

Questo significa che non dovete aspettare che Claude finisca per dargli nuove istruzioni. Potete scrivere un prompt mentre lavora e lui lo processerà alla prossima pausa naturale nel workflow.

PARTE 5 — MODALITÀ DI PERMESSO E PIANIFICAZIONE

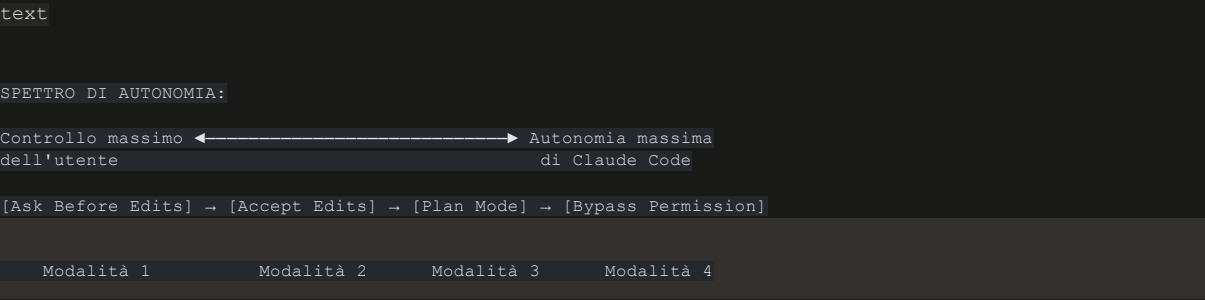
CAPITOLO 17: LE QUATTRO MODALITÀ DI PERMESSO

17.1 — Definizione del Concetto

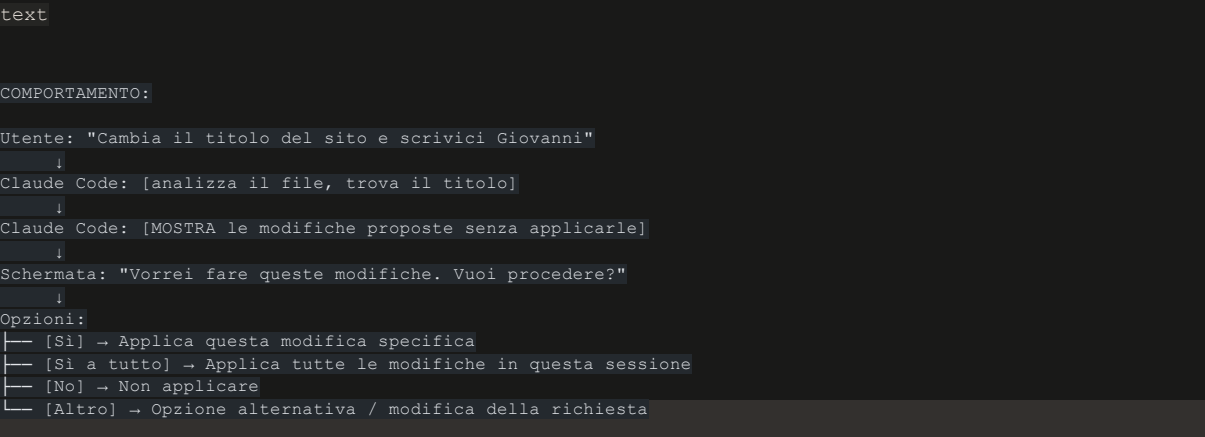
Le modalità di permesso (permission modes) sono i livelli di autonomia che l'utente concede a Claude Code durante il lavoro. Determinano quanto controllo l'utente mantiene sulle azioni di Claude Code — dalla supervisione totale di ogni singola modifica, fino alla completa autonomia operativa. La scelta della modalità corretta in ogni fase del lavoro è una decisione strategica che impatta direttamente su velocità, sicurezza e qualità del risultato.

17.2 — Spiegazione Espansa

Esistono quattro modalità di permesso, disposte in ordine crescente di autonomia:



Modalità 1: Ask Before Edits (Default)



Caratteristiche chiave:

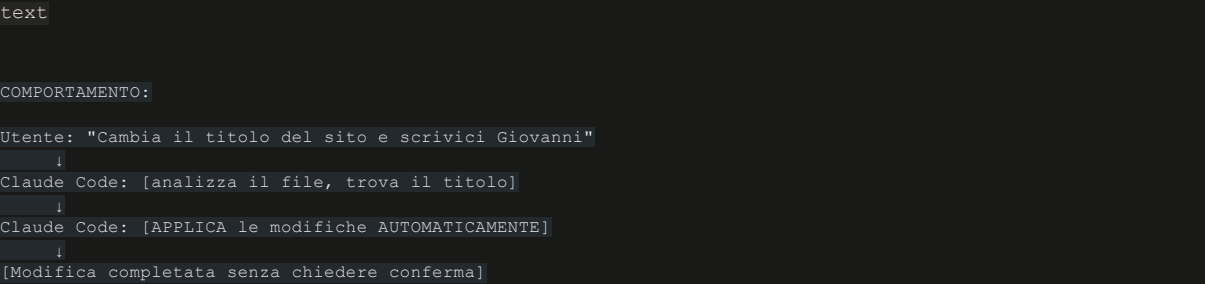
- È il comportamento di default quando si installa Claude Code
- Claude Code propone le modifiche ma non le applica fino a conferma esplicita
- L'utente vede esattamente cosa verrà modificato prima che succeda
- Ogni modifica richiede un'approvazione separata (a meno che non si scelga "Sì a tutto")

Quando usarla:

- Quando state imparando Claude Code e volete capire cosa fa
- Quando lavorate su file critici che non volete siano modificati per errore
- Quando volete controllo granulare su ogni singola modifica
- In ambienti di produzione dove ogni cambiamento deve essere approvato

Limitazione:
È la modalità più lenta perché richiede intervento umano per ogni modifica. Su progetti con decine o centinaia di modifiche, diventa impraticabile.

Modalità 2: Accept Edits (Accettazione Automatica delle Modifiche)



```
↓
⚠ ECCEZIONE: Chiede sempre conferma per:
• Creare nuovi file
• Cancellare file esistenti
```

Caratteristiche chiave:

- Le modifiche ai file esistenti vengono applicate automaticamente
- La creazione e cancellazione di file richiede sempre conferma
- È un buon compromesso tra velocità e sicurezza
- Elimina la necessità di approvare ogni singolo edit

Quando usarla:

- Quando avete un'idea chiara di cosa volete e non serve revisionare ogni modifica
- Per task di refactoring dove molti file devono essere modificati
- Quando volete velocità ma mantenete la protezione contro creazione/cancellazione accidentale

Limitazione importante notata dall'autore:

Accept Edits non può creare o cancellare file autonomamente. Per queste operazioni, chiederà sempre il consenso. Questo è un limite di sicurezza intenzionale che protegge da cancellazioni accidentali.

Modalità 3: Plan Mode (Pianificazione)

Il Plan Mode è sufficientemente importante da meritare un capitolo dedicato (Capitolo 18). Qui ne forniamo la definizione sintetica nel contesto delle quattro modalità.

text

COMPORTAMENTO:

```
Utente: "Costruiscimi un'app simile a Trello con autenticazione e pagamenti"
↓
Claude Code: [ANALIZZA la richiesta]
↓
Claude Code: [CREA UN PIANO strutturato con subtask]
↓
Claude Code: [PRESENTA il piano all'utente]
↓
Utente: [Revisiona il piano, approva/modifica]
↓

Claude Code: [Esegue il piano approvato]
```

Caratteristiche chiave:

- Claude Code non esegue nulla immediatamente
- Prima crea un piano strutturato e lo presenta per approvazione
- L'utente può modificare, aggiungere o rimuovere elementi dal piano
- Solo dopo l'approvazione, Claude Code procede all'esecuzione
- Durante l'esecuzione, segue la checklist del piano approvato

Modalità 4: Bypass Permission (Dangerously Skip Permissions)

Anche questa modalità merita un capitolo dedicato (Capitolo 19). Definizione sintetica:

text

COMPORTAMENTO:

```
Utente: "Costruiscimi un'app simile a Trello con autenticazione e pagamenti"
```

```
↓
Claude Code: [ESEGUE TUTTO in completa autonomia]
↓
[Crea file, modifica file, cancella file, esegue comandi]
↓
[NESSUNA richiesta di conferma per nulla]
↓
[Risultato finale presentato quando tutto è completato]
```

Caratteristiche chiave:

- Autonomia totale: Claude Code può fare qualsiasi cosa senza chiedere
- Include la capacità di creare e cancellare file
- È la modalità più veloce ma anche la più rischiosa
- L'autore la usa frequentemente ma avverte dei rischi

17.3 — Perché Questo Concetto È Importante

La scelta della modalità di permesso non è una preferenza estetica. È una decisione strategica che deve essere presa in base a:

1. La fase del progetto — Pianificazione? Esecuzione? Debugging?
2. Il livello di rischio — File critici? Dati sensibili? Produzione?
3. La qualità del piano — Piano dettagliato? Prompt vago?
4. L'esperienza dell'utente — Principiante? Esperto?

text

MATRICE DECISIONALE:

	Rischio Basso	Rischio Alto
Piano Buono	Bypass Permission	Plan Mode → poi Accept Edits
Piano Vago/Assente	Accept Edits	Ask Before Edits

17.4 — Meccanismo Sottostante: Propagazione dei Permessi

L'autore introduce un concetto critico che molti ignorano: i permessi si propagano a tutti i sotto-agenti.

"Quando diamo una permission all'agente principale, tutte queste permission si replicano anche a tutti i sottoagenti o a qualsiasi cosa concateniamo dopo."

text

PROPAGAZIONE DEI PERMESSI:

```
Agente Principale (Bypass Permission)
  ↓ eredita
Sub-agente Researcher (Bypass Permission)
  ↓ eredita
Sub-agente Reviewer (Bypass Permission)
  ↓ eredita
Sub-agente QA (Bypass Permission)
```

```
IMPLICAZIONE: Se l'agente principale ha Bypass Permission,
TUTTI i sotto-agenti avranno Bypass Permission.
Non è possibile dare Bypass Permission al principale
```

```
e Ask Before Edits a un sotto-agente.
```

Questo ha implicazioni di sicurezza significative: se date bypass permission all'agente principale e uno dei sotto-agenti commette un errore, quell'errore viene eseguito senza alcuna richiesta di conferma.

17.5 — Interpretazione Pratica: Come Cambiare Modalità

Nell'IDE (VS Code / Antigravity)

Nella parte superiore del pannello Claude Code, ci sono pulsanti/selector per cambiare tra le modalità:

- Ask Before Edits
- Edit Automatically (Accept Edits)
- Plan Mode
- Bypass Permission (se abilitato nelle impostazioni)

Nel Terminal

```
text
```

```
NAVIGAZIONE MODALITÀ NEL TERMINAL:
```

```
Shift + Freccia su + Tab → Mostra le modalità disponibili
                        → Navigare con le frecce
                        → Selezionare con Enter
```

```
Oppure all'avvio:
```

```
$ claude → Avvia in modalità default
$ claude --dangerously-skip-permissions → Avvia in Bypass Permission
```

Abilitare Bypass Permission (non attivo di default)

In VS Code:

1. Icona ingranaggio (⚙️) → Settings
2. Cercare "Claude"
3. Abilitare "Allow Dangerously Skip Permission"

In Antigravity:

1. Icona ingranaggio in alto a destra → Settings
2. Cercare "Claude Code"
3. Abilitare "Dangerously Skip Permission"

Nel Terminal:

Usare il flag `--dangerously-skip-permissions` all'avvio, oppure configurare un alias come l'autore che usa "YOLO" (You Only Live Once).

17.6 — Gestione Avanzata dei Permessi nel Terminal

Nel Terminal, esiste un sistema di permessi più granulare accessibile tramite il comando `permissions`:

text

SISTEMA DI PERMESSI GRANULARE:

Allow (Consenti):

- Il tool è sempre permesso senza chiedere
- Esempio: "Allow bash" = i comandi bash vengono eseguiti automaticamente

Ask (Chiedi):

- Chiede sempre il permesso prima di usare il tool
- Esempio: "Ask write" = chiede conferma prima di ogni scrittura su file

Deny (Nega):

- Il tool non viene mai usato, nemmeno se richiesto
- Esempio: "Deny delete" = impossibile cancellare file in qualsiasi circostanza

Workspace:

- Permessi specifici per il workspace/progetto corrente

- Permette configurazioni diverse per progetti diversi

Questo sistema è più granulare delle quattro modalità principali perché permette di controllare i singoli tool individualmente anziché applicare una politica uniforme a tutti i tool.

17.7 — Errori Comuni

Errore 1: Restare sempre in Ask Before Edits

Molti utenti non escono mai dalla modalità default per paura di perdere il controllo. Questo rallenta enormemente il workflow. Una volta che avete familiarità con Claude Code e avete un buon piano, passare ad Accept Edits o Bypass Permission è una scelta di produttività essenziale.

Errore 2: Andare direttamente in Bypass Permission senza piano

Questo è l'estremo opposto ed è molto pericoloso. L'autore racconta il caso di una persona che: *"ha cancellato completamente qualsiasi cosa all'interno del suo computer. Aveva dato un piano povero, aveva fatto bypass permission, e quello che è successo è che il computer ha continuato a fare ricerca per qualche ora finché poi non ha deciso che la soluzione migliore per risolvere il problema era cancellare tutto quanto."*

Errore 3: Non capire la propagazione ai sotto-agenti

Se date bypass permission all'agente principale, ogni sotto-agente chiamato eredita la stessa autonomia. Questo è particolarmente rischioso con sotto-agenti che fanno operazioni distruttive (es. pulizia codice, ristrutturazione file).

Errore 4: Cambiare modalità nel mezzo di un'operazione critica

Passare da Plan Mode a Bypass Permission mentre Claude sta ancora pianificando può creare confusione. Completate la fase corrente prima di cambiare modalità.

Errore 5: Non abilitare Bypass Permission nelle impostazioni e non capire perché non appare

La modalità Bypass Permission non è visibile di default. Deve essere esplicitamente abilitata nelle impostazioni dell'IDE. Molti utenti cercano questa opzione senza trovarla perché non hanno modificato le impostazioni.

17.8 — Insight Avanzato

Il quinto livello: Don't Ask (solo Terminal)

L'autore menziona una modalità aggiuntiva disponibile solo nel Terminal e non presente nelle IDE:

text

PERMESSI NEL TERMINAL:

```
Allow → "We always allow this tool"
Ask → "We always ask permission" (default)
Deny → "We always reject request to use denied tools"
```

Deny è il "Don't Ask" — non chiede nemmeno, rifiuta direttamente.

La differenza tra "Ask" e "Deny" è sottile ma importante:

- Ask: "Posso usare questo tool?" → L'utente può dire sì o no
- Deny: Il tool non viene mai usato, Claude Code non chiede nemmeno. Il tool è effettivamente disabilitato.

Questo è utile per disabilitare completamente strumenti che non volete siano mai usati nel vostro progetto (per esempio, disabilitare la capacità di cancellare file in un ambiente di produzione).

Workflow raccomandato dall'autore:

L'autore rivela il suo workflow personale basato sulla citazione del creatore di Claude Code, Boris:

"Come disse Boris, creatore di Claude Code — e che lui stesso fa questa cosa — spende gran parte del suo tempo in Plan Mode, e una volta che il piano ha senso ed è fatto in maniera corretta, allora Claude Code può fare il cosiddetto one-shot."

text

WORKFLOW BORIS/AUTORE:

```
Fase 1: PLAN MODE (70-80% del tempo)
├─ Definire il progetto
├─ Creare il piano
├─ Revisionare il piano
├─ Modificare il piano
├─ Ri-revisionare
├─ Approvare il piano finale
└─ Tempo: la maggior parte della sessione
```

```
Fase 2: BYPASS PERMISSION (20-30% del tempo)
├─ Eseguire il piano approvato
├─ Claude Code lavora in autonomia
├─ One-shot (idealmente)
└─ Tempo: relativamente breve
```

Proporzione ideale: PIANO >> ESECUZIONE

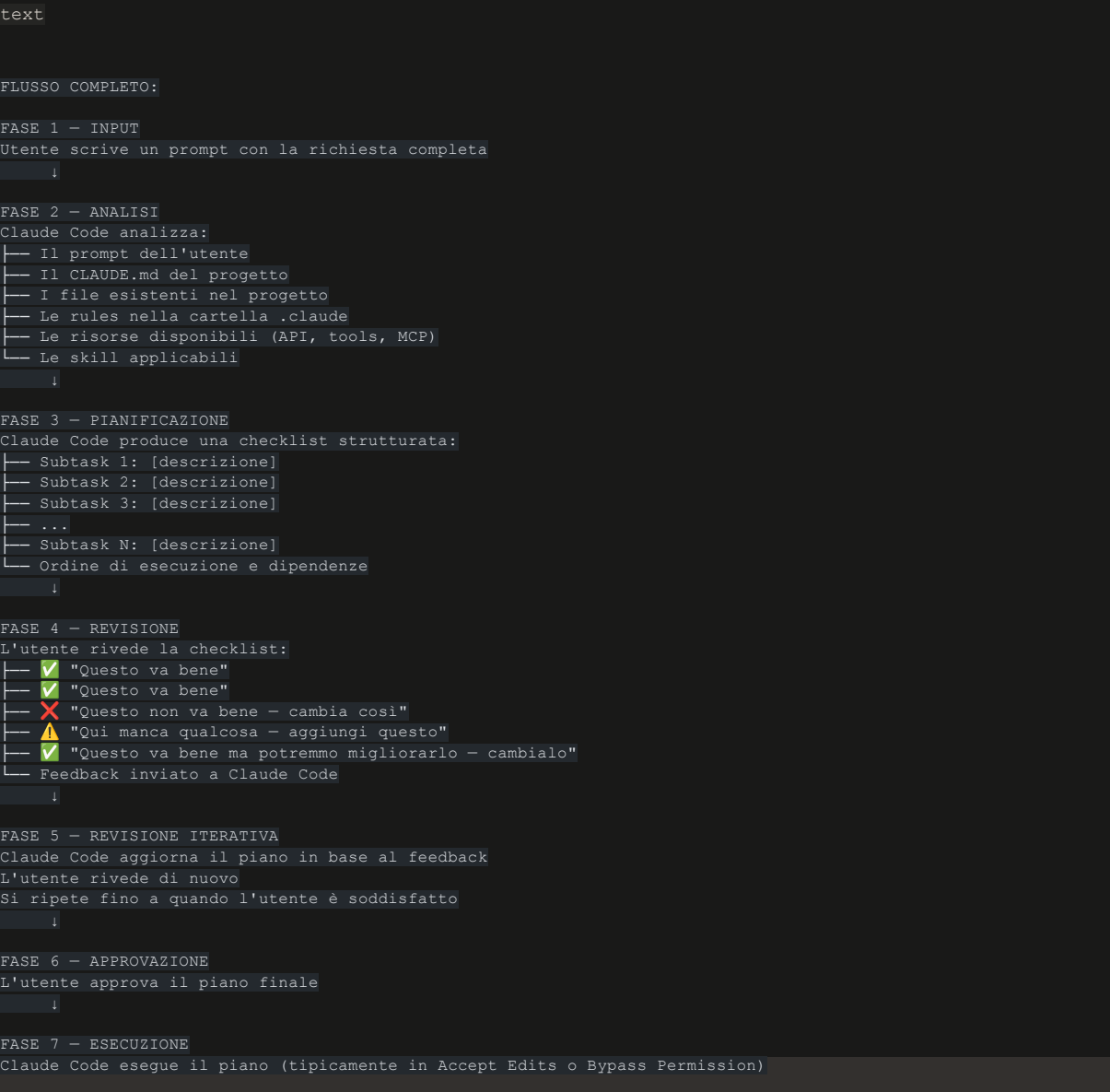
CAPITOLO 18: PLAN MODE — L'APPROCCIO STRATEGICO

18.1 — Definizione del Concetto

Il Plan Mode è la modalità in cui Claude Code, anziché eseguire immediatamente le istruzioni, crea prima un piano strutturato composto da subtask ordinate, lo presenta all'utente per revisione e approvazione, e solo successivamente procede all'esecuzione. È la modalità strategica per eccellenza e quella che l'autore considera la più importante per progetti di qualsiasi complessità.

18.2 — Spiegazione Espansa

Il Flusso del Plan Mode



L'autore è molto chiaro sull'approccio alla revisione: *"Plan mode significa semplicemente: continuiamo ad insistere fino alla morte sulla nostra checklist fino a che non siamo soddisfatti. E poi, una volta che abbiamo pianificato il tutto, allora muoviamoci ad accettare gli edits e ad andare a costruire."*

La parola chiave è "fino alla morte" — non è un'esagerazione. La qualità del piano è direttamente proporzionale al tempo investito nella sua revisione.

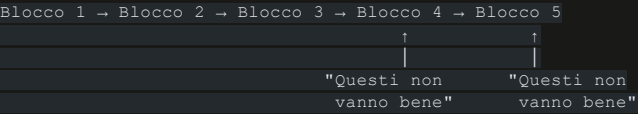
18.3 — Perché Questo Concetto È Importante

L'autore presenta un'analisi visiva che dimostra perché il Plan Mode è cruciale, usando l'esempio dei "blocchi":

Scenario SENZA Plan Mode (Esecuzione Diretta):

text

COSTRUZIONE DIRETTA SENZA PIANO:



- CONSEGUENZA:
- Tempo speso: 15 minuti per costruire 5 blocchi
 - Scoperta: 2 blocchi (4 e 5) non vanno bene
 - Problema: I blocchi sbagliati influenzano la struttura complessiva
 - Rischio: Potrebbe essere necessario RICOSTRUIRE TUTTO
 - Tempo perso: 15 minuti + tempo di ricostruzione
 - Tempo totale potenziale: 1-5 ORE

Scenario CON Plan Mode:

text

PIANIFICAZIONE PRIMA DELL'ESECUZIONE:

- Piano:
- ☐ Blocco 1: [descrizione] → Ok
 - ☐ Blocco 2: [descrizione] → Ok
 - ☐ Blocco 3: [descrizione] → Ok
 - ☐ Blocco 4: [descrizione] → Non va bene → Modificato
 - ☐ Blocco 5: [descrizione] → Non va bene → Modificato

- Piano Rivisto:
- ☐ Blocco 1: [descrizione] → Ok
 - ☐ Blocco 2: [descrizione] → Ok
 - ☐ Blocco 3: [descrizione] → Ok
 - ☐ Blocco 4: [nuova descrizione] → Ok
 - ☐ Blocco 5: [nuova descrizione] → Ok

- CONSEGUENZA:
- Tempo di pianificazione: 10-15 minuti
 - Tutti i blocchi sono validati PRIMA della costruzione
 - Nessun rischio di dover ricostruire
 - Esecuzione: veloce e one-shot
 - Tempo totale: 20-30 minuti MAX

La differenza è lampante:

- Senza piano: 15 minuti di costruzione + 1-5 ore di ricostruzione = 1-5+ ore
- Con piano: 10-15 minuti di pianificazione + 10-15 minuti di esecuzione = 20-30 minuti

L'autore riassume: *"Plan mode significa semplicemente: continuiamo ad insistere fino alla morte sulla nostra checklist fino a che non siamo soddisfatti."*

18.4 — Interpretazione Pratica: Domande Intelligenti del Piano

Durante il Plan Mode, Claude Code non si limita a creare un piano. Pone anche domande strategiche che aiutano a raffinare il progetto. L'autore mostra due esempi concreti dalla costruzione dell'app Trello:

Domanda 1 — Scelta tecnica:

- Claude: "Quale metodo di autenticazione preferisci?"
- A) Magic Link - l'utente riceve un'email e si autentica da lì
 - B) Login diretto - l'utente inserisce credenziali e entra subito"

Questa domanda è importante perché la scelta tecnica influenza l'architettura dell'intera applicazione. Prenderla nella fase di pianificazione evita di dover ristrutturare l'app in seguito.

Domanda 2 — Sicurezza:

Claude: "La chiave Stripe che hai condiviso è una chiave live di produzione.
Vuoi usare chiavi test per lo sviluppo?"

Le chiavi test iniziano con sk_test_"

Claude Code identifica un rischio di sicurezza nel prompt dell'utente e lo segnala proattivamente. Questo tipo di feedback è uno dei valori aggiunti del Plan Mode.

18.5 — Meccanismo Sottostante: La Checklist come Contratto

Il piano prodotto dal Plan Mode funziona come un contratto tra l'utente e Claude Code. Una volta approvato:

1. Claude Code sa esattamente cosa deve fare
2. L'utente sa esattamente cosa aspettarsi
3. Ogni deviazione dal piano è un errore identificabile
4. Il progresso è misurabile (quanti item della checklist sono completati)

ANALOGIA DEL CONTRATTO:

Piano = Contratto firmato tra utente e Claude Code

Clausola 1: Fase tech stack setup ✓ (completata)

Clausola 2: Database schema ✓ (completata)

Clausola 3: Autenticazione ✓ (completata)

Clausola 4: Componenti UI □ (in corso)

Clausola 5: Integrazione Stripe □ (in attesa)

Se Claude devia dal piano → L'utente può dire "questo non era nel piano"

Se l'utente aggiunge richieste → "Questo non era nel piano — facciamo un nuovo piano"

18.6 — Caso Pratico: Il Social Media Manager

L'autore fornisce il suo esempio più significativo di Plan Mode:

CASO: SOCIAL MEDIA MANAGER

Tempo di Planning: 2 ore e 30 minuti

└─ Definizione delle piattaforme (YouTube IT/EN, LinkedIn, Meta)

└─ Definizione dei workflow per ogni piattaforma

└─ Definizione delle skill necessarie

└─ Definizione delle integrazioni (API per ogni piattaforma)

└─ Definizione del database di riferimento

└─ Revisione del piano

└─ Seconda revisione

└─ Approvazione finale

Tempo di Esecuzione: 3 ore

└─ Claude Code in Bypass Permission

└─ ONE-SHOT (nessun ritorno necessario)

└─ Tutti i collegamenti funzionanti

| └─ YouTube

| └─ LinkedIn

| └─ Meta

└─ Sistema completo e operativo

TEMPO TOTALE: 5 ore e 30 minuti

ITERAZIONI POST-ESECUZIONE: ZERO

RISULTATO: Sistema completo e in produzione

La proporzione è significativa: 45% pianificazione, 55% esecuzione, e zero correzioni post-esecuzione. L'investimento nella pianificazione ha prodotto un'esecuzione one-shot perfetta.

18.7 — Errori Comuni

Errore 1: Approvare il piano troppo velocemente

La tentazione di premere "Approva" per passare subito all'esecuzione è forte. Ma ogni minuto speso a revisionare il piano risparmia potenzialmente ore di debugging e ricostruzione. L'autore è chiaro: *"Questa è la parte su cui ci focalizziamo di più."*

Errore 2: Non dare feedback specifico durante la revisione

Dire "non mi piace" non è feedback utile. Dire "il punto 4 non mi piace perché dovremmo usare PostgreSQL anziché SQLite, e il punto 5 manca di error handling" è feedback che Claude Code può utilizzare per migliorare il piano.

Errore 3: Saltare il Plan Mode per task "semplici"

Molte task che sembrano semplici si rivelano complesse durante l'esecuzione. L'autore raccomanda il Plan Mode per qualsiasi task che coinvolga più di un file o più di una funzionalità.

Errore 4: Non fare Plan Mode prima di Bypass Permission

Fare Bypass Permission senza un piano approvato è la ricetta per il disastro. L'autore lo raccomanda esplicitamente come workflow sequenziale: prima Plan Mode per creare e approvare il piano, poi Bypass Permission per eseguirlo.

Errore 5: Confondere Plan Mode con una modalità di esecuzione lenta

Plan Mode non è "fare le cose lentamente". È investire tempo nella direzione prima di investire tempo nell'esecuzione. Il tempo totale è quasi sempre inferiore rispetto all'esecuzione diretta senza piano.

18.8 — Insight Avanzato

La transizione Plan Mode → Bypass Permission:

L'autore mostra la transizione pratica nel suo workflow:

TRANSIZIONE PRATICA:

1. Attivare Plan Mode
2. Scrivere il prompt completo
3. Revisionare il piano generato
4. Iterare fino a soddisfazione
5. Premere 'S' (per Save/Submit il piano approvato)
6. Attivare "Auto Accept" (accettazione automatica)
7. Passare a Bypass Permission
8. Claude Code esegue il piano in autonomia

L'autore descrive il punto 5-7: "Premiamo S e auto accept,

e poi andiamo in bypass permission."

Questo passaggio è il momento in cui l'utente "passa il controllo" a Claude Code. È il momento in cui la fiducia nel piano si traduce in azione autonoma.

Il piano come protezione economica:

Per chi usa il piano API, il Plan Mode è anche una protezione economica. Pianificare costa relativamente poco in token (la conversazione di pianificazione è breve e focalizzata). L'esecuzione costa molto di più. Un piano sbagliato che porta a un'esecuzione da rifare significa pagare due volte per lo stesso lavoro.

Per chi usa il piano subscription con Agent Teams, la logica è simile: gli Agent Teams consumano velocemente il budget mensile. Un piano accurato prima di lanciare un Agent Team può risparmiare centinaia di euro di consumo inutile.

CAPITOLO 19: BYPASS PERMISSION — AUTONOMIA MASSIMA

19.1 — Definizione del Concetto

Bypass Permission (ufficialmente "Dangerously Skip Permissions") è la modalità in cui Claude Code opera in completa autonomia, senza richiedere alcuna approvazione per qualsiasi operazione — incluse la creazione e la cancellazione di file. È la modalità più potente e più rischiosa, che richiede un piano ben definito come prerequisito e una comprensione chiara delle implicazioni.

19.2 — Spiegazione Espansa

Il nome stesso della modalità contiene un avvertimento: "Dangerously" (pericolosamente). Anthropic ha intenzionalmente inserito la parola "pericolosamente" nel nome per segnalare che questa modalità richiede consapevolezza dei rischi.

Cosa può fare Claude Code in Bypass Permission:

CAPACITÀ COMPLETE:

- ✓ Leggere qualsiasi file nel progetto
- ✓ Modificare qualsiasi file nel progetto
- ✓ Creare nuovi file
- ✓ CANCELLARE file esistenti ← La differenza critica
- ✓ Eseguire comandi bash
- ✓ Installare pacchetti/dipendenze
- ✓ Modificare configurazioni di sistema
- ✓ Navigare nel web (se MCP attivo)
- ✓ Chiamare sotto-agenti con gli stessi permessi
- ✓ Tutto quanto SENZA CHIEDERE MAI conferma

La differenza critica rispetto ad "Accept Edits" è nella capacità di creare e cancellare file autonomamente. In Accept Edits, Claude Code chiede sempre conferma prima di creare o cancellare un file. In Bypass Permission, può farlo senza alcuna richiesta.

19.3 — Perché Questo Concetto È Importante

L'importanza di Bypass Permission è duplice:

Lato positivo — Produttività massima:

Quando il piano è buono e le istruzioni sono chiare, Bypass Permission permette a Claude Code di lavorare alla massima velocità possibile. Non ci sono interruzioni per chiedere permessi, non ci sono attese per la conferma dell'utente. Claude Code procede dall'inizio alla fine del piano senza fermarsi.

L'autore dimostra questo con il caso del social media manager: dopo 2.5 ore di pianificazione, Claude Code ha costruito l'intero sistema in 3 ore senza mai fermarsi — un one-shot completo.

Lato negativo — Rischio massimo:
L'autore racconta un caso reale che serve da avvertimento:

"È successo che una persona, mi sembra un paio di mesi fa, abbia cancellato completamente qualsiasi cosa all'interno del suo computer. Aveva dato un piano povero, aveva fatto bypass permission, e quello che è successo è che il computer ha continuato a fare ricerca per qualche ora, finché poi non ha deciso che la soluzione migliore per risolvere il problema era cancellare tutto quanto."

Analisi del caso:

CASO DI DISASTRO:

- Causa 1: Piano povero (insufficiente per guidare Claude Code)
- Causa 2: Bypass Permission attivato (nessun controllo)
- Causa 3: Claude Code ha operato per ore senza supervisione
- Causa 4: Claude Code ha determinato che la "soluzione" era cancellare tutto

Risultato: TUTTO il contenuto del computer cancellato

Prevenzione: Piano dettagliato + Checkpoint attivi + Supervisione periodica

19.4 — Interpretazione Pratica: Quando Usare Bypass Permission

L'autore fornisce una guida chiara basata sulla sua esperienza e su quella del creatore di Claude Code:

USARE BYPASS PERMISSION QUANDO:

- ✓ Avete completato un piano dettagliato in Plan Mode
- ✓ Il piano è stato revisionato e approvato
- ✓ I checkpoint/rewind sono attivi (/config → Rewind: ON)
- ✓ Avete un backup del progetto (o è su GitHub)
- ✓ Le istruzioni nel CLAUDE.md sono chiare e specifiche
- ✓ I vincoli negativi sono definiti ("NON fare X, NON cancellare Y")
- ✓ Siete nelle vicinanze del computer per monitorare periodicamente

NON USARE BYPASS PERMISSION QUANDO:

- ✗ Non avete un piano chiaro
- ✗ Il prompt è vago o aspirazionale
- ✗ Lavorate su file di produzione senza backup
- ✗ I checkpoint non sono attivi
- ✗ Non avete definito vincoli negativi
- ✗ State usando il piano API (i costi possono esplodere)

✗ È la prima volta che fate un task simile

19.5 — Meccanismo Sottostante: La Relazione Piano-Esecuzione

La chiave per usare Bypass Permission in modo sicuro è la qualità del piano. L'autore stabilisce una relazione diretta:

RELAZIONE PIANO ↔ BYPASS PERMISSION:

- Piano eccellente + Bypass Permission = ONE-SHOT (risultato perfetto)
- Piano buono + Bypass Permission = Buon risultato con poche correzioni
- Piano mediocre + Bypass Permission = Risultato mediocre, molte correzioni
- Piano povero + Bypass Permission = DISASTRO POTENZIALE

La qualità del piano è la PRECONDIZIONE per Bypass Permission sicuro.

L'autore riassume con le parole di Boris: *"Spende gran parte del suo tempo in Plan Mode, e una volta che il piano ha senso ed è fatto in maniera corretta, allora Claude Code può fare il cosiddetto one-shot."*

19.6 — Come Attivare Bypass Permission

Il processo è diverso per ogni interfaccia:

Nel Terminal (metodo più diretto):

```
# Avvio con flag
$ claude --dangerously-skip-permissions

# L'autore usa un alias personalizzato:
$ yolo
# Che corrisponde a:
# alias yolo="claude --dangerously-skip-permissions"
```

L'autore spiega: *"YOLO, You Only Live Once — per me vuol dire avvia Claude Code con bypass permission."*

In VS Code:

1. Icona ingranaggio (⚙️) → Settings
2. Cercare "Claude"
3. Checkbox: "Allow Dangerously Skip Permission" → ☒
4. Nel pannello Claude Code, selezionare la modalità Bypass Permission

In Antigravity:

1. Icona ingranaggio IN ALTO A DESTRA → Settings
2. Cercare "Claude Code"
3. Checkbox: "Dangerously Skip Permission" → ☒
4. Nel pannello Claude Code, selezionare la modalità Bypass Permission

Nota importante: La posizione dell'icona ingranaggio è diversa nei due IDE. L'autore lo specifica: *"La rotellina adesso è qua in cima, in alto a destra"* (per Antigravity), mentre in VS Code è nella barra laterale.

19.7 — Errori Comuni

Errore 1: Bypass Permission come modalità predefinita

Alcuni utenti, dopo aver visto la velocità del Bypass Permission, lo usano per tutto. Questo è pericoloso perché non tutte le task meritano autonomia totale. Per task esplorative, debugging o modifiche a file critici, Ask Before Edits o Accept Edits sono più appropriate.

Errore 2: Lasciare il computer incustodito per ore durante Bypass Permission

Anche con un buon piano, Claude Code può incontrare situazioni impreviste e prendere decisioni autonome che non sono ottimali. Monitorare periodicamente (anche solo guardando lo schermo ogni 10-15 minuti) previene situazioni problematiche.

Errore 3: Non avere checkpoint attivi

Senza checkpoint, un'operazione distruttiva non è reversibile (almeno non facilmente). Verificare sempre che `/config` → `Rewind/Checkpoint: ON` prima di usare Bypass Permission.

Errore 4: Non definire vincoli negativi nel CLAUDE.md
Senza vincoli espliciti su cosa NON fare, Claude Code in Bypass Permission potrebbe:

- Cancellare file che considera obsoleti
- Ristrutturare completamente l'architettura del progetto
- Modificare configurazioni di sistema
- Eliminare codice che considera ridondante

Vincoli come "NON cancellare MAI i file nella cartella /config" o "NON modificare i file .env" sono essenziali.

Errore 5: Avere paura irrazionale di Bypass Permission
L'autore nota che *"un po' di persone hanno diciamo paura"* di questa modalità. Con le precauzioni appropriate (piano, checkpoint, vincoli), Bypass Permission è lo strumento più produttivo disponibile. La paura irrazionale porta a non usarlo mai, il che significa perdere il beneficio della produttività massima.

19.8 — Insight Avanzato

Il workflow ideale completo:

Combinando tutti i concetti discussi in questa sezione, il workflow ideale dell'autore è:

```
WORKFLOW COMPLETO IDEALE:

FASE 0 - PREPARAZIONE (una tantum per progetto)
├─ Creare la struttura .claude con rules, skills, agents
├─ Configurare il CLAUDE.md con contesto e vincoli
├─ Attivare Autocompact, Thinking Mode, Checkpoint
└─ Installare MCP necessari (es. Chrome Dev Tool)

FASE 1 - PLAN MODE (60-80% del tempo totale)
├─ Scrivere il prompt completo con tutti i requisiti
├─ Includere riferimenti visivi (screenshot, immagini)
├─ Includere dati tecnici (API key, stili CSS)
├─ Revisionare il piano generato
├─ Iterare il piano fino a soddisfazione totale
├─ Porre domande su punti ambigui
└─ Approvare il piano finale

FASE 2 - BYPASS PERMISSION (20-40% del tempo totale)
├─ Attivare bypass permission
├─ Attivare auto-accept
├─ Claude Code esegue il piano in autonomia
├─ Monitorare periodicamente
├─ Hook sonoro per notifica di completamento
└─ Risultato finale (idealmente one-shot)

FASE 3 - VALIDAZIONE
├─ Chiamare il sub-agente Reviewer (zero-context code review)
├─ Chiamare il sub-agente QA (test automatici)
├─ Verificare manualmente le funzionalità critiche

└─ Fare deployment se tutto è ok
```

Questo workflow è il distillato dell'intera esperienza dell'autore e rappresenta il modo più efficiente di utilizzare Claude Code per progetti di qualsiasi complessità.

PARTE 6 — CONTEXT MANAGEMENT

"Il Context Management è una delle cose più importanti che ci sia per ottenere prompt efficaci e un elevato ROI da Claude Code."
— Dalla guida originale

Introduzione alla Parte 6

Il Context Management — ovvero la gestione del contesto — rappresenta il cuore pulsante dell'efficienza operativa in Claude Code. Se le Parti precedenti di questo manuale vi hanno insegnato *cosa* sia Claude Code e *come* si configuri un progetto, questa Parte vi insegna *come pensare* quando interagite con il sistema.

Ogni singola parola che inviate a Claude, ogni file che viene letto, ogni MCP installato, ogni sub-agente chiamato — tutto questo occupa spazio all'interno di una risorsa finita chiamata finestra di contesto. Chi comprende questa risorsa e la gestisce con intelligenza ottiene risultati straordinariamente migliori. Chi la ignora si ritrova con un sistema che "allucina", dimentica istruzioni, perde coerenza e brucia tempo e denaro.

Questa Parte è composta da quattro capitoli:

Capitolo	Titolo	Focus Principale
20	Comprendere Contesto e Token	Cosa sono, come funzionano, perché contano
21	Analisi e Monitoraggio del Contesto	Strumenti pratici per vedere dove va il contesto

22	Autocompact e Densità Informativa	Come Claude comprime e riutilizza lo spazio
23	Primacy Bias, Recency Bias e Lost in the Middle	Come l'LLM "ricorda" e come sfruttarlo

CAPITOLO 20

Comprendere Contesto e Token

20.1 — Introduzione al Concetto di Contesto

Definizione del Concetto

Quando si parla di contesto in Claude Code, ci si riferisce alla quantità totale di informazioni che il modello di linguaggio (LLM) è in grado di "tenere in mente" durante una singola sessione di lavoro. Immaginate il contesto come la memoria di lavoro di Claude: tutto ciò che il sistema sa, vede, ricorda e può utilizzare per rispondere ai vostri prompt in un dato momento.

Questa memoria di lavoro non è infinita. Ha una dimensione massima fissa, misurata in unità chiamate token. Una volta che questa memoria si riempie, il sistema deve fare delle scelte: comprimere informazioni precedenti, dimenticarne alcune o degradare nella qualità delle risposte.

Spiegazione Approfondita

Il contesto funziona esattamente come una scrivania fisica. Immaginate di avere una scrivania di dimensioni fisse:

- Potete appoggiarci sopra documenti, foto, appunti, strumenti
- Più cose ci mettete, meno spazio avete per lavorare
- Se la scrivania si riempie completamente, dovete togliere qualcosa per fare spazio a qualcosa di nuovo
- Se ammassate troppa roba, non riuscite più a trovare quello che vi serve

In Claude Code, questa "scrivania" contiene:

- Le istruzioni di sistema iniettate da Anthropic (il system prompt)
- Il vostro file CLAUDE.md e tutte le regole del progetto
- I file di memoria (memory.md)
- Le definizioni dei tool di sistema
- Le definizioni degli MCP installati
- Le skill caricate
- Tutta la vostra conversazione (ogni messaggio che avete scritto e ogni risposta di Claude)

Perché Questo Concetto è Fondamentale

Comprendere il contesto è la differenza tra usare Claude Code come un chatbot qualsiasi e usarlo come uno strumento professionale che genera valore reale. Senza questa comprensione:

- Non capirete perché Claude "dimentica" cose che gli avete detto
- Non capirete perché le risposte diventano meno accurate dopo conversazioni lunghe
- Non saprete come organizzare le informazioni per ottenere risultati migliori
- Sprecherete contesto prezioso con informazioni irrilevanti
- Non potrete fare scelte informate su cosa installare (MCP, skill, sub-agenti)

Interpretazione Pratica

A livello pratico, gestire il contesto significa:

1. Sapere quanto contesto è stato usato in ogni momento
2. Sapere da cosa è occupato (conversazione? MCP? system prompt?)
3. Decidere consapevolmente cosa aggiungere e cosa togliere
4. Scrivere prompt concisi che non sprechino spazio
5. Strutturare il progetto in modo da minimizzare l'occupazione di contesto inutile

Errori Comuni

Errore	Conseguenza	Soluzione
Ignorare completamente il contesto	L'LLM perde coerenza dopo pochi messaggi	Monitorare regolarmente con <code>/context</code>
Installare troppi MCP contemporaneamente	Il contesto si riempie prima ancora di iniziare a lavorare	Installare solo ciò che serve, convertire in skill
Scrivere prompt lunghissimi e ridondanti	Spreco di token preziosi	Aumentare la densità informativa dei prompt
Non fare mai compattazione	Il contesto si satura rapidamente	Usare <code>autocompact</code> o <code>/compact</code> manualmente

Mettere tutto nel CLAUDE.md principale	File monolitico che occupa troppo contesto	Spezzare in regole modulari nella cartella .claude
--	--	--

Insight Avanzato

Il contesto non è solo una questione di "capienza". È una questione di qualità cognitiva. Anche quando il contesto non è completamente pieno, la qualità delle risposte di Claude degrada in modo proporzionale alla quantità di informazioni presenti. Questo avviene perché il modello deve "distribuire la sua attenzione" su tutti gli elementi presenti nel contesto. Meno rumore c'è, più nitida è la risposta.

Pensate a questo come ascoltare una persona in una stanza silenziosa versus ascoltarla in un mercato affollato. In entrambi i casi "sentite", ma la qualità dell'ascolto è radicalmente diversa.

20.2 — Cosa Sono i Token

Definizione del Concetto

Un token è l'unità fondamentale con cui gli LLM misurano e processano il testo. In termini semplificati — come suggerisce la guida originale — potete pensare a un token come una parola. In realtà tecnicamente un token corrisponde a circa 3-4 lettere, ma per finalità pratiche e per questo corso, l'approssimazione "un token $\approx$ una parola" è sufficiente.

Spiegazione Approfondita

Quando voi scrivete un messaggio a Claude, quel messaggio viene scomposto in token prima di essere processato. Ad esempio:

- La frase "Ciao come stai" potrebbe essere scomposta in 3-4 token
- La frase "Per favore analizza il codice del mio progetto e trova i bug" potrebbe essere 10-12 token
- Un intero file di codice di 500 righe potrebbe essere migliaia di token

Il modello Claude ha un limite massimo di token per il contesto. A seconda del modello utilizzato, questo limite può essere:

- 200.000 token (200K) per le configurazioni standard
- 1.000.000 token (1M) per configurazioni specifiche

Questo numero rappresenta il totale di tutto ciò che può stare nella "scrivania" del modello: input vostro, output di Claude, istruzioni di sistema, MCP, skill, memoria — tutto insieme.

Il Meccanismo Sottostante

I token non corrispondono esattamente alle parole perché il sistema di tokenizzazione (il processo con cui il testo viene convertito in token) funziona diversamente dalla divisione per parole che facciamo noi umani:

Esempio di tokenizzazione approssimativa:

```
Testo: "Costruiamo un'applicazione web"
Token: ["Costru", "iamo", " un", "", "applic", "azione", " web"]
Risultato: ~7 token per 3 parole
```

```
Testo: "Hello world"
Token: ["Hello", " world"]
```

```
Risultato: ~2 token per 2 parole
```

Notate come le parole italiane tendono a consumare più token rispetto a quelle inglesi. Questo accade perché i modelli LLM sono stati addestrati prevalentemente su testo inglese, quindi il tokenizzatore è più "efficiente" con l'inglese.

Perché i Token Contano

I token sono la valuta di Claude Code. Ogni azione ha un costo in token:

Azione	Consumo Token Approssimativo
Un messaggio breve dell'utente	50-200 token
Una risposta breve di Claude	200-500 token
Lettura di un file di codice medio	1.000-5.000 token
System prompt di Anthropic	~10.000-20.000 token
Un MCP leggero (Chrome Dev Tool)	~200 token (0,1% del contesto)
Un MCP pesante (ClickUp)	~54.000 token (27% del contesto)
Le skill del progetto	~600 token (0,3% del contesto)

Interpretazione Pratica

Nella barra di stato di Claude Code (quella che avete configurato seguendo le istruzioni nella Parte 2 del manuale), vedrete due informazioni relative ai token:

1. Percentuale di contesto utilizzato: ad esempio "14% used" — questo indica quanto della finestra totale è occupato
2. Token totali disponibili: ad esempio "200K" — questo è il limite massimo
3. Costo API equivalente: se usaste il piano API al posto dell'abbonamento, questo sarebbe il costo effettivo della chiamata

Queste informazioni sono fondamentali per prendere decisioni in tempo reale durante il lavoro.

Nota Importante sui Piani

Se utilizzate il piano Pro (\$17/mese) o il piano Max (\$100/mese o superiore), il costo dei token è incluso nell'abbonamento. Il costo visualizzato nella barra di stato è puramente informativo — vi mostra quanto avreste speso se foste sul piano API. Tuttavia, anche con un piano in abbonamento, la gestione del contesto resta fondamentale perché la qualità delle risposte dipende da quanto efficientemente usate il contesto disponibile, non solo da quanto ne avete.

Errori Comuni

1. Pensare che "tanto ho token illimitati con l'abbonamento": l'abbonamento vi dà accesso illimitato alle chiamate, ma ogni singola sessione ha un limite di contesto fisso. Se lo riempite, le prestazioni degradano indipendentemente dal piano.
2. Non distinguere tra token di input e token di output: i token che voi scrivete (input) e quelli che Claude produce (output) occupano entrambi spazio nel contesto. Una risposta molto lunga di Claude consuma contesto esattamente come un prompt lungo vostro.
3. Ignorare il consumo "invisibile": prima ancora che voi scriviate il primo messaggio, una percentuale significativa del contesto è già occupata dal system prompt di Anthropic, dagli MCP e dalle configurazioni del progetto.

Insight Avanzato

Esiste una relazione diretta tra il numero di token nel contesto e la latenza (tempo di risposta) di Claude. Più token ci sono nel contesto, più tempo impiega Claude per elaborare una risposta, perché deve "leggere" e "considerare" tutto ciò che è presente. Questo significa che un contesto snello non solo produce risposte migliori, ma anche risposte più veloci. L'ottimizzazione del contesto è quindi un'ottimizzazione sia qualitativa che temporale.

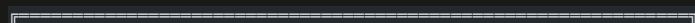
20.3 — La Composizione del Contesto in Claude Code

Definizione del Concetto

Il contesto in Claude Code non è composto solo dalla vostra conversazione. È un aggregato di molteplici fonti che vengono tutte caricate nella finestra di contesto prima e durante la sessione di lavoro. Comprendere questa composizione è essenziale per gestire il contesto in modo professionale.

Spiegazione Approfondita — Mappa Completa del Contesto

Dalla guida originale e dall'analisi pratica del comando `/context`, sappiamo che il contesto è composto dalle seguenti componenti, elencate nell'ordine in cui vengono caricate:



FINESTRA DI CONTESTO (es. 200.000 token)
1. SYSTEM PROMPT (iniettato da Anthropic) → Non modificabile dall'utente → Circa 10-12% del contesto
2. SYSTEM TOOLS → Comandi bash, read, write, edit, etc. → Definizioni dei tool disponibili
3. MCP TOOLS → Chrome Dev Tool, ClickUp, etc. → VARIABILE: da 0,1% a 27%+ del contesto
4. MEMORY FILES → memory.md → auto_memory.md → CLAUDE.md del progetto → Rules del workspace
5. SKILL → Generalmente molto leggere (~0,3%)
6. MESSAGGI (la vostra conversazione) → Input dell'utente + Output di Claude → Cresce con ogni scambio
7. AUTOCOMPACT BUFFER → ~33.000 token riservati → Spazio per la compattazione automatica
8. SPAZIO LIBERO → Disponibile per nuovi messaggi e operazioni

Il Problema del Contesto "Pre-Occupato"

Un dato fondamentale emerso dalla guida originale è questo: prima ancora di scrivere il vostro primo messaggio, una percentuale significativa del contesto è già occupata.

Nell'esempio pratico mostrato nella guida:

Componente	% del Contesto
------------	----------------

System Prompt (Anthropic)	~10%
System Tools	~0,7%
MCP Tools (solo Chrome Dev Tool)	~0,1%
Memory Files	~4-5%
Skill	~0,3%
Totale pre-occupato (configurazione leggera)	~15-16%

Ma quando vengono installati MCP pesanti:

Componente	% del Contesto
System Prompt (Anthropic)	~10%
System Tools	~0,7%
MCP Tools (Chrome Dev Tool + ClickUp + Canva)	~28%

Memory Files	~4-5%
Skill	~0,3%
Totale pre-occupato (configurazione pesante)	~43%

Questo significa che nel secondo caso, quasi metà del contesto è già occupata prima ancora di cominciare a lavorare. Restate con solo il 57% per la vostra effettiva conversazione e il lavoro produttivo.

Perché Questo è Critico

Immaginate di avere un serbatoio da 100 litri per un viaggio. Se prima di partire qualcuno ci mette dentro 43 litri di sabbia, vi restano solo 57 litri per la benzina. Arriverete molto meno lontano.

Lo stesso vale per il contesto. Se il 43% è occupato da MCP che forse non vi servono, avrete:

- Meno spazio per conversazioni complesse
- Compattazioni più frequenti (con perdita potenziale di informazioni)
- Qualità delle risposte degradata più rapidamente
- Sessioni di lavoro più corte prima di dover resettare

Interpretazione Pratica — Come Verificare

Per vedere la composizione esatta del vostro contesto in qualsiasi momento, utilizzate il comando:

```
/context
```

Questo comando produce un'analisi dettagliata che mostra:

- Ogni componente del contesto
- La percentuale occupata da ciascuno
- Il totale utilizzato
- Lo spazio libero rimanente

Questo è il vostro cruscotto di guida. Consultarlo regolarmente è un'abitudine che distingue l'utente esperto dal principiante.

Errori Comuni

1. Non controllare mai il contesto: molti utenti non sanno nemmeno che il comando `/context` esiste. Volano alla cieca.

2. Attribuire errori di Claude alla "stupidità del modello" quando in realtà il contesto è saturo: se Claude inizia a dimenticare istruzioni o a dare risposte incoerenti, la prima cosa da verificare è la percentuale di contesto utilizzata.
3. Non considerare il costo degli MCP: come visto, un singolo MCP come ClickUp può occupare il 27% del contesto. È un prezzo enorme da pagare se non lo state effettivamente usando.

Insight Avanzato

La percentuale di contesto utilizzato e la percentuale di contesto nella barra di stato non sono necessariamente uguali. La guida originale nota specificamente questa discrepanza. Il motivo è che la barra di stato mostra una stima semplificata, mentre il comando `/context` fornisce l'analisi granulare reale. Usate sempre `/context` per le decisioni importanti.

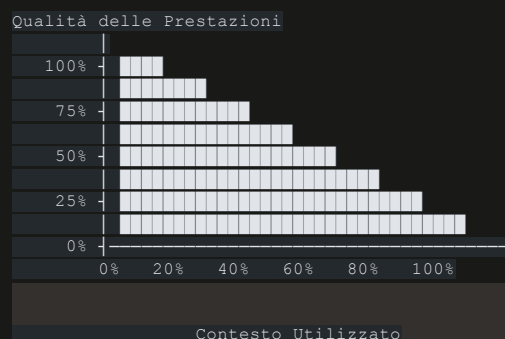
20.4 — La Relazione tra Contesto e Prestazioni

Definizione del Concetto

Esiste una relazione inversamente proporzionale tra la quantità di contesto occupato e la qualità delle prestazioni di Claude. Questa relazione non è lineare: le prestazioni non degradano in modo uniforme, ma seguono un pattern specifico legato ai bias cognitivi del modello.

Spiegazione Approfondita

La guida originale introduce un grafico concettuale che possiamo rappresentare così:



Le prestazioni diminuiscono man mano che il contesto si riempie, e la diminuzione accelera nella seconda metà. Questo significa che:

- Da 0% a 30% di contesto: prestazioni eccellenti
- Da 30% a 60% di contesto: prestazioni buone, leggero calo
- Da 60% a 80% di contesto: prestazioni in calo visibile
- Da 80% a 100% di contesto: prestazioni significativamente degradate

Implicazione Pratica Diretta

Quando nella guida l'autore vede che il contesto è al 66%, dice immediatamente a Claude: *"Sei al 66% del contesto, cosa che significa che comincerai a perderti a livello di performance."* E chiede di salvare le informazioni importanti in memoria per poter iniziare una nuova sessione pulita.

Questa è la strategia corretta: non aspettare che il contesto sia al 90% per reagire. Già al 60-70% è il momento di:

1. Compattare il contesto (`/compact`)
2. Salvare informazioni critiche nella memoria
3. Considerare di iniziare una nuova sessione
4. Fornire a Claude un prompt di continuazione per la sessione successiva

Perché le Prestazioni Degradano

Le prestazioni degradano per diverse ragioni tecniche che è utile comprendere almeno a livello intuitivo:

1. Dispersione dell'attenzione: il modello deve distribuire la sua "attenzione computazionale" su tutti i token presenti. Più token ci sono, meno attenzione viene data a ciascuno.
2. Conflitto di istruzioni: con più contesto, aumenta la probabilità che ci siano istruzioni contraddittorie o ambigue, causando incertezza nel modello.
3. Allucinazioni: quando il contesto è saturo, il modello è più propenso a "inventare" informazioni anziché ammettere di non sapere, perché ha troppi pattern tra cui scegliere.
4. Perdita di focus: le istruzioni iniziali (il `CLAUDE.md`, le regole) vengono "diluite" dalla massa di conversazione successiva.

CAPITOLO 21

Analisi e Monitoraggio del Contesto

21.1 — Il Comando `/context`

Definizione del Concetto

Il comando `/context` è lo strumento diagnostico primario per il Context Management in Claude Code. Digitando questo comando nel terminal, si ottiene un'analisi completa e granulare di come il contesto è attualmente distribuito tra le varie componenti.

Spiegazione Approfondita

Quando digitate `/context` nel terminal di Claude Code, il sistema produce un report che include:

1. System Prompt: la percentuale occupata dalle istruzioni iniettate da Anthropic
2. System Tools: la percentuale occupata dalle definizioni dei tool di sistema (`bash`, `read`, `write`, `edit`, etc.)
3. MCP Tools: la percentuale occupata da ogni MCP installato, elencato individualmente
4. Memory Files: la percentuale occupata dai file di memoria (`memory.md`, `auto_memory.md`, `CLAUDE.md`, `rules`, etc.)
5. Skill: la percentuale occupata dalle skill del progetto
6. Messages: la percentuale occupata dalla conversazione attuale
7. Autocompact Buffer: lo spazio riservato per la compattazione automatica
8. Spazio Libero: la percentuale ancora disponibile

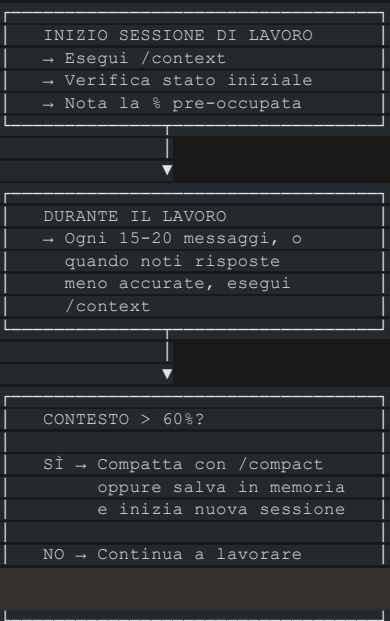
Come Leggere il Report

Quando vedete il report del contesto, la prima cosa da fare è identificare eventuali anomalie. Un'anomalia è qualsiasi componente che occupa una percentuale inaspettatamente alta. Ad esempio:

- Se gli MCP Tools occupano il 27%, avete un problema di contesto
- Se i Messages occupano il 60%, probabilmente è ora di compattare o iniziare una nuova sessione
- Se le Skill occupano più del 2-3%, potreste avere skill troppo verbose che necessitano di ottimizzazione

Interpretazione Pratica — Workflow di Monitoraggio

Il monitoraggio del contesto dovrebbe seguire questo workflow:



Perché Monitorare Regolarmente

Il monitoraggio regolare serve a tre scopi:

1. Prevenzione: identificare problemi di contesto prima che impattino la qualità del lavoro
2. Decisione informata: sapere se potete installare un nuovo MCP o se dovete prima liberare spazio
3. Apprendimento: con il tempo, svilupperete un'intuizione su come le diverse azioni impattano il contesto

Errori Comuni

1. Monitorare solo quando qualcosa va storto: a quel punto il danno è fatto. Il contesto è già saturo e le risposte sono degradate. Monitorate proattivamente.
2. Non sapere che il comando esiste: molti utenti di Claude Code non hanno mai digitato `/context`. È come guidare un'auto senza mai guardare il livello della benzina.
3. Confondere la barra di stato con l'analisi reale: la barra di stato in basso al terminal mostra una percentuale approssimativa. Il comando `/context` è l'unica fonte accurata.

21.2 — La Barra di Stato (Status Line)

Definizione del Concetto

La Status Line è una barra informativa posizionata nella parte inferiore del terminal di Claude Code che mostra in tempo reale informazioni chiave sulla sessione corrente: percentuale di contesto utilizzato, costo equivalente API, token totali disponibili e durata della sessione.

Spiegazione Approfondita

La Status Line mostra le seguenti informazioni:

Elemento	Significato	Esempio
% used	Percentuale del contesto utilizzata	14% used
Cost	Costo equivalente se si usasse il piano API	\$0.03
Token totali	Dimensione totale della finestra di contesto	200K
Durata sessione	Da quanto tempo è attiva la sessione	5m

Come Configurare la Status Line

Per ottenere la Status Line, esistono due metodi:

Metodo 1 — Comando diretto:

```
/status line
```

Premete Enter e la barra verrà configurata automaticamente.

Metodo 2 — Prompt a Claude Code:

Nel caso in cui il Metodo 1 non funzionasse, l'autore della guida suggerisce un approccio ingegnoso:

1. Fate uno screenshot della barra di stato che volete replicare (ad esempio dalla guida o da un video)
2. Incollatelo nel terminal
3. Dite a Claude: *"Per favore fai sì che io abbia sotto al terminal queste cose qui"*
4. Claude configurerà automaticamente la Status Line

Questa è un'applicazione pratica di un principio fondamentale: la vostra vita con Claude Code sarà sempre alla distanza di un buon prompt dal risolvere i vostri problemi.

Interpretazione Pratica

La Status Line è il vostro indicatore di carburante durante la guida. Non è precisa come il comando `/context` (che è il vostro cruscotto diagnostico completo), ma è sempre visibile e vi dà un'indicazione rapida di dove siete.

Abitudini corrette con la Status Line:

- Guardarla dopo ogni scambio significativo con Claude
- Preoccuparvi quando supera il 50-60%: è il momento di iniziare a pensare alla gestione del contesto
- Non ignorarla mai quando supera il 70%: a questo punto le azioni di gestione del contesto diventano urgenti

La Discrepanza tra Percentuale e Token

La guida originale nota che la percentuale mostrata nella Status Line e il numero di token "non sono uguali". Questo avviene perché:

- La percentuale è calcolata sul contesto totale disponibile
- Il numero di token potrebbe riferirsi al totale di token processati (input + output cumulativi), che può superare il contesto disponibile grazie alla compattazione
- La durata della sessione influenza questa discrepanza perché la compattazione ridistribuisce i token

La regola pratica è: fidatevi della percentuale per le decisioni operative. Se dice 60%, agite di conseguenza, indipendentemente dal numero assoluto di token mostrato.

21.3 — Il Comando `/config` e le Informazioni di Sistema

Definizione del Concetto

Il comando `/config` apre un pannello di configurazione completo che mostra tutte le impostazioni attive di Claude Code. Tra le informazioni disponibili in questo pannello, ci sono anche dati relativi alla gestione del contesto.

Spiegazione Approfondita

Quando digitate `/config` nel terminal, potete navigare tra diverse sezioni usando le frecce su/giù e il tasto Tab. Le sezioni principali sono:

Sezione Config:

- Autocompact: mostra se la compattazione automatica è attiva (on/off)
- Thinking Mode: mostra se la modalità di pensiero estensivo è attiva
- Rewind Checkpoint: mostra se i checkpoint di ripristino sono attivi
- Tema e personalizzazione visiva
- Teammate Mode e altre impostazioni avanzate

Sezione Usage (raggiungibile premendo Tab):

- Mostra quanto del modello è stato utilizzato nella sessione corrente
- Nell'esempio della guida: "28% utilizzato al momento"

Sezione Status (raggiungibile premendo Tab di nuovo):

- Mostra un riepilogo delle informazioni della Status Line

Autocompact nel Config

L'impostazione più rilevante per il Context Management nel pannello `/config` è Autocompact. Quando questa è impostata su "on":

- Claude compatta automaticamente il contesto quando raggiunge una certa soglia
- Non dovete fare nulla manualmente per attivare la compattazione
- Il sistema gestisce autonomamente la densità delle informazioni

Quando è su "off":

- Il contesto si riempie linearmente senza compressione
- Dovete usare manualmente `/compact` per liberare spazio
- Avete più controllo ma più responsabilità

Raccomandazione: tenete Autocompact su on. La gestione automatica è generalmente superiore a quella manuale per la maggior parte degli utenti. Potrete sempre usare `/compact` manualmente in aggiunta quando necessario.

Rewind Checkpoint nel Config

Questa funzione è rilevante per il Context Management perché i checkpoint occupano spazio. Quando è su "on":

- Claude salva dei punti di ripristino a cui potete tornare
- Potete dire "torna alla versione precedente" e Claude lo farà
- Questi checkpoint consumano una piccola quantità di contesto aggiuntiva

Quando è su "off":

- Non ci sono checkpoint
- Non potete fare rollback delle azioni
- Risparmiate una piccola quantità di contesto

Raccomandazione: tenete i checkpoint su on. Lo spazio che consumano è minimo rispetto al vantaggio di poter tornare indietro in caso di errore.

CAPITOLO 22

Auto Compact e Densità Informativa

22.1 — Il Concetto di Densità Informativa

Definizione del Concetto

La densità informativa è il rapporto tra la quantità di informazione utile contenuta in un testo e il numero di token utilizzati per esprimerla. Un testo ad alta densità informativa comunica molta informazione con pochi token. Un testo a bassa densità informativa spreca token con parole inutili, ripetizioni e informazioni irrilevanti.

Spiegazione Approfondita

La guida originale fornisce un esempio perfetto per comprendere questo concetto. Immaginate di scrivere questo prompt a Claude:

Prompt a bassa densità informativa:

"Ciao sono Giovanni ho 30 anni il mio compleanno il 27 di febbraio quindi qualche giorno fa mi piacciono le pentole ah no le pentole non c'entrano vivo a Lussemburgo e bla bla bla"

In questo prompt ci sono informazioni rilevanti e informazioni irrilevanti mescolate insieme:

Informazione	Rilevante?	Motivo
"Ciao"	No	Convenevole inutile per il contesto
"sono Giovanni"	Sì	Identità dell'utente
"ho 30 anni"	Sì	Dato anagrafico potenzialmente utile
"il mio compleanno il 27 di febbraio"	Sì	Dato specifico
"qualche giorno fa"	No	Deduzione inutile, spreca token
"mi piacciono le pentole"	No	Totalmente irrilevante
"ah no le pentole non c'entrano"	No	Correzione di un errore proprio, doppio spreco

"vivo a Lussemburgo"	Sì	Dato geografico
"bla bla bla"	No	Riempitivo

Prompt ad alta densità informativa (equivalente):

"Giovanni, 30 anni, Lussemburgo, compleanno 27 febbraio"

Stesse informazioni utili, un quinto dei token. Questo è il principio della densità informativa.

Il Meccanismo dell'Autocompact

L'Autocompact è la funzione di Claude Code che automatizza questo processo di aumento della densità informativa. Ecco come funziona:

PROCESSO DI AUTOCOMPACT

PRIMA della compattazione:

Utente: Ciao, mi chiamo Giovanni, vivo a Lussemburgo, ho 30 anni, il mio compleanno è il 27 febbraio, mi piacciono le pentole ah no le pentole non c'entrano...
Claude: Ciao Giovanni! Piacere di conoscerti! Come posso aiutarti oggi? Vedo che vivi a Lussemburgo, bellissimo posto! E auguri in ritardo per il tuo compleanno!
Utente: Grazie, allora vorrei...
[... altri 50 messaggi di conversazione ...]
CONSUMO: 85% del contesto

DOPO la compattazione:

• User: Giovanni, 30 anni, Lussemburgo, compleanno 27 febbraio
• Richiesta principale: [sintesi della task]
• Azioni completate: [lista bullet point]
• Stato attuale: [sintesi dello stato]
• Decisioni prese: [lista decisioni]
CONSUMO: 25% del contesto

Come potete vedere, il processo di compattazione:

- 1. Elimina tutte le parole ridondanti e i convenevoli
- 2. Sintetizza le conversazioni in bullet point ad alta densità
- 3. Preserva le informazioni chiave e le decisioni prese
- 4. Riduce drasticamente il consumo di contesto

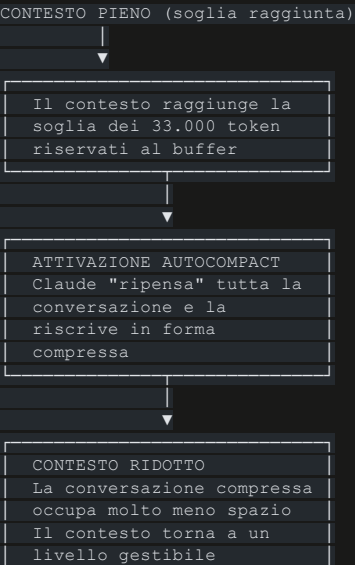
Perché l'Autocompact è Fondamentale

Senza Autocompact, ogni sessione di lavoro con Claude Code avrebbe una durata massima limitata dalla dimensione del contesto. Con conversazioni intensive, potreste esaurire il contesto in 20-30 messaggi. L'Autocompact vi permette di estendere le sessioni molto oltre questo limite, comprimendo periodicamente le informazioni accumulate.

Il Buffer di Autocompact

L'Autocompact utilizza un buffer riservato di circa 33.000 token. Questo buffer funziona come una zona di transito:

MECCANISMO DEL BUFFER



Quando il contesto raggiunge la soglia del buffer, Claude si prende un momento (noterete che "pensa" più a lungo del solito) e riscrive tutta la conversazione in formato compresso. Dopo questa operazione, il contesto torna a un livello più basso e potete continuare a lavorare.

Interpretazione Pratica — Il Formato Compattato

Dopo una compattazione, se esaminate il contesto compattato (visibile con il comando `/compact`), vedrete qualcosa del genere:

Esempio di contesto compattato:

- User message: importa i tre subagenti
- My action: creati reviewer, researcher, QA
- User message: chiama il reviewer subagent
- My action: review del codice, 8 fix applicati
- User message: chiama il QA subagent
- My action: test eseguiti, tutti passati
- Stato corrente: app funzionante, deploy pendente

Questo è ciò che la guida chiama formato "ad alta densità" — tutta la conversazione precedente condensata in bullet point essenziali.

22.2 — Il Comando /compact

Definizione del Concetto

Il comando `/compact` è lo strumento manuale per attivare la compattazione del contesto. A differenza dell'Autocompact (che si attiva automaticamente a una certa soglia), `/compact` può essere eseguito in qualsiasi momento per forzare una compressione della conversazione.

Spiegazione Approfondita

Quando digitate `/compact` nel terminal:

1. Claude analizza tutta la conversazione presente nel contesto
2. Identifica le informazioni essenziali: decisioni prese, azioni completate, stato attuale, istruzioni pendenti
3. Riscrive tutto in formato bullet point ad alta densità
4. Sostituisce la conversazione originale con la versione compressa
5. Il contesto si riduce significativamente

Quando Usare /compact Manualmente

Situazione	Azione Raccomandata
Contesto sopra il 60% e dovete continuare a lavorare	Eseguite <code>/compact</code>
State per dare un prompt complesso che richiede molto contesto	Eseguite <code>/compact</code> prima
Claude inizia a "dimenticare" cose dette in precedenza	Probabilmente il contesto è saturo, eseguite <code>/compact</code>

State per chiamare un sub-agente che produrrà molto output	Liberate spazio prima con <code>/compact</code>
Volete iniziare una fase nuova del progetto nella stessa sessione	Eseguite <code>/compact</code> per "pulire" la fase precedente

Differenza tra `/compact` e Nuova Sessione

Una domanda che molti utenti si pongono è: "È meglio compattare o iniziare una nuova sessione?" La risposta dipende dalla situazione:

Usate `/compact` quando:

- Il lavoro è continuo e avete bisogno del contesto precedente
- Siete a metà di un'implementazione
- Le informazioni accumulate sono ancora rilevanti

Iniziate una nuova sessione quando:

- Passate a un argomento o task completamente diverso
- Il contesto è oltre l'80% anche dopo compattazione
- Volete un "foglio bianco" per una nuova fase

Strategia combinata (raccomandata dall'autore della guida):

1. Quando il contesto supera il 65-70%
2. Dite a Claude: *"Salva in memoria le informazioni importanti per la prossima sessione"*
3. Claude scrive le informazioni nel `memory.md`
4. Iniziate una nuova sessione
5. Nella nuova sessione, Claude recupera automaticamente le informazioni salvate in memoria
6. Continuate il lavoro con un contesto fresco

Questo è esattamente ciò che l'autore della guida fa nel video quando è al 66% di contesto e deve ancora integrare Stripe nell'applicazione Trello.

Errori Comuni

1. Usare `/compact` troppo frequentemente: ogni compattazione perde una piccola quantità di sfumature e dettagli. Se compattate ogni 5 messaggi, state perdendo troppe informazioni.
2. Non usare `/compact` mai: l'estremo opposto. Se lasciate che il contesto si saturi senza mai intervenire, la qualità degraderà inesorabilmente.
3. Compattare e poi ripetere le stesse informazioni: se dopo `/compact` riscrivete tutto quello che avevate detto prima, avete annullato il vantaggio della compattazione.
4. Aspettarsi che la compattazione preservi tutto: la compattazione è una lossy compression — una compressione con perdita. Le informazioni principali vengono preservate, ma i dettagli minori potrebbero andare persi. Per questo è importante salvare informazioni critiche nel `memory.md` prima di compattare.

Insight Avanzato

La qualità della compattazione dipende dalla qualità della vostra conversazione. Se avete scritto prompt chiari, strutturati e con informazioni ben organizzate, la compattazione produrrà un riassunto eccellente. Se la vostra

conversazione è caotica, piena di ripensamenti e deviazioni, la compattazione potrebbe perdere informazioni importanti perché non riesce a distinguerle dal rumore.

Questo crea un circolo virtuoso: scrivere prompt migliori → compattazione più efficace → più contesto disponibile → risposte migliori → workflow più efficiente.

22.3 — Strategie Pratiche per l'Efficienza del Contesto

La Lista delle Cose da Fare e Non Fare

La guida originale fornisce un "80/20" delle best practice per il Context Management. Espandiamo ciascuna:

✓ COSE DA FARE:

1. Lanciare `/init` quando si inizia un nuovo progetto

Il comando `/init` crea automaticamente un `CLAUDE.md` strutturato secondo le best practice. Questo assicura che il contesto iniziale sia organizzato in modo efficiente. Un `CLAUDE.md` ben strutturato consuma meno contesto di uno caotico per la stessa quantità di informazioni.

2. Mettere le regole più importanti in cima al `CLAUDE.md`

Questo è legato direttamente al fenomeno del Primacy Bias che tratteremo nel Capitolo 23. Le regole posizionate all'inizio del `CLAUDE.md` vengono "ricordate" meglio dall'LLM. Esempi di regole critiche da mettere in cima:

- "Non cancellare mai il file X"
- "Non rimuovere le API key"
- "Non modificare il database di produzione"
- "Chiedi sempre conferma prima di cancellare file"

3. Tenere sotto controllo il file `CLAUDE.md` e sfoltirlo regolarmente

Il `CLAUDE.md` tende a crescere nel tempo man mano che aggiungete regole e istruzioni. Periodicamente:

- Rimuovete regole che non sono più rilevanti
- Consolidate regole simili
- Spostate regole specifiche nelle sotto-cartelle (rules/)
- Verificate che non ci siano ripetizioni

4. Spezzare il `CLAUDE.md` in regole modulari

Come spiegato nelle Parti precedenti del manuale, invece di avere un unico `CLAUDE.md` monolitico, è molto più efficiente spezzare le regole in file separati nella cartella `.claude/rules/`:

Prima (inefficiente):
`CLAUDE.md` → un unico file enorme che viene caricato per intero

Dopo (efficiente):
`CLAUDE.md` → piccolo, solo regole essenziali
`.claude/rules/design-fidelity.md` → regole di design
`.claude/rules/security.md` → regole di sicurezza

`.claude/rules/screenshot-workflow.md` → regole per gli screenshot

Il vantaggio è che Claude carica solo le regole rilevanti per la task corrente, non tutte le regole di tutti gli aspetti del progetto.

5. Inserire errori ricorrenti direttamente nel CLAUDE.md

Quando Claude continua a fare lo stesso errore nonostante le vostre correzioni, la soluzione è codificarlo nel CLAUDE.md:

"Se Claude te lo fa due-tre volte, inseriscilo nel CLAUDE.md. Lo fai hard code dentro e hai risolto i tuoi problemi."

Questo trasforma un problema ricorrente (che consuma contesto ogni volta che dovete correggerlo) in una regola permanente (che previene il problema alla fonte).

6. Raccogliere regolarmente le best practice aggiornate

L'autore della guida condivide la sua strategia personale per rimanere aggiornato:

- Va su X (Twitter)
- Segue Boris (il fondatore di Claude Code) e altri power user
- Usa Grok per sintetizzare le novità: *"Per favore raccoglimi tutto quello che è successo nell'ultimo mese in termini di best practice di Claude e riassumilo così che io possa inglobarle dentro al mio progetto"*
- Incorpora le nuove best practice nel proprio CLAUDE.md

❌ COSE DA NON FARE:

1. Non buttate dentro guide inutili o documentazioni API

"Le API sono legate al vostro conto in banca."

Caricare documentazioni API complete, guide di terze parti o blocchi enormi di testo nel contesto è uno spreco catastrofico. Questi documenti possono consumare decine di migliaia di token senza fornire valore proporzionale. Se avete bisogno di informazioni da una documentazione, chiedete a Claude di cercarle specificamente (usando il Dev Tool MCP o un sub-agente researcher) invece di caricare tutto il documento.

2. Non scrivete regole vaghe o aspirazionali

Regole come:

- ❌ *"Non fare errori"* — è inutile e vaga
- ❌ *"Fammi diventare ricco"* — non è un'istruzione operativa
- ❌ *"Scrivi codice perfetto"* — non definisce cosa significhi "perfetto"

Regole efficaci sono:

- ✅ *"Quando modifichi un file, crea sempre un backup prima"*
- ✅ *"Ogni funzione deve avere una documentazione inline"*
- ✅ *"Dopo ogni modifica CSS, fai uno screenshot e confronta con il design di riferimento"*

Le regole vaghe non solo non aiutano Claude, ma consumano contesto senza produrre valore.

3. Non sprecate contesto con il ciclo di verifica umano quando potete automatizzarlo

Il pattern inefficiente è:

```
Utente: "Fai X"  
Claude: [fa X]  
Utente: "Hai fatto bene?"  
Claude: "Sì"  
Utente: "Sei sicuro?"
```

Claude: "Sì, sono sicuro"

Ogni messaggio "Hai fatto bene?" e "Sei sicuro?" è uno spreco di contesto. Invece, codificate la verifica nel CLAUDE.md: "Dopo ogni modifica, verifica automaticamente il risultato confrontandolo con il riferimento. Se ci sono differenze, correggi e ripeti."

CAPITOLO 23

Primacy Bias, Recency Bias e Lost in the Middle

23.1 — Introduzione ai Bias Cognitivi degli LLM

Definizione del Concetto

I bias cognitivi degli LLM sono pattern sistematici nel modo in cui i modelli di linguaggio processano e "ricordano" le informazioni all'interno del contesto. Esattamente come gli esseri umani hanno bias cognitivi che influenzano la memoria e il giudizio, anche gli LLM presentano pattern prevedibili che dobbiamo conoscere e sfruttare.

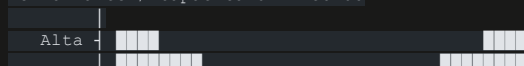
Spiegazione Approfondita

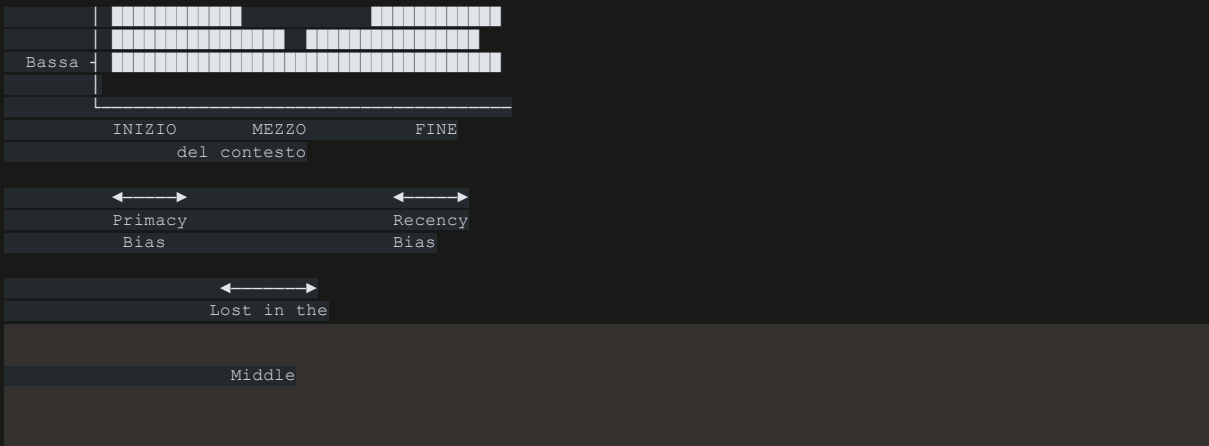
La guida originale introduce tre concetti fondamentali che rappresentano il cuore del Context Management avanzato:

1. Primacy Bias (Bias di Primato): il modello ricorda molto bene le informazioni posizionate all'inizio del contesto
2. Recency Bias (Bias di Recenza): il modello ricorda molto bene le informazioni posizionate alla fine del contesto
3. Lost in the Middle (Perso nel Mezzo): il modello ha difficoltà significative a ricordare e utilizzare le informazioni posizionate nel mezzo del contesto

Questi tre fenomeni insieme creano una curva di "attenzione" del modello che ha una forma caratteristica a U:

Performance / Capacità di Ricordo





23.2 — Il Primacy Bias (Bias di Primato)

Definizione del Concetto

Il Primacy Bias è la tendenza degli LLM a dare maggiore peso e attenzione alle informazioni che appaiono all'inizio del contesto. Proprio come gli esseri umani tendono a ricordare meglio le prime informazioni ricevute (il "primo impatto"), gli LLM processano le istruzioni iniziali con maggiore fedeltà.

Spiegazione Approfondita

Questo bias ha implicazioni enormi per come strutturate il vostro CLAUDE.md e i vostri prompt. Le istruzioni posizionate all'inizio del contesto:

- Vengono seguite con maggiore coerenza
- Vengono rispettate anche quando il contesto si riempie
- Hanno un effetto più duraturo sulla sessione di lavoro
- Resistono meglio alla "diluizione" causata da messaggi successivi

Applicazione Pratica Diretta

Regola d'oro del Primacy Bias:

Mettete le regole più importanti PER PRIME nel CLAUDE.md.

Esempi concreti:

in Markdown

```
# CLAUDE.md — Struttura Ottimizzata per Primacy Bias

## REGOLE CRITICHE (INIZIO = massima attenzione)
- NON cancellare MAI il file .env
- NON rimuovere MAI le API key dal codice
- NON modificare MAI il database di produzione
- Chiedi SEMPRE conferma prima di eliminare file

## Regole Operative (mezzo = attenzione moderata)
- Usa TypeScript per tutti i nuovi file
- Segui le convenzioni ESLint del progetto
- Scrivi commenti in italiano

## Preferenze Stilistiche (fine = attenzione elevata grazie al Recency)
- Preferisci componenti funzionali a classi
```

- Usa Tailwind CSS per lo styling

Notate come le regole critiche per la sicurezza sono posizionate all'inizio, dove il Primacy Bias garantisce la massima attenzione. Le regole operative sono nel mezzo (dove saranno meno "ricordate" ma non sono critiche se occasionalmente ignorate). Le preferenze stilistiche sono alla fine, dove il Recency Bias le mantiene presenti.

L'Analogia della Lista della Spesa

L'autore della guida usa un'analogia perfetta. Dice di avere "una memoria personale di un pesce rosso" e racconta:

Immaginate che qualcuno vi dica: "Per favore vai al supermercato alle 6 e compra: biscotti, latte, pane, uova, burro, farina, zucchero, sale, olio, aceto."

La maggior parte delle persone ricorderà:

- Biscotti (primo elemento — Primacy Bias) ✓
- Aceto (ultimo elemento — Recency Bias) ✓
- Pochi o nessuno degli elementi nel mezzo ✗

Lo stesso identico pattern si applica agli LLM.

Errori Comuni

1. Mettere regole critiche nel mezzo del CLAUDE.md: è il peggior posto possibile. Saranno le prime a essere "dimenticate" quando il contesto si riempie.
2. Mettere disclaimer o introduzioni lunghe all'inizio: se iniziate il CLAUDE.md con tre paragrafi di spiegazione generale del progetto, state "sprecando" la posizione più pregiata (l'inizio) per informazioni a basso impatto.
3. Non ordinare le regole per importanza: molti utenti scrivono le regole nell'ordine in cui le pensano, non nell'ordine di importanza. Ristrutturate il CLAUDE.md mettendo le regole più critiche prima.

23.3 — Il Recency Bias (Bias di Recenza)

Definizione del Concetto

Il Recency Bias è la tendenza degli LLM a dare maggiore peso alle informazioni che appaiono alla fine del contesto — cioè le informazioni più recenti nella conversazione. L'ultimo messaggio che inviate, le ultime istruzioni che date, le ultime informazioni che condividete avranno un impatto sproporzionatamente elevato sulla risposta di Claude.

Spiegazione Approfondita

Il Recency Bias funziona a vostro vantaggio in diversi modi:

1. Correzioni immediate funzionano bene: se Claude fa un errore e lo correggete immediatamente, la correzione (essendo l'informazione più recente) viene rispettata con alta fedeltà.
2. L'ultimo prompt è il più influente: se dovete dare un'istruzione critica, fatelo nell'ultimo messaggio prima che Claude inizi a lavorare.
3. Le decisioni recenti prevalgono: se all'inizio della conversazione avete detto "usa il colore blu" e alla fine dite "usa il colore rosso", Claude userà il rosso (Recency Bias) a meno che la regola del blu non sia nel CLAUDE.md (Primacy Bias del system prompt).

Applicazione Pratica

Potete sfruttare il Recency Bias in modo strategico:

Strategia "Rinforzo Finale":

Prima di dare un comando complesso a Claude, ripetete le istruzioni più importanti nell'ultimo messaggio:

```
Utente: "Costruisci il componente di autenticazione.  
RICORDA:  
- usa Supabase come backend  
- email + password, NO magic link  
- salva nome e email nel database
```

```
Procedi."
```

Le tre istruzioni dopo "RICORDA" sono posizionate alla fine del prompt (Recency Bias), quindi Claude le seguirà con maggiore fedeltà.

Strategia "Prompt di Continuazione":

Quando salvate informazioni in memoria per continuare in una nuova sessione, l'autore della guida mostra che Claude produce un "prompt di continuazione" — un messaggio predefinito da usare nella sessione successiva. Questo prompt, essendo il primo messaggio della nuova sessione, beneficerà del Primacy Bias nella nuova sessione E del Recency Bias (perché è l'ultimo contesto significativo salvato).

23.4 — Lost in the Middle — Il Fenomeno della Zona Cieca

Definizione del Concetto

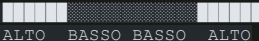
Lost in the Middle è un fenomeno documentato nella ricerca accademica sugli LLM secondo il quale le informazioni posizionate nella parte centrale del contesto vengono processate con minore attenzione e fedeltà rispetto a quelle posizionate all'inizio o alla fine. È la "zona cieca" del modello.

Spiegazione Approfondita

L'autore della guida fa riferimento a un documento di ricerca specifico chiamato "Lost in the Middle" che ha studiato questo fenomeno in modo rigoroso. Il risultato chiave è:

MODELLO DI ATTENZIONE DELL'LLM

Posizione nel contesto: INIZIO —————> FINE

Livello di attenzione: 
ALTO BASSO BASSO ALTO

Qualità delle risposte: 
basate su info in questa ALTA BASSA BASSA ALTA

posizione:

Questo fenomeno spiega perché:

- Le istruzioni nel CLAUDE.md (inizio del contesto) vengono generalmente rispettate bene
- I vostri ultimi messaggi (fine del contesto) ricevono risposte accurate
- Le istruzioni date a metà di una lunga conversazione vengono spesso "dimenticate" o ignorate

Implicazioni per il Context Management

La comprensione del Lost in the Middle ha implicazioni profonde per come strutturare il vostro lavoro:

1. Struttura del CLAUDE.md:

```
INIZIO (Primacy Bias — massima attenzione)  
├─ Regole di sicurezza critiche  
├─ Vincoli inviolabili
```

```
|— Istruzioni fondamentali del progetto
|
MEZZO (Lost in the Middle - minima attenzione)
|— Dettagli operativi secondari
|— Preferenze minori
|— Informazioni di contesto generiche
|
FINE (Recency Bias - alta attenzione)
|— Istruzioni operative correnti
|— Standard di qualità

|— Formato di output desiderato
```

2. Strategia di conversazione:

Se dovete dare un'istruzione importante nel mezzo di una conversazione lunga, ripetetela. Ditela una volta quando la pensate (sarà nel mezzo), e poi ripetetela prima del prompt operativo (sarà alla fine). In questo modo, coprite sia la posizione originale che la posizione di Recency.

3. Importanza delle regole modulari:

Questo fenomeno è un'altra ragione per cui è fondamentale spezzare il CLAUDE.md in regole modulari. Quando Claude deve seguire una regola specifica (ad esempio `design-fidelity.md`), carica solo quel file. Quel file, essendo piccolo, non soffre del Lost in the Middle perché non c'è abbastanza "mezzo" da perdersi.

Confronto:

- CLAUDE.md monolitico (5.000 token): le regole nel mezzo (token 1.500-3.500) saranno nella zona cieca
- Regola modulare (300 token): l'intero file è abbastanza corto che Claude lo processa interamente con alta attenzione

4. L'effetto amplificato dalla distanza:

L'autore della guida usa l'analogia dell'arco e del bersaglio per illustrare un concetto correlato: man mano che il progetto diventa più grande (la distanza dal bersaglio aumenta), l'intervallo di incertezza si amplifica. Lo stesso vale per il Lost in the Middle: man mano che il contesto si riempie, la "zona cieca" nel mezzo diventa proporzionalmente più grande e più problematica.

```
CONTESTO PICCOLO (20% pieno):
■■■ ■■■ Zona cieca piccola e gestibile

CONTESTO MEDIO (50% pieno):
■■■■ ■■■■ Zona cieca più grande

CONTESTO GRANDE (80% pieno):
■■■■■ ■■■■■■ Zona cieca estesa e pericolosa
```

Errori Comuni

1. Dare istruzioni critiche una sola volta a metà conversazione: se date un'istruzione importante al messaggio #15 di una conversazione di 40 messaggi, al messaggio #30 Claude potrebbe averla completamente "dimenticata".
2. Pensare che Claude "faccia apposta" a ignorare istruzioni: non è malizia, è un bias cognitivo del modello. La soluzione non è arrabbiarsi, ma posizionare strategicamente le informazioni.
3. Non ripetere mai le istruzioni importanti: la ripetizione strategica è uno strumento legittimo e importante nel prompt engineering. Non è ridondanza — è resilienza informativa.
4. Caricare documenti enormi nel contesto: un documento di 50.000 token avrà una zona cieca enorme nel mezzo. Claude "vedrà" bene l'inizio e la fine del documento, ma perderà dettagli cruciali nel mezzo. Meglio spezzare il documento in sezioni più piccole e caricarle una alla volta.

Insight Avanzato

Il Lost in the Middle è uno dei motivi principali per cui i sub-agenti sono così preziosi per il Context Management. Quando un sub-agente (ad esempio il Researcher) fa una ricerca che produce 100.000 token di risultati, quei token

vivono nel contesto del sub-agente, non nel contesto principale. Il sub-agente, avendo processato tutto quel materiale, produce un riassunto di 2.000 token che viene inviato all'agente principale.

Questo elimina il problema del Lost in the Middle perché:

- Nel sub-agente: il documento lungo soffre del Lost in the Middle, ma il sub-agente ha come unico compito processarlo, quindi può iterare e verificare
- Nell'agente principale: arrivano solo 2.000 token di risultato, che non creano alcuna zona cieca

È come avere un assistente che legge un libro di 500 pagine e vi fa un riassunto di 5 pagine. Voi leggete 5 pagine (nessuna zona cieca), non 500 (zona cieca enorme).

23.5 — Strategia Integrata di Context Management

Definizione del Concetto

La Strategia Integrata di Context Management è l'applicazione combinata di tutti i principi discussi in questa Parte del manuale per massimizzare l'efficienza e la qualità del lavoro con Claude Code.

Il Framework Completo

Combinando tutti i concetti appresi, ecco il framework completo per la gestione del contesto:

FRAMEWORK DI CONTEXT MANAGEMENT

FASE 1: PRIMA DI INIZIARE

- Verificare gli MCP installati → rimuovere quelli non necessari
- Verificare che il CLAUDE.md sia conciso e ben strutturato
- Verificare che le regole siano spezzate in file modulari
- Eseguire /context per vedere lo stato iniziale
- Obiettivo: tenere il contesto pre-occupato sotto il 20%

FASE 2: STRUTTURA DEL CLAUDE.MD

- Regole critiche (sicurezza, vincoli) → INIZIO
- Dettagli operativi → MEZZO
- Standard di qualità e formato → FINE
- Tutto il resto → file modulari in .claude/rules/

FASE 3: DURANTE IL LAVORO

- Scrivere prompt concisi ad alta densità informativa
- Monitorare il contesto con /context ogni 15-20 messaggi
- Compattare con /compact quando si supera il 60%
- Ripetere istruzioni importanti prima dei comandi critici
- Non caricare documenti enormi → usare sub-agenti

FASE 4: GESTIONE DELLE TRANSIZIONI

- Al 65-70% di contesto → salvare in memoria
- Chiedere a Claude il prompt di continuazione
- Iniziare nuova sessione con contesto fresco
- Nella nuova sessione: "Continua con [task]"

FASE 5: OTTIMIZZAZIONE CONTINUA

- Errori ricorrenti → codificarli nel CLAUDE.md
- MCP pesanti usati una volta → convertire in skill
- Regole obsolete → rimuovere dal CLAUDE.md

- Raccogliere nuove best practice mensilmente

Riepilogo delle Metriche Chiave

Metrica	Soglia Verde	Soglia Gialla	Soglia Rossa
Contesto pre-occupato	< 20%	20-35%	> 35%
Contesto durante il lavoro	< 50%	50-70%	> 70%
Skill nel contesto	< 1%	1-3%	> 3%
MCP nel contesto	< 5%	5-15%	> 15%
Messaggi nel contesto	< 40%	40-60%	> 60%

Il Principio Fondamentale

Se dovessimo riassumere l'intero Context Management in una singola frase, sarebbe questa:

Ogni token nel contesto deve guadagnarsi il suo posto. Se un'informazione non contribuisce direttamente alla qualità del risultato, non dovrebbe essere nel contesto.

Questo principio guida ogni decisione: cosa installare, cosa scrivere nel CLAUDE.md, come strutturare i prompt, quando compattare, quando iniziare una nuova sessione. È il metro con cui misurare ogni azione che impatta il contesto.

Riepilogo della Parte 6

In questa Parte avete appreso:

1. Cosa sono i token e come funzionano come unità di misura del contesto
2. Come è composto il contesto in Claude Code (system prompt, tools, MCP, memoria, skill, messaggi, buffer)
3. Come monitorare il contesto con `/context`, `/config` e la Status Line

- 4. Come funziona l'Autocompact e come usare `/compact` manualmente
- 5. Il concetto di densità informativa e come scrivere prompt efficienti
- 6. Il Primacy Bias e perché le regole importanti vanno all'inizio
- 7. Il Recency Bias e come sfruttare la posizione finale per istruzioni critiche
- 8. Il Lost in the Middle e perché le informazioni nel mezzo del contesto vengono "dimenticate"
- 9. La strategia integrata che combina tutti questi principi in un framework operativo

Questa conoscenza è la base su cui si costruisce l'uso professionale di Claude Code. Senza Context Management, state usando Claude Code come un chatbot. Con il Context Management, lo state usando come uno strumento di produttività professionale.

PARTE 7 — SUB-AGENTI E AGENT TEAMS

"Il motivo per cui utilizziamo questi agenti è perché possiamo tenere la task principale nel main agent ma poi andiamo ad utilizzare questi sub-agenti per riuscire ad efficientare l'utilizzo del contesto all'interno di Claude Code."
— Dalla guida originale

Introduzione alla Parte 7

Se la Parte 6 vi ha insegnato a comprendere e gestire il contesto come risorsa finita, questa Parte vi insegna come moltiplicare la vostra capacità operativa senza saturare quella risorsa. I sub-agenti e gli Agent Teams rappresentano l'evoluzione dall'uso individuale di Claude Code all'uso orchestrato: passate da essere un singolo operatore che lavora con un singolo assistente a diventare un direttore d'orchestra che coordina un intero team di intelligenze specializzate.

Questa Parte è composta da tre capitoli:

Capitolo	Titolo	Focus Principale
24	I Sub-agenti — Researcher, Reviewer, QA	Cosa sono, come crearli, perché usare questi tre

25	Agent Teams — Collaborazione Multi-Agente	Come funzionano i team, la comunicazione bidirezionale
26	Costi, ROI e Utilizzo Strategico	Quando conviene usarli e quando no

CAPITOLO 24

I Sub-agenti — Researcher, Reviewer, QA

24.1 — Cosa Sono i Sub-agenti

Definizione del Concetto

Un sub-agente (o sottoagente) è un'istanza separata di Claude che viene invocata dall'agente principale per svolgere una task specifica in modo isolato. Il sub-agente opera nel proprio contesto indipendente, esegue il compito assegnatogli e restituisce solo il risultato finale all'agente principale. È come delegare un compito a un collaboratore che lavora in una stanza separata e vi porta solo il prodotto finito.

Spiegazione Approfondita

Per comprendere i sub-agenti, è fondamentale capire la differenza tra lavorare tutto in un unico contesto e delegare a contesti separati.

Scenario SENZA sub-agenti (tutto nel contesto principale):

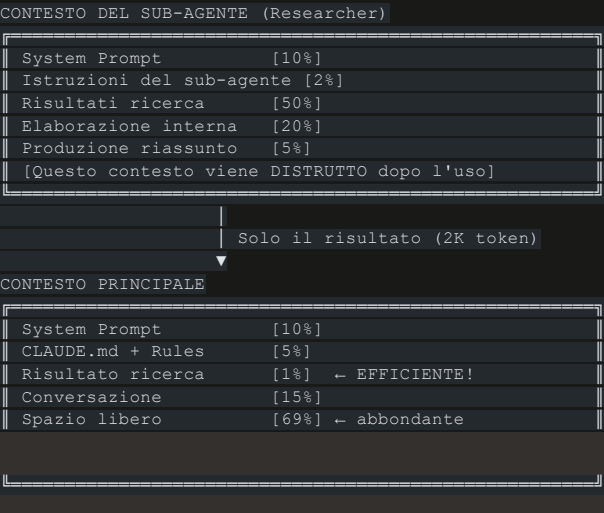
Immaginate di chiedere a Claude di fare una ricerca sulle best practice per costruire un sito internet. Claude naviga il web, legge documentazione, analizza articoli — e tutto questo materiale (potenzialmente 100.000 token) viene caricato nel vostro contesto principale. Risultato: il contesto si riempie a metà solo per una ricerca, e vi resta poco spazio per il lavoro vero e proprio.

CONTESTO PRINCIPALE (senza sub-agenti)

System Prompt	[10%]
CLAUDE.md + Rules	[5%]
Risultati ricerca	[50%] ← PROBLEMA!
Conversazione	[15%]
Spazio libero	[20%] ← troppo poco

Scenario CON sub-agenti (contesti separati):

La stessa ricerca viene delegata a un sub-agente Researcher. Questo sub-agente opera nel proprio contesto da 200.000 token, fa tutta la ricerca, processa 100.000 token di materiale e produce un riassunto di 2.000 token. Solo questi 2.000 token vengono inviati al contesto principale.

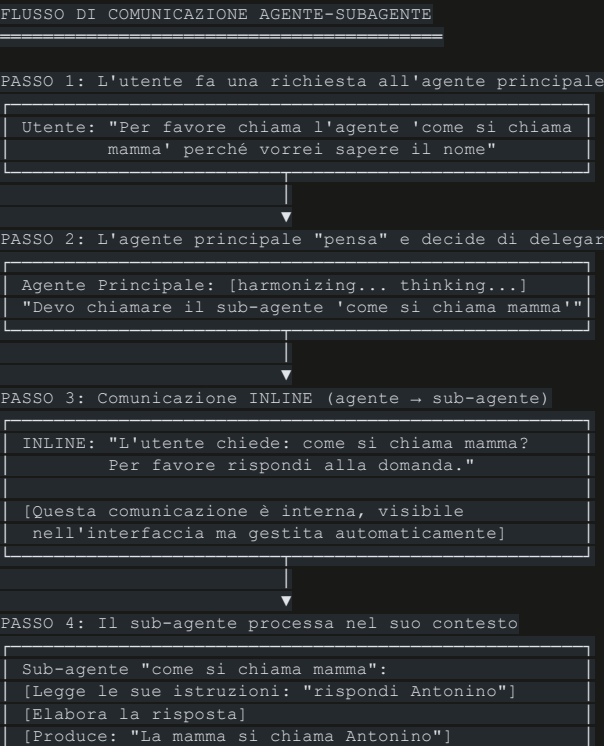


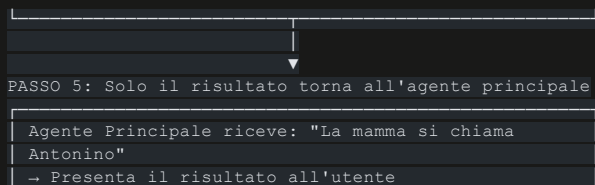
La differenza è drammatica: da 50% di contesto occupato a 1%. Questo è il potere dei sub-agenti.

Il Meccanismo Sottostante — Come Funziona la Comunicazione

La guida originale mostra un concetto chiave: la comunicazione tra agente principale e sub-agente è visibile nell'interfaccia attraverso quello che viene chiamato "inline".

Ecco il flusso completo:





Il punto cruciale è il Passo 5: l'utente non vede mai il contesto interno del sub-agente. Non vede i ragionamenti, non vede i file letti, non vede le iterazioni interne. Riceve solo il risultato. Questo è ciò che rende i sub-agenti così efficienti per il context management.

Perché i Sub-agenti Sono Fondamentali

I sub-agenti risolvono simultaneamente tre problemi:

1. Problema di contesto: delegare il lavoro pesante fuori dal contesto principale
2. Problema di specializzazione: ogni sub-agente può avere istruzioni specifiche per un tipo di task
3. Problema di parallelizzazione: più sub-agenti possono lavorare contemporaneamente su task diverse

Errori Comuni

Errore	Spiegazione	Conseguenza
Non usare sub-agenti per ricerche	Tutto il materiale di ricerca finisce nel contesto principale	Contesto saturo dopo una sola ricerca
Creare sub-agenti troppo generici	Un sub-agente senza istruzioni specifiche non è meglio dell'agente principale	Risultati generici e poco utili
Non sapere che i sub-agenti esistono	Molti utenti lavorano sempre e solo nel contesto principale	Efficienza drasticamente ridotta
Usare sub-agenti per task che richiedono il contesto del progetto	Il sub-agente non ha accesso al contesto principale	Risultati decontestualizzati

24.2 — Come Creare un Sub-agente

Definizione del Concetto

Un sub-agente è, nella sua essenza, un file di testo in formato Markdown posizionato nella cartella `.claude/agents/` del progetto. Questo file contiene le istruzioni che definiscono il comportamento, le competenze e i limiti del sub-agente.

Spiegazione Approfondita — La Struttura del File

Quando create un sub-agente, il file Markdown contiene diverse sezioni delimitate da separatori (righe tratteggiate `---`). La guida originale mostra che un file sub-agente tipico include:

```
in Markdown

---
model: haiku
max_tokens: [limite di token per le risposte]
---

# Nome del Sub-agente

## Descrizione
[Cosa fa questo sub-agente]

## Istruzioni
[Come deve comportarsi]

## Vincoli
[Cosa NON deve fare]

## Formato di Output

[Come deve presentare i risultati]
```

Analisi della struttura:

La sezione tra i tratteggi (`---`) è chiamata frontmatter e contiene metadati tecnici:

- `model:` specifica quale modello LLM utilizzare per questo sub-agente. La guida indica che Haiku è il modello più utilizzato per i sub-agenti al momento della registrazione. Haiku è un modello più leggero e veloce di Opus o Sonnet, perfetto per task specializzate che non richiedono il modello più potente.
- `max_tokens:` limita la quantità di token che il sub-agente può usare per le risposte.

Il corpo del file contiene le istruzioni in linguaggio naturale che definiscono il comportamento del sub-agente.

Procedura Pratica per Creare un Sub-agente

Metodo 1 — Creazione Manuale:

1. Navigate alla cartella `.claude/agents/` nel vostro progetto
2. Create un nuovo file con estensione `.md` (esempio: `researcher.md`)
3. Scrivete le istruzioni seguendo la struttura mostrata sopra
4. Salvate il file

Metodo 2 — Creazione Assistita da Claude (raccomandato):

Questo è il metodo utilizzato nella guida originale. Potete chiedere direttamente a Claude di creare il sub-agente per voi:

```
"Per favore guarda la documentazione ufficiale di Claude
e creami un sub-agente [nome]. Deve fare [descrizione]."
```

```
Popola il file con le best practice della documentazione ufficiale."
```

Claude andrà a consultare la documentazione ufficiale di Anthropic, capirà la struttura corretta dei sub-agenti e creerà il file con le best practice aggiornate.

Metodo 3 — Importazione da Template Globali:

Se avete già sub-agenti configurati nella vostra cartella globale (~/.claude/agents/), potete chiedere a Claude di importarli nel progetto corrente:

```
text
```

```
"Per favore importa i tre sub-agenti che sono nella mia  
cartella globale .claude/agents/: il reviewer, il researcher e il QA."
```

Dove Vivono i Sub-agenti

I sub-agenti possono esistere a diversi livelli dell'architettura Claude Code:

```
LIVELLO LOCAL (dentro il progetto):  
progetto/  
└─ .claude/  
    └─ agents/  
        ├── researcher.md   ← Sub-agente di questo progetto  
        ├── reviewer.md    ← Sub-agente di questo progetto  
        └─ qa.md           ← Sub-agente di questo progetto
```

```
LIVELLO GLOBAL (nel computer dell'utente):  
~/.claude/  
└─ agents/  
    ├── researcher.md   ← Disponibile per TUTTI i progetti  
    ├── reviewer.md    ← Disponibile per TUTTI i progetti  
  
    └─ qa.md           ← Disponibile per TUTTI i progetti
```

Il vantaggio di avere sub-agenti a livello globale è che non dovete ricrearli per ogni nuovo progetto. Potete importarli o Claude li troverà automaticamente.

Come Chiamare un Sub-agente

Una volta creato, chiamare un sub-agente è semplice come scrivere un prompt:

```
"Per favore chiama il sub-agente reviewer e assicurati  
di rivedere tutto il codice."
```

Oppure, in modo più diretto:

```
"Chiama il researcher sub-agent per fare una ricerca
sulle best practice per [argomento]."
```

Claude riconosce il nome del sub-agente, lo attiva nel suo contesto separato, gli invia la task e raccoglie il risultato.

Perché Usare Haiku per i Sub-agenti

La scelta di Haiku come modello per i sub-agenti è strategica:

Caratteristica	Haiku	Opus/Sonnet
Velocità	Molto veloce	Più lento
Costo per token	Molto basso	Più alto
Capacità cognitiva	Sufficiente per task specializzate	Superiore per task complesse
Ideale per	Ricerche, review, test	Ragionamento complesso, architettura

Poiché i sub-agenti eseguono task specializzate e ben definite (non ragionamento generale complesso), non hanno bisogno del modello più potente. Haiku è sufficiente e molto più economico, il che è particolarmente importante considerando che i sub-agenti possono consumare molti token nel loro contesto interno.

Errori Comuni nella Creazione

1. Creare sub-agenti senza consultare la documentazione: la struttura dei sub-agenti evolve. Chiedete sempre a Claude di verificare le best practice aggiornate dalla documentazione ufficiale.
2. Non specificare il modello nel frontmatter: se non specificate il modello, il sub-agente potrebbe usare lo stesso modello dell'agente principale (Opus/Sonnet), consumando inutilmente risorse più costose.
3. Scrivere istruzioni troppo lunghe nel sub-agente: ricordate che anche le istruzioni del sub-agente occupano contesto (nel contesto del sub-agente). Istruzioni concise e precise producono risultati migliori.
4. Non definire chiaramente il formato di output: se non dite al sub-agente COME presentare i risultati, potrebbe produrre output troppo lunghi che poi occuperanno troppo spazio nel contesto principale.

24.3 — Il Researcher Sub-agent

Definizione del Concetto

Il Researcher è un sub-agente specializzato nella ricerca di informazioni, best practice, documentazione e dati online. Il suo scopo è raccogliere grandi quantità di informazioni, processarle e restituire all'agente principale solo un riassunto sintetico e azionabile.

Spiegazione Approfondita

Il Researcher è il primo dei tre sub-agenti raccomandati dall'autore della guida, e il suo funzionamento illustra perfettamente il vantaggio dei sub-agenti per il context management.

Il Problema che il Researcher Risolve:

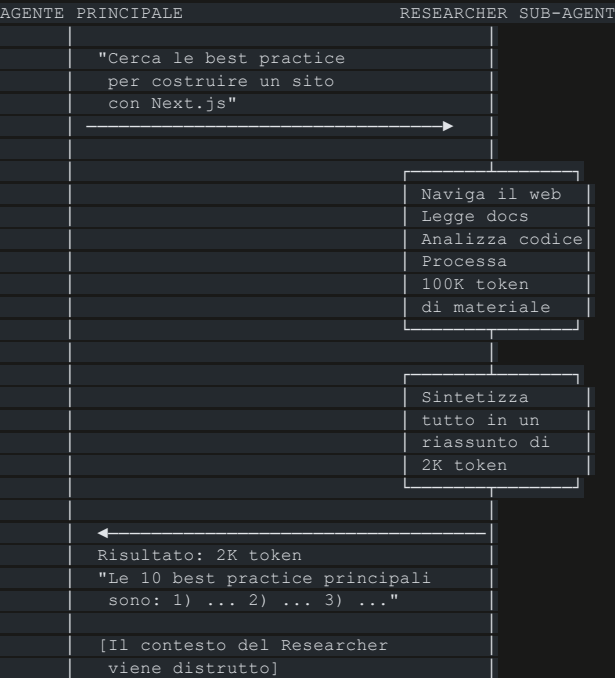
Quando chiedete a Claude di fare una ricerca (ad esempio: "Cerca le best practice per costruire un sito internet con Next.js"), il processo di ricerca genera una quantità enorme di dati:

- Pagine web visitate e scrappate
- Documentazione tecnica letta
- Articoli analizzati
- Codice di esempio trovato
- Discussioni su forum consultate

Tutto questo materiale può facilmente raggiungere i 100.000 token. Se questa ricerca avviene nel contesto principale, avete bruciato metà del vostro contesto solo per la fase di ricerca, prima ancora di iniziare a costruire qualcosa.

Come il Researcher Risolve il Problema:

FLUSSO DEL RESEARCHER SUB-AGENT



Il risultato netto è che nel contesto principale entrano solo 2.000 token di risultato altamente raffinato, invece di 100.000 token di materiale grezzo. Questo è un miglioramento del 98% nell'efficienza del contesto.

Applicazione Pratica — Quando Usare il Researcher

Scenario	Usare il Researcher?	Motivo
Cercare best practice per un framework	✔ Sì	Produce molti dati da sintetizzare
Analizzare documentazione di una libreria	✔ Sì	Documentazioni sono tipicamente enormi
Confrontare diverse soluzioni tecniche	✔ Sì	Richiede lettura di multiple fonti
Chiedere un parere su un errore specifico	✘ No	Task troppo piccola, non serve delegare
Fare un semplice calcolo	✘ No	Non c'è ricerca da fare
Esplorare un topic nuovo per il progetto	✔ Sì	Esplorazione = molti dati in input

L'Esempio Pratico dalla Guida

L'autore della guida fornisce un esempio concreto: prima di costruire un sito internet, invece di chiedere all'agente principale di cercare le best practice (inquinando il contesto), fa questa sequenza:

1. Chiama il Researcher sub-agent: *"Cerca le best practice per costruire un sito internet"*
2. Il Researcher fa la ricerca nel suo contesto separato
3. Torna con un riassunto sintetico delle best practice
4. L'agente principale usa queste best practice per costruire il sito

Questo approccio produce risultati superiori perché:

- Il contesto principale resta pulito per il lavoro di costruzione
- Le best practice sono già sintetizzate e pronte all'uso
- Non c'è rischio di "polluting the context" (inquinamento del contesto)

Insight Avanzato — Il Concetto di "Polluting the Context"

La guida introduce il termine "polluting the context" (inquinare il contesto) per descrivere ciò che accade quando si caricano nel contesto principale informazioni che non sono direttamente rilevanti per la task corrente. Ogni informazione irrilevante:

- Occupa spazio prezioso
- Aumenta il rischio di "Lost in the Middle"
- Può confondere il modello durante il ragionamento
- Degrada la qualità complessiva delle risposte

Il Researcher sub-agent è la soluzione primaria a questo problema: tutta la "sporcizia" della ricerca resta nel suo contesto (che viene poi distrutto), e solo il "distillato" puro entra nel contesto principale.

24.4 — Il Reviewer Sub-agent

Definizione del Concetto

Il Reviewer è un sub-agente specializzato nella revisione completa del codice. La sua caratteristica unica e fondamentale è che opera con zero contesto — non ha alcuna conoscenza pregressa del progetto, della conversazione o delle decisioni prese. Riceve semplicemente tutto il codice e lo rivede con occhi completamente freschi.

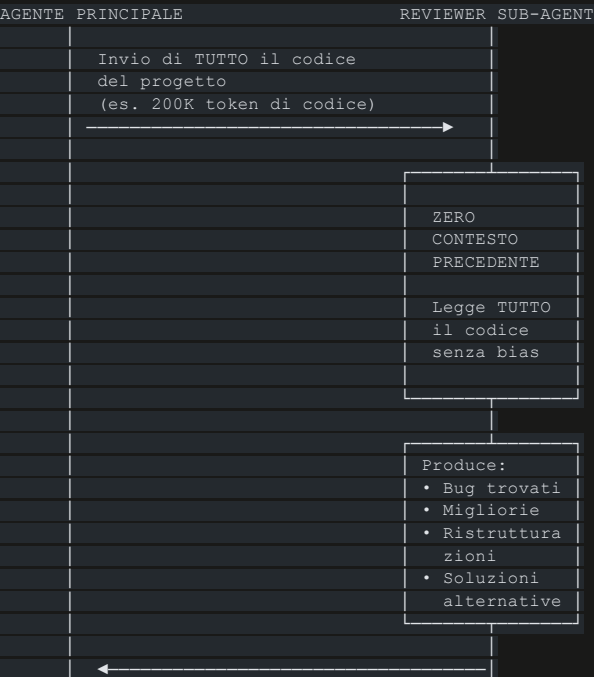
Spiegazione Approfondita

Il Reviewer rappresenta un concetto potentissimo: la revisione imparziale. Quando l'agente principale lavora su un progetto per ore, accumula bias e assunzioni. Ha preso decisioni, ha fatto scelte architetturali, ha risolto problemi in un certo modo. Il Reviewer non sa nulla di tutto questo. Vede solo il codice risultante e lo giudica per quello che è, non per come ci si è arrivati.

Il Meccanismo del Reviewer:

text

FLUSSO DEL REVIEWER SUB-AGENT



	Risultato: ~2K token	
	"8 fix applicati:	
	Critical: 2, High: 3, Medium: 3	
	CLAUDE.md ristrutturato	

--	--	--

Perché "Zero Contesto" è un Vantaggio

A prima vista, potrebbe sembrare uno svantaggio che il Reviewer non conosca il contesto del progetto. In realtà è il suo punto di forza più grande:

1. Nessun bias di conferma:
L'agente principale potrebbe aver scritto codice in un certo modo perché "funzionava al momento" o perché era la soluzione più rapida. Il Reviewer non ha questo bias — valuta il codice oggettivamente.
2. Prospettiva fresca:
Spesso in programmazione (e nella vita), quando si lavora troppo a lungo su qualcosa, si perdono di vista soluzioni migliori. Il Reviewer può dire: *"Perché l'hai fatto così? Ci sono soluzioni molto più semplici."*
3. Scoperta di pattern nascosti:
Senza contesto, il Reviewer analizza il codice basandosi solo sulla sua qualità intrinseca. Può identificare pattern architetturali problematici che l'agente principale non notava perché li aveva costruiti gradualmente.

Cosa Fa il Reviewer in Pratica

Dalla guida originale, quando il Reviewer viene chiamato su un progetto reale, produce:

- Fix critici: problemi di sicurezza, bug che causano crash
- Fix ad alta priorità: problemi di performance, logica errata
- Fix a media priorità: miglioramenti di codice, refactoring
- Ristrutturazione del CLAUDE.md: lo rende più conciso e ben organizzato
- Creazione di regole: genera file di regole per la cartella `.claude/rules/`
- Creazione di skill: se identifica pattern ripetitivi, può suggerire di creare skill

Nell'esempio della guida, il Reviewer applicava 8 fix categorizzati per priorità (Critical, High, Medium) e ristrutturava il CLAUDE.md rendendolo più pulito e modulare.

Applicazione Pratica — Quando Usare il Reviewer

Scenario	Usare il Reviewer?	Motivo
Dopo aver completato un MVP	✔ Sì	Revisione completa prima di procedere
Dopo ogni fase significativa del progetto	✔ Sì	Catch errori accumulati

Prima del deployment in produzione

✓ Assolutamente sì

Ultima verifica critica

Dopo ogni singolo piccolo cambiamento

✗ No

Spreco di risorse per task troppo piccola

Quando il codice "funziona ma non mi convince"

✓ Sì

Validazione oggettiva

L'Impatto sul CLAUDE.md

Un aspetto particolarmente prezioso del Reviewer, evidenziato nella guida, è la sua capacità di ristrutturare il CLAUDE.md. Dopo la revisione, il Reviewer produce un CLAUDE.md che:

"È pulitissimo, non c'è praticamente nulla, quattro regole in croce, do/don't, what/how. Un gioiello per quando cominceremo ad usarlo seriamente."

Questo è l'ideale: un CLAUDE.md conciso che contiene solo le informazioni essenziali, con tutto il resto scaricato nelle regole modulari. Il Reviewer riesce a fare questo perché, non avendo contesto, non ha "attaccamento emotivo" alle regole ridondanti e le elimina senza esitazione.

24.5 — Il QA Sub-agent

Definizione del Concetto

Il QA (Quality Assurance) è un sub-agente specializzato nell'esecuzione di test e nella verifica che il codice funzioni correttamente. È la componente che si assicura che tutto ciò che è stato costruito funzioni come previsto, senza bug e senza regressioni.

Spiegazione Approfondita

Il QA sub-agent rappresenta la fase finale del ciclo di qualità. Se il Researcher trova le informazioni giuste per costruire qualcosa, e il Reviewer verifica che il codice sia ben scritto, il QA verifica che il codice funzioni effettivamente.

Le Attività del QA Sub-agent:

text

=====

1. TEST FUNZIONALI

- └─ Verifica che ogni funzione faccia ciò che deve fare
- └─ Testa i percorsi principali dell'applicazione
- └─ Verifica che input/output siano corretti

2. TEST DI REGRESSIONE

- └─ Verifica che le modifiche recenti non abbiano rotto

```
funzionalità precedenti
└─ Confronta il comportamento attuale con quello atteso
```

```
3. TEST DI INTEGRAZIONE
└─ Verifica che i diversi componenti lavorino insieme
└─ Testa le connessioni con servizi esterni
   (Supabase, Stripe, etc.)
```

```
4. TEST DI EDGE CASE
└─ Testa scenari limite (input vuoti, valori estremi)
└─ Verifica la gestione degli errori
```

```
5. FIX AUTOMATICI
└─ Se trova bug, li corregge automaticamente
```

```
└─ Verifica che il fix non introduca nuovi problemi
```

Come Funziona in Pratica

Nell'esempio della guida, quando il QA sub-agent viene chiamato sull'applicazione Trello:

1. Riceve tutto il codice del progetto
2. Esegue una batteria di test
3. Identifica eventuali problemi
4. Applica fix automaticamente (se gli è stato dato il permesso)
5. Verifica che i fix funzionino
6. Produce un report: *"Risultati QA: costruito, passato dopo il fix"*

Il dettaglio importante è che il QA può applicare fix direttamente, senza chiedere all'utente. Questo è particolarmente utile quando si lavora in modalità bypass permission e si vuole che il sistema si auto-corregga.

La Sinergia tra i Tre Sub-agenti

I tre sub-agenti raccomandati formano un sistema completo di garanzia della qualità:

text

FLUSSO IDEALE DI UN PROGETTO CON TUTTI E TRE I SUB-AGENTI

```
FASE 1: RICERCA
  Researcher → "Ecco le best practice per [argomento]"
  |
  ▼
```

```
FASE 2: COSTRUZIONE
  Agente Principale → costruisce il progetto usando
                    le best practice del Researcher
  |
  ▼
```

```
FASE 3: REVISIONE
  Reviewer → "Ho trovato 8 problemi, ecco i fix.
             Ho ristrutturato il CLAUDE.md.
             Ho creato regole e skill."
  |
  ▼
```

```
FASE 4: TESTING
  QA → "Ho testato tutto, trovati 3 bug,
        fixati automaticamente.
        Tutti i test passano ora."
  |
  ▼
```

```
FASE 5: RISULTATO FINALE
  Progetto completo, testato, ottimizzato,
  con CLAUDE.md pulito e regole modulari
```

Questa sequenza può essere eseguita dando pochi prompt:

```
text
```

```
Prompt 1: "Chiama il researcher sub-agent per cercare  
le best practice per [progetto]"
```

```
Prompt 2: "Usando le best practice trovate, costruisci  
[progetto]"
```

```
Prompt 3: "Chiama il reviewer sub-agent per rivedere  
tutto il codice"
```

```
Prompt 4: "Chiama il QA sub-agent per verificare che
```

```
tutto funzioni"
```

Quattro prompt per un ciclo completo di sviluppo professionale.

Il Vantaggio Economico dei Sub-agenti vs Lavoro nel Contesto Principale

L'autore della guida sottolinea un punto fondamentale: utilizzare sub-agenti è un investimento che si ripaga. Anche se i sub-agenti consumano token nei loro contesti separati, il risparmio nel contesto principale è tale che:

- Potete lavorare per sessioni molto più lunghe senza degrado di qualità
- Evitate di dovere ricominciare da zero perché il contesto si è saturato
- Ogni iterazione è più efficiente perché il contesto principale è pulito
- La qualità finale è superiore perché ogni fase è gestita da uno specialista

Errori Comuni nell'Uso dei Sub-agenti

1. Non usarli perché "tanto Claude può fare tutto": Claude può fare tutto, ma la qualità è drasticamente superiore con sub-agenti specializzati.
2. Usare i sub-agenti per task banali: chiamare un sub-agente per rinominare un file è spreco. I sub-agenti eccellono su task complesse che generano molti token.
3. Non parallelizzare i sub-agenti: Reviewer e QA possono lavorare contemporaneamente (in terminal separati). Non c'è motivo di aspettare che uno finisca per lanciare l'altro.
4. Dimenticare di importare i risultati: dopo che un sub-agente ha finito, assicuratevi che i fix e le modifiche siano effettivamente applicati nel progetto principale.

24.6 — Parallelizzazione dei Sub-agenti

Definizione del Concetto

La parallelizzazione dei sub-agenti consiste nell'eseguire più sub-agenti contemporaneamente in terminal separati, in modo che lavorino in parallelo su task diverse anziché in sequenza.

Spiegazione Approfondita

Nella guida originale, l'autore mostra come parallelizzare i sub-agenti aprendo più istanze di Claude Code:

```
text
```

```
TERMINAL 1 (Agente Principale)
```

```
$ claude --dangerously-skip-  
permissions
```

```
> "Per favore chiama il  
reviewer sub-agent..."
```

TERMINAL 2 (Parallelo)

```
$ claude --dangerously-skip-  
permissions  
  
> "Per favore chiama il  
QA sub-agent..."
```

TERMINAL 3 (Parallelo)

```
$ claude --dangerously-skip-  
permissions  
  
> "Per favore fai una ricerca  
sulla pasta con il  
researcher sub-agent..."
```

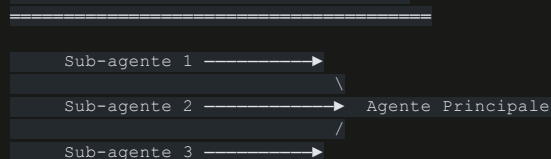
Ogni istanza ha il proprio contesto indipendente. Non si influenzano a vicenda. Tutte hanno accesso allo stesso file system (stessa cartella del progetto), quindi le modifiche fatte da un sub-agente sono visibili agli altri.

L'Importanza del Flusso Monodirezionale

Un punto critico sottolineato nella guida è che con i sub-agenti tradizionali, il flusso di informazioni è monodirezionale: dai sub-agenti all'agente principale.

text

FLUSSO MONODIREZIONALE DEI SUB-AGENTI



OGNI FRECCIA = solo risultato (2K token circa)

NESSUNA COMUNICAZIONE TRA SUB-AGENTI

I sub-agenti non si parlano tra loro. Sub-agente 1 non sa cosa sta facendo Sub-agente 2, e viceversa. Questo è un vantaggio per il context management (nessun overhead di comunicazione) ma un limite per task che richiedono collaborazione. Per superare questo limite, Anthropic ha introdotto gli Agent Teams, che vedremo nel capitolo successivo.

CAPITOLO 25

Agent Teams — Collaborazione Multi-Agente

25.1 — Cosa Sono gli Agent Teams

Definizione del Concetto

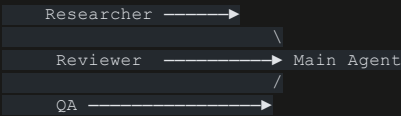
Gli Agent Teams sono una funzionalità relativamente recente introdotta da Anthropic che permette di creare un team di agenti collaborativi che possono comunicare tra loro in modo bidirezionale. A differenza dei sub-agenti tradizionali (dove la comunicazione è solo dal sub-agente verso l'agente principale), negli Agent Teams ogni agente può parlare con ogni altro agente del team.

Spiegazione Approfondita

La differenza fondamentale tra sub-agenti e Agent Teams si capisce meglio con un diagramma comparativo:

SUB-AGENTI (comunicazione monodirezionale):

text

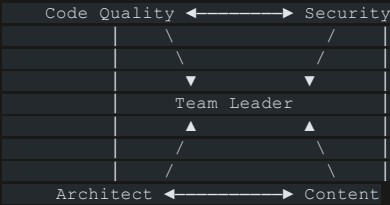


- Ogni sub-agente lavora isolato
- Nessuna comunicazione tra sub-agenti
- Solo il risultato va all'agente principale

- Costo: BASE (1x)

AGENT TEAMS (comunicazione bidirezionale):

text



- Ogni agente può parlare con ogni altro agente
- Condivisione di informazioni reciproca
- Coordinamento automatico dei lavori

- Costo: ELEVATO (3-5x rispetto ai sub-agenti)

Il Meccanismo della Comunicazione Bidirezionale

Negli Agent Teams, quando un agente scopre qualcosa di rilevante per un altro agente, può comunicarglielo direttamente. Questo elimina il collo di bottiglia dell'agente principale come unico punto di comunicazione.

Esempio pratico dalla guida:

L'autore analizza una repository GitHub con un Agent Team di 4 teammate:

- Code Quality: analizza la qualità del codice
- Security: analizza la sicurezza

- Architect: analizza l'architettura
- Content: analizza la documentazione

Durante l'analisi, il teammate Security potrebbe trovare una vulnerabilità che impatta l'architettura. In un sistema con sub-agenti tradizionali, dovrebbe:

1. Comunicare il problema all'agente principale
2. L'agente principale dovrebbe poi comunicarlo al sub-agente Architect
3. L'Architect dovrebbe poi proporre una soluzione
4. La soluzione tornerebbe all'agente principale

Con un Agent Team, il processo è diretto:

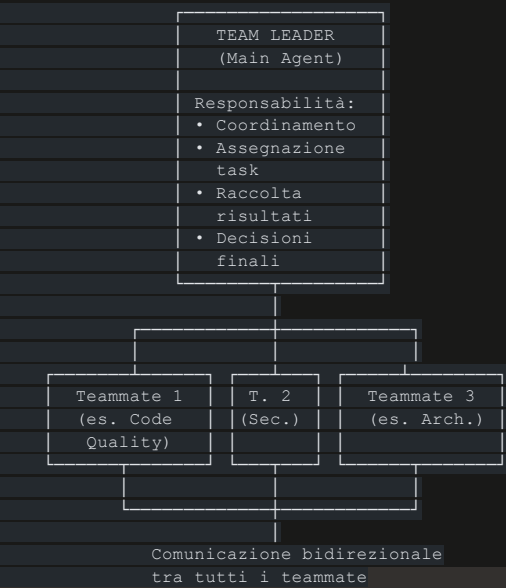
1. Security comunica direttamente ad Architect: *"Ho trovato una vulnerabilità nella struttura X. Devi ristrutturare Y."*
2. Architect modifica l'architettura
3. Entrambi aggiornano il Team Leader

Questo è molto più veloce e produce risultati più coerenti.

La Struttura del Team

text

STRUTTURA DI UN AGENT TEAM



Come il Team Leader "Spawna" i Teammate

Quando date un comando per creare un Agent Team, il Team Leader (l'agente principale) analizza la task e decide:

1. Quanti teammate servono (potete specificarlo voi o lasciarlo decidere a lui)
2. Quali ruoli assegnare a ciascun teammate
3. Quali responsabilità specifiche dare a ciascuno
4. Come suddividere il lavoro per evitare duplicazioni

Nell'esempio della guida, l'autore chiede:

"Crea un agent team con un massimo di quattro compagni di squadra per analizzare questa repository."

Il Team Leader decide autonomamente di creare:

- Code Quality teammate
- Security teammate
- Architect teammate
- Content teammate

Ciascuno con responsabilità specifiche e complementari.

L'Interfaccia di Monitoraggio

La guida mostra che è possibile monitorare i teammate in tempo reale. Premendo Shift + freccia giù nel terminal, si accede a una visualizzazione live che mostra:

```
text

VISUALIZZAZIONE LIVE AGENT TEAMS
=====

┌── Main (Team Leader) ──┐
│ Contesto: 45.000/200.000 tokens │
│ Tools chiamati: 12 │
│ Stato: coordinamento │
└────────────────────────┘

┌── Code Quality ──┐
│ Contesto: 171.000/200.000 tokens │ ← QUASI PIENO
│ Tools chiamati: 47 │
│ Stato: analisi in corso │
└────────────────────────┘

┌── Security ──┐
│ Contesto: 89.000/200.000 tokens │
│ Tools chiamati: 63 │
│ Stato: scan vulnerabilità │
└────────────────────────┘

┌── Architect ──┐
│ Contesto: 120.000/200.000 tokens │
│ Tools chiamati: 35 │
│ Stato: revisione struttura │
└────────────────────────┘

┌────────────────────────┘
```

Dalla guida emerge un dato importante: il teammate Code Quality ha raggiunto i 171.000 token nel suo contesto. Successivamente fa un reset del context e scende a 59.000, dimostrando che anche i teammate hanno la capacità di fare context management automatico.

Le Shared Task — Coordinamento Automatico

Una caratteristica specifica degli Agent Teams è la gestione delle shared task (task condivise). Anthropic ha implementato un meccanismo che impedisce a due agenti di lavorare sulla stessa identica cosa:

"Loro sanno in automatico che cosa uno sta facendo e quindi Anthropic in automatico ha fatto sì che noi non avessimo mai il problema di avere due agent che lavorano sulla stessa identica cosa."

Questo significa che se il teammate Code Quality sta analizzando il file `auth.js`, il teammate Security sa che quel file è "occupato" e analizzerà prima altri file, per poi tornare su `auth.js` quando sarà libero (con eventuali note del teammate precedente).

25.2 — Come Abilitare e Utilizzare gli Agent Teams

Definizione del Concetto

Gli Agent Teams funzionano attualmente solo nel terminal e richiedono un processo di abilitazione specifico. Non sono disponibili nella GUI degli IDE (VS Code o Antigravity).

Procedura di Abilitazione

La guida mostra il processo pratico:

Passo 1 — Consultare la documentazione:

```
text
```

```
"Per favore, utilizzando questa documentazione  
[link alla documentazione ufficiale],
```

```
potresti abilitarmi gli Agent Teams?"
```

Passo 2 — Claude configura il sistema:

Claude legge la documentazione e configura automaticamente gli Agent Teams nel vostro ambiente.

Passo 3 — Verificare l'abilitazione:

Dopo la configurazione, Claude conferma che gli Agent Teams sono abilitati e vi spiega come usarli.

Come Creare un Team per una Task Specifica

Una volta abilitati, potete creare un team con un singolo prompt:

```
text
```

```
"Crea un agent team con un massimo di [N] compagni  
di squadra per [descrizione della task].
```

```
Vorrei che ogni teammate avesse un ruolo specifico  
e che analizzassero [cosa] in parallelo.
```

```
Alla fine, portami [tipo di risultato desiderato]."
```

Esempio concreto dalla guida:

```
text
```

```
"Ho una repository [link]. Vorrei che tu la analizzassi  
creando un agent team con un massimo di quattro compagni  
di squadra. Vorrei che poi tu mi portassi delle migliorie  
che possiamo fare a tutto tondo, non solo in ambito sicurezza
```

```
ma anche a livello di codice."
```

Navigazione e Monitoraggio durante l'Esecuzione

Durante l'esecuzione di un Agent Team:

Azione	Tasto/Comando	Risultato
Vedere i teammate attivi	Guarda la barra in basso	Mostra: main, architect, code quality, etc.
Navigare tra i teammate	Shift + freccia giù	Visualizzazione live con dettagli
Vedere il consumo di contesto	Visualizzazione live	Token e % per ogni teammate
Vedere i tool chiamati	Visualizzazione live	Numero di tool call per teammate
Vedere il costo in tempo reale	Status line	Costo cumulativo aggiornato

25.3 — Casi d'Uso Avanzati per gli Agent Teams

Analisi di Repository

Il caso d'uso principale mostrato nella guida è l'analisi di repository GitHub. Questo è un uso ideale degli Agent Teams perché:

- La repository è grande (centinaia di file)
- Serve analizzarla da prospettive diverse (sicurezza, qualità, architettura, documentazione)
- I teammate possono lavorare in parallelo su file diversi
- Le scoperte di un teammate possono informare il lavoro degli altri

Il risultato finale è un report completo con migliorie prioritzate: *"Ha trovato tutte le migliorie, fixare bug critici, aggiungere un capitolo, etc."*

Creazione di Contenuti Multipli in Parallelo

Un secondo caso d'uso potente illustrato nella guida è la creazione di contenuti multipli:

text

FLUSSO DI CREAZIONE ADS CON AGENT TEAMS

ROUND 1: Generazione iniziale

Prompt: "Fammi 5 proposte di ads totalmente diverse"

Team Leader

- Teammate 1 → Ad creativa #1
- Teammate 2 → Ad creativa #2
- Teammate 3 → Ad creativa #3
- Teammate 4 → Ad creativa #4
- Teammate 5 → Ad creativa #5

Tutto in PARALLELO → risultato in minuti

ROUND 2: Selezione e iterazione

Utente: "Mi piacciono la #2 e la #4.
Fammi 5 copie diverse di ciascuna."

Team Leader

- Sotto-team per Ad #2
 - Copia A
 - Copia B
 - Copia C
 - Copia D
 - Copia E
- Sotto-team per Ad #4
 - Copia A
 - Copia B
 - Copia C
 - Copia D
 - Copia E

ROUND 3: Targeting

Utente: "Bellissima la versione visiva!
Fammine 5 uguali ma targetizzate per:
elettricisti, imprenditori, meccanici,
medici, avvocati"

Team Leader

- Versione per elettricisti
- Versione per imprenditori
- Versione per meccanici
- Versione per medici
- Versione per avvocati

In un'ora, con questo processo iterativo, si possono produrre 50-60 ads complete, ciascuna targetizzata per un segmento specifico. L'equivalente umano richiederebbe giorni di lavoro di un team creativo.

Il Calcolo del ROI

L'autore della guida fornisce un calcolo ROI concreto per l'uso degli Agent Teams nel contesto business:

text

CALCOLO ROI PER AGENT TEAMS (Ads)

COSTO:

- Agent Teams per 1 ora: ~€500 (scenario pessimistico)

RICAVO:

- 50-60 ads prodotte in 1 giorno
- Se il ticket medio cliente = €1.000
- Se un lead costa €50 (costo acquisizione)
- Se le ads convertono 1 lead/giorno per 30 giorni
- Ricavo mensile: €1.000 × 30 = €30.000

$ROI = (\text{€}30.000 - \text{€}500) / \text{€}500 = 5.900\%$

CONFRONTO UMANO:

- Team creativo umano: 2 settimane per 50 ads
- Costo team umano: €5.000-€10.000
- Agent Teams: 1 ora, €500

- Risparmio: 10-20x

Questo calcolo è ovviamente semplificato, ma illustra il principio: per un business con budget adeguato, gli Agent Teams possono avere un ROI devastante.

CAPITOLO 26

Costi, ROI e Utilizzo Strategico

26.1 — Il Costo Reale degli Agent Teams

Definizione del Concetto

Gli Agent Teams consumano token a una velocità significativamente superiore rispetto ai sub-agenti tradizionali o al lavoro nel contesto singolo. L'autore della guida stima che il consumo sia 3-5 volte superiore rispetto all'uso di sub-agenti per la stessa task.

Spiegazione Approfondita — Perché Costano di Più

Il costo elevato degli Agent Teams deriva da tre fattori:

1. Comunicazione bidirezionale:

Ogni comunicazione tra teammate genera token sia in uscita (dal mittente) che in ingresso (nel destinatario). Con 4 teammate che comunicano tutti con tutti, il numero di comunicazioni possibili cresce in modo combinatorio.

text

COMUNICAZIONI POSSIBILI CON N TEAMMATE

Con 2 teammate: 2 comunicazioni possibili

A ↔ B

Con 3 teammate: 6 comunicazioni possibili

A ↔ B, A ↔ C, B ↔ C

Con 4 teammate: 12 comunicazioni possibili

A ↔ B, A ↔ C, A ↔ D, B ↔ C, B ↔ D, C ↔ D

Con 5 teammate: 20 comunicazioni possibili

Formula: $N \times (N-1)$ comunicazioni bidirezionali

Ogni comunicazione consuma token. Più teammate avete, più comunicazioni avvengono, più token vengono consumati.

2. Contesti multipli simultanei:

Ogni teammate ha il proprio contesto completo. Con 4 teammate più il Team Leader, avete 5 contesti attivi contemporaneamente, ciascuno con il proprio system prompt, i propri tool e la propria conversazione.

3. Context Management interno:

Come mostrato nella guida, i teammate possono raggiungere i 171.000 token nel loro contesto e poi fare reset a 59.000. Questo processo di compattazione e reset consuma ulteriori risorse computazionali.

I Numeri Reali dalla Guida

L'autore condivide dati concreti durante la sessione di Agent Teams:

Tempo trascorso	Costo accumulato	Osservazione
~5 minuti	€3	"Solo l'inizio"
~7 minuti	€5	"Ogni volta che fa bip ho speso €5"
Fine sessione	€10-20 (stimato)	Analisi completa di una repository

Per confronto, una sessione di lavoro normale con un singolo agente su un piano abbonamento da €17/mese costa effettivamente... €17/mese per uso illimitato. La differenza è abissale.

L'Avvertimento dell'Autore

"Per favore non dimenticatevi mai che siate consapevoli di quello che state facendo perché se non avete i soldi da buttare tipo €10-€20 beh evitate perché nel senso non ha poi tutto questo senso."

E ancora:

"Nonostante sembri sexy a dirlo, facendogli andare 15 minuti spendete €80."

Questi avvertimenti sono fondamentali. L'hype online intorno agli Agent Teams è enorme, ma l'autore (che spende circa €400/mese in Claude Code) è molto pragmatico: non li usa regolarmente perché il costo è troppo elevato per la maggior parte dei casi d'uso quotidiani.

26.2 — Quando Usare gli Agent Teams vs i Sub-agenti

Definizione del Concetto

La scelta tra Agent Teams e sub-agenti non è una questione di "quale è migliore" ma di quale è appropriato per la task specifica, considerando il rapporto costo-beneficio.

Framework Decisionale Completo

text

ALBERO DECISIONALE: AGENT TEAMS vs SUB-AGENTI

```
La task richiede COLLABORAZIONE tra agenti?
├── NO → Usa SUB-AGENTI
│   ├── Task indipendenti (ricerca, review, test)
│   ├── Risultato di un agente non dipende dagli altri
│   └── Costo: BASSO
└── SÌ → La task genera un ROI significativo?
    ├── NO → Usa SUB-AGENTI in sequenza
    │   ├── Fai prima il Researcher
    │   ├── Poi il Reviewer
    │   ├── Poi il QA
    │   └── Comunica i risultati manualmente
    └── SÌ → Usa AGENT TEAMS
        ├── Analisi complesse multi-dimensional
        ├── Creazione massiva di contenuti
        ├── Task che richiedono coordinamento
        └── Budget disponibile per il costo
```

Tabella Comparativa Dettagliata

Caratteristica	Sub-agenti	Agent Teams
Comunicazione	Mono-direzionale (→ principale)	Bidirezionale (tutti ↔ tutti)
Costo	Basso (1x)	Alto (3-5x)
Velocità	Veloce per task singole	Velocissimo per task parallele

Coordinamento	Manuale (tramite utente)	Automatico (tramite Team Leader)
Context overhead	Minimo	Significativo
Ideale per	Task indipendenti e specializzate	Task complesse che richiedono collaborazione
Disponibilità	GUI e Terminal	Solo Terminal
Rischio di costo	Basso e prevedibile	Alto e potenzialmente imprevedibile
Esempi ideali	Ricerca, review, test	Analisi repository, creazione ads, audit

Gli Use Case Ideali per Ciascuno

Sub-agenti — Use Case Ideali:

1. Ricerca di best practice (Researcher)
2. Revisione del codice dopo una fase di sviluppo (Reviewer)
3. Test e quality assurance (QA)
4. Qualsiasi task che un singolo specialista può fare indipendentemente

Agent Teams — Use Case Ideali:

1. Analisi completa di repository grandi (multi-prospettiva)
2. Creazione massiva di contenuti con variazioni (ads, post, email)
3. Refactoring completo di un'applicazione (serve coordinamento tra frontend, backend, test)
4. Audit aziendale completo (sicurezza + codice + architettura + documentazione)
5. Qualsiasi task che in un'azienda richiederebbe un team di persone che si parlano

La Regola Pratica

Se la task può essere fatta da una singola persona competente → Sub-agente

Se la task richiede un team di persone che collaborano → Agent Team

26.3 — Strategia di Utilizzo Pragmatica

Definizione del Concetto

L'approccio pragmatico all'utilizzo di Agent Teams e sub-agenti si basa sul principio del ROI consapevole: ogni decisione di spesa deve essere giustificata da un ritorno misurabile.

Il Framework ROI per la Decisione

Prima di lanciare un Agent Team, fatevi queste domande:

text

CHECKLIST PRE-LANCIO AGENT TEAM

- ☐ 1. VALORE: Quanto vale il risultato di questa task?
 - Se la task produce un asset che genera ricavo: PROCEDERE
 - Se la task è esplorativa o personale: USARE SUB-AGENTI
- ☐ 2. ALTERNATIVA UMANA: Quanto costerebbe farlo manualmente?
 - Se il team umano costerebbe 10x+ il costo dell'Agent Team: PROCEDERE
 - Se il costo è comparabile: VALUTARE caso per caso
- ☐ 3. TEMPO: Quanto tempo risparmiamo?
 - Se risparmiamo giorni/settimane: PROCEDERE
 - Se risparmiamo minuti: NON GIUSTIFICATO
- ☐ 4. QUALITÀ: L'Agent Team produce risultati migliori?
 - Se la parallelizzazione migliora la qualità: PROCEDERE
 - Se la qualità è equivalente ai sub-agenti: USARE SUB-AGENTI
- ☐ 5. BUDGET: Posso permettermi il costo?
 - Se €10-80 per sessione sono sostenibili: PROCEDERE
 - Se ogni euro conta: USARE SUB-AGENTI

Esempi di ROI dal Mondo Reale (dalla Guida)

Esempio 1: Analisi Repository per un Cliente

text

Costo Agent Team: €20 (una sessione di ~15 minuti)
Costo alternativo: 3 developer × 2 settimane = €6.000+
Valore deliverable: Report completo con priorità di intervento
ROI: Estremo (300x)

Verdetto: USARE AGENT TEAMS 

Esempio 2: Creazione Ads per Campagna

text

Costo Agent Team: €500 (sessione intensiva di 1 ora)
Costo alternativo: Team creativo × 2 settimane = €5.000-10.000
Valore deliverable: 50-60 ads targetizzate
ROI: 10-20x

Verdetto: USARE AGENT TEAMS 

Esempio 3: Rinominare file e fare piccole modifiche

text

Costo Agent Team: €10
Costo alternativo: 5 minuti di lavoro manuale = €0
Valore deliverable: File rinominati (valore quasi nullo)
ROI: Negativo

Verdetto: NON USARE AGENT TEAMS ❌ (usare singolo agente)

Il Pubblico Target per gli Agent Teams

L'autore è molto chiaro su chi dovrebbe usare gli Agent Teams:

- Business con budget per AI: aziende che hanno allocato budget per strumenti AI e possono assorbire costi di €100-500 per sessione
- Consulenti AI che vendono servizi: il costo dell'Agent Team viene ribaltato sul cliente con markup
- Progetti ad alto valore: dove il risultato dell'Agent Team vale migliaia di euro

Chi NON dovrebbe usare gli Agent Teams:

- Hobbisti e curiosi: il costo è troppo alto per l'esplorazione
- Progetti personali a basso budget: i sub-agenti fanno il 90% del lavoro a una frazione del costo
- Task semplici: sprecare €20 per qualcosa che un singolo agente può fare in 2 minuti

Insight Avanzato — Il Pattern "Agent Team come MVP → Skill"

Un pattern avanzato che emerge dalla guida è usare gli Agent Teams come strumento di prototipazione rapida per poi convertire il risultato in skill riutilizzabili:

text

PATTERN: AGENT TEAM → SKILL CONVERSION
=====

FASE 1: Usa un Agent Team per fare qualcosa di complesso
(costo: €50)

FASE 2: Analizza come i teammate hanno lavorato

FASE 3: Estrai i pattern e le procedure usate

FASE 4: Converti queste procedure in SKILL

FASE 5: D'ora in poi, usa le SKILL invece dell'Agent Team
(costo: ~€0.01 per chiamata)

RISULTATO: Investimento una tantum → risparmio perpetuo

Questo è simile al pattern MCP → Skill discusso nella guida: usate lo strumento costoso (Agent Team/MCP) per capire COME fare qualcosa, poi codificate quel "come" in una skill che costa quasi nulla da eseguire.

Riepilogo della Parte 7

In questa Parte avete appreso:

1. Cosa sono i sub-agenti: istanze separate di Claude con contesto indipendente che restituiscono solo il risultato all'agente principale
2. Come creare sub-agenti: file Markdown nella cartella `.claude/agents/` con frontmatter (modello, limiti) e istruzioni in linguaggio naturale
3. I tre sub-agenti raccomandati:
 - Researcher: ricerca informazioni, restituisce sintesi (100K → 2K token)
 - Reviewer: revisione a zero contesto, ristrutturazione codice e CLAUDE.md
 - QA: test funzionali, di regressione e di integrazione con fix automatici
4. La comunicazione monodirezionale: i sub-agenti non si parlano tra loro, tutto passa attraverso l'agente principale
5. Gli Agent Teams: team collaborativi con comunicazione bidirezionale tra tutti i teammate
6. Come abilitare e usare gli Agent Teams: solo via terminal, con Team Leader che spawna teammate
7. I costi reali: Agent Teams costano 3-5x più dei sub-agenti, €10-80+ per sessione
8. Quando usare cosa: sub-agenti per task indipendenti, Agent Teams per task che richiedono collaborazione
9. La strategia ROI: ogni decisione di utilizzo deve essere giustificata da un ritorno misurabile
10. Il pattern di conversione: usare strumenti costosi (Agent Teams) per prototipare, poi convertire in skill economiche

PARTE 8 — IL SISTEMA DELLE SKILL

"Le skill di Claude sono la funzione più sottovalutata del momento. Chi le usa è 10x più veloce di chi non le conosce."
— Dalla guida originale (esempio di LinkedIn Post generato dal sistema)

Introduzione alla Parte 8

Se i sub-agenti e gli Agent Teams rappresentano la vostra capacità di delegare e parallelizzare, le skill rappresentano la vostra capacità di codificare l'eccellenza in processi ripetibili. Una skill ben costruita trasforma un processo che richiederebbe 20 minuti di prompt, riflessione e iterazione in un singolo comando che produce risultati consistenti ogni volta.

Questa è probabilmente la Parte più direttamente collegata alla monetizzazione di Claude Code. Le skill sono ciò che vi permette di creare asset riutilizzabili, vendibili e scalabili. Un'agenzia di social media che costruisce le proprie skill ha un vantaggio competitivo enorme rispetto a chi usa Claude Code come un chatbot avanzato.

Questa Parte è composta da quattro capitoli:

Capitolo	Titolo	Focus Principale
27	Architettura delle Skill	Come sono strutturate internamente
28	Creare Skill Personalizzate	Il processo completo di creazione
29	Il Marketplace delle Skill	Dove trovare e importare skill esterne
30	La Qualità dei Dati di Riferimento	Perché il reference data è tutto

CAPITOLO 27

Architettura delle Skill

27.1 — Cosa Sono le Skill

Definizione del Concetto

Una skill in Claude Code è un processo codificato e ripetibile che insegna a Claude come eseguire una task specifica seguendo una procedura predefinita. Potete pensare a una skill come a una ricetta dettagliata che uno chef (Claude) segue per preparare un piatto specifico ogni volta con lo stesso livello di qualità.

Spiegazione Approfondita

L'analogia della ricetta, introdotta nella guida originale, è illuminante:

"Claude possiamo vederlo come uno chef a cui diamo un sacco di ricette. Una ricetta serve per cucinare la pasta, una per la pizza, una per fare un buon caffè. E gli diciamo quali sono gli ingredienti, quali sono gli strumenti che deve usare e tutte quelle cose lì."

Questa analogia cattura perfettamente l'essenza delle skill. Senza skill, Claude è uno chef talentuoso ma senza ricette: può cucinare qualcosa, ma il risultato varierà ogni volta e dipenderà dalla vostra capacità di descrivere cosa volete. Con le skill, Claude è uno chef con un ricettario completo: sa esattamente cosa fare, in quale ordine e con quali ingredienti.

Dove Vivono le Skill

Le skill possono esistere a diversi livelli dell'architettura Claude Code, esattamente come le regole e i sub-agenti:

text

LIVELLI DI POSIZIONAMENTO DELLE SKILL

LIVELLO LOCAL (dentro il progetto corrente):

```
progetto/
├── .claude/
│   └── skills/
│       ├── linkedin-post/
│       │   ├── skill.md
│       │   └── scripts/
│       │       └── generate_post.py
│       ├── publish/
│       │   ├── skill.md
│       │   └── scripts/
│       │       ├── build_schedule.py
│       │       ├── check_meta.py
│       │       ├── check_youtube.py
│       │       └── upload_youtube.py
│       └── audit/
│           ├── skill.md
│           └── scripts/
│               └── run_audit.py
```

LIVELLO GLOBAL (disponibile per tutti i progetti):

```
~/.claude/
├── skills/
│   ├── [stesse strutture di sopra]
│   └── [accessibili da qualsiasi progetto]
```

LIVELLO LEGACY:

[Formato precedente, meno strutturato,

che la guida menziona ma non approfondisce]

La scelta di dove posizionare una skill dipende dal suo ambito di utilizzo:

Posizione	Quando usarla	Esempio
Local	Skill specifica per un progetto	Skill di deploy per un'app specifica

Global

Skill riutilizzabile in tutti i progetti

Skill di generazione LinkedIn post

L'Impatto delle Skill sul Contesto

Un dato fondamentale emerso dalla guida è l'efficienza delle skill in termini di consumo di contesto:

text

CONFRONTO CONSUMO CONTESTO

Skill del progetto:	~0,3% del contesto
MCP leggero (Dev Tool):	~0,1% del contesto
MCP pesante (ClickUp):	~27% del contesto
System Prompt (Anthropic):	~10% del contesto

CONCLUSIONE: Le skill sono 90 volte più efficienti

di un MCP pesante per fornire funzionalità a Claude.

Questo dato da solo giustifica l'investimento nella creazione di skill personalizzate: sono il modo più efficiente per dare capacità a Claude senza saturare il contesto.

27.2 — La Struttura Interna di una Skill

Definizione del Concetto

Ogni skill è composta da due componenti fondamentali contenuti in una cartella dedicata:

1. Il file skill.md — l'orchestratore (la checklist)
2. La cartella scripts/ — il codice eseguibile

Questi due componenti lavorano insieme ma hanno ruoli molto diversi.

Spiegazione Approfondita — Il File skill.md (L'Orchestratore)

L'autore della guida usa una metafora potente per spiegare il ruolo del file skill.md:

"Dovete pensare a questa skill.md come un orchestrator. Immaginatevi un'orchestra: c'è il direttore d'orchestra — il tizio con le bacchette — e poi ci sono quelli che suonano l'orchestra."

text

ANATOMIA DEL FILE SKILL.MD

SKILL.MD
(Il Direttore d'Orchestra)
DESCRIZIONE DELLA SKILL
"Pubblica Real Short Video e posta su

```

YouTube, Instagram o Facebook in maniera
automatica"

```

```

PIATTAFORME E STRUMENTI
• YouTube: Python API
• Instagram: tramite Chrome Dev Tool MCP
• Facebook: tramite Chrome Dev Tool MCP

```

```

CHECKLIST DI VALIDAZIONE INPUT
☐ Gather input: which platform?
☐ Gather input: which content?
☐ If valid → procedi
☐ If invalid → richiedi correzione

```

```

SEQUENZA DI ESECUZIONE
1. Chiama build_schedule.py
2. Chiama check_meta.py
3. Chiama check_youtube.py
4. Chiama upload_youtube.py

```

```

GESTIONE ERRORI
• Se uno step fallisce → log errore
• Tenta auto-correzione
• Se non risolvibile → notifica utente

```

Il file `skill.md` è essenzialmente la mappa di navigazione della skill. Dice a Claude:

- Che cosa deve fare questa skill
- Quali input raccogliere dall'utente
- Come validare gli input
- In quale ordine eseguire le operazioni
- Quali script chiamare
- Come gestire gli errori

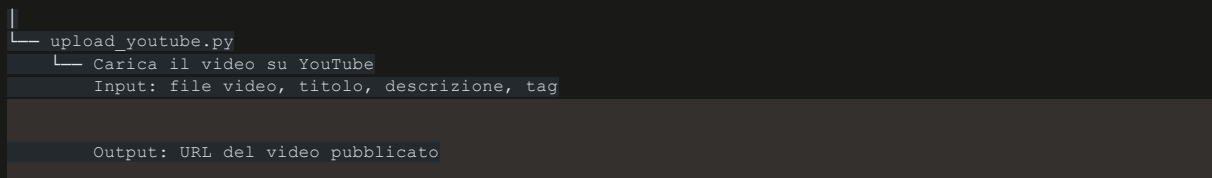
La Cartella scripts/ (I Musicisti dell'Orchestra)

Se il file `skill.md` è il direttore d'orchestra, gli script sono i musicisti. Sono loro che effettivamente "suonano" — eseguono il codice reale che compie le operazioni.

text

ANATOMIA DELLA CARTELLA SCRIPTS/

```
scripts/
├── build_schedule.py
│   ├── Costruisce la mappatura di quando pubblicare
│   ├── Input: preferenze utente (giorno, ora, piattaforma)
│   └── Output: schedule strutturato
├── check_meta.py
│   ├── Verifica se su Meta (Facebook/Instagram)
│   ├── c'è già un post programmato
│   ├── Input: schedule, credenziali Meta
│   └── Output: stato (libero/occupato)
├── check_youtube.py
│   ├── Verifica lo stato del canale YouTube
│   ├── Input: credenziali YouTube API
│   └── Output: stato canale, video recenti
```



Gli script sono codice deterministico — non sono prompt LLM. Sono programmi Python (o altro linguaggio) che eseguono operazioni specifiche e prevedibili. Questo è un punto cruciale che vedremo in dettaglio.

La Relazione tra skill.md e scripts/

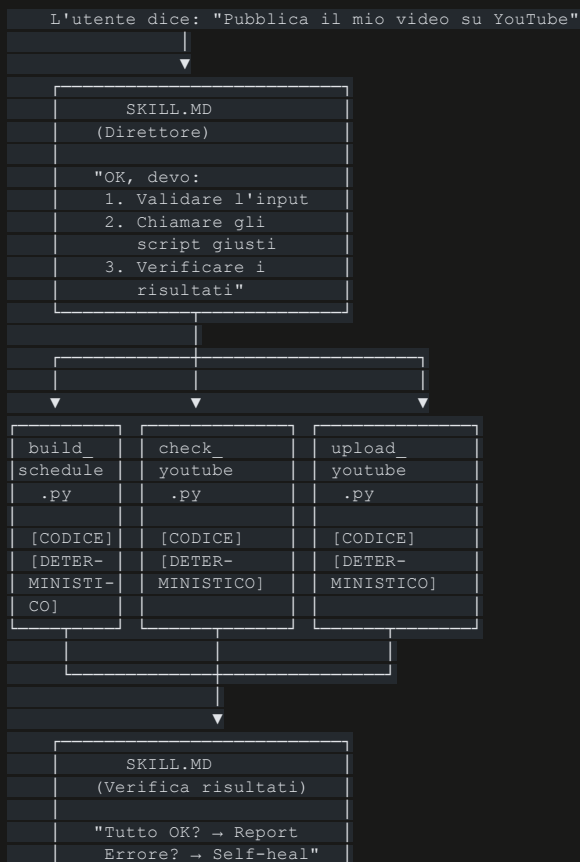
La guida originale chiarisce un malinteso molto comune:

"Molte persone fanno una skill.md dicendo che è semplicemente un markdown, ma questo è un po' forviante."

La skill non è il file markdown. La skill è l'insieme di skill.md + scripts/. Il markdown è il direttore d'orchestra, ma senza i musicisti (gli script) non c'è concerto.

text

INTERAZIONE TRA SKILL.MD E SCRIPTS



27.3 — Il Processo di Self-Healing

Definizione del Concetto

Il self-healing (auto-guarigione) è il processo attraverso il quale una skill è in grado di auto-correggersi quando qualcosa va storto durante l'esecuzione. Questo processo è reso possibile dalla combinazione del file skill.md (che contiene le istruzioni per la gestione degli errori) e del CLAUDE.md (che contiene le regole generali del progetto).

Spiegazione Approfondita

Quando uno script all'interno di una skill fallisce o produce un risultato non conforme, il processo di self-healing si attiva:

text

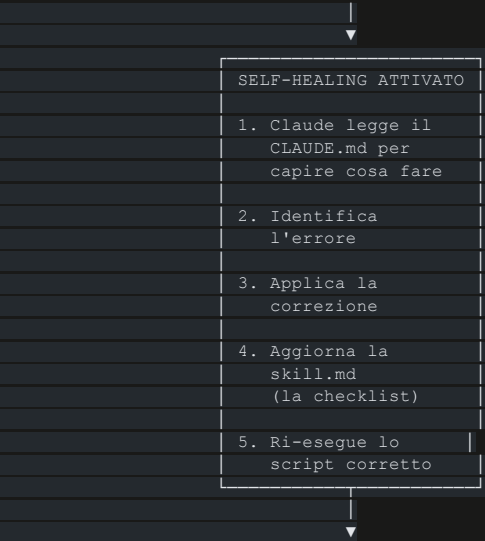
PROCESSO DI SELF-HEALING

ESECUZIONE NORMALE:

skill.md → script_1.py → script_2.py → script_3.py → OUTPUT ✓

QUANDO QUALCOSA VA STORTO:

skill.md → script_1.py → script_2.py → ERRORE! ✗



L'aspetto più importante del self-healing è il Passo 4: Claude aggiorna la checklist (skill.md) per includere la correzione. Questo significa che la prossima volta che la skill viene eseguita, l'errore non si ripresenterà perché la checklist è stata migliorata.

Questo crea un ciclo virtuoso di miglioramento continuo:

text

CICLO DI MIGLIORAMENTO DELLA SKILL

Esecuzione 1 → Errore A → Fix A → Skill aggiornata
Esecuzione 2 → Errore B → Fix B → Skill aggiornata
Esecuzione 3 → Nessun errore → Output perfetto
Esecuzione 4 → Nessun errore → Output perfetto

...la skill diventa sempre più robusta nel tempo

Perché il Self-Healing è Fondamentale

Senza self-healing, ogni errore richiederebbe l'intervento umano. Con il self-healing:

1. Riduzione dell'intervento umano: la skill si ripara da sola nella maggior parte dei casi
2. Miglioramento continuo: ogni errore rende la skill più robusta
3. Scalabilità: potete lanciare skill e andare a fare altro, sapendo che si auto-correggeranno
4. Documentazione automatica: le correzioni vengono codificate nella checklist, creando documentazione vivente

Il Ruolo del CLAUDE.md nel Self-Healing

"Quand'è che il file skill.md entra in funzione? Quando qualcosa va storto. Molti confondono perché molti dicono 'La skill è il file markdown' — la skill non è il file markdown."

Il CLAUDE.md fornisce il contesto generale che Claude usa per capire come reagire agli errori. La skill.md fornisce la procedura specifica per la skill corrente. Insieme, danno a Claude abbastanza informazioni per:

- Diagnosticare cosa è andato storto
- Decidere la strategia di correzione
- Applicare il fix
- Verificare che il fix funzioni
- Aggiornare la documentazione per prevenire recidive

27.4 — Skill vs Script: La Natura Deterministica

Definizione del Concetto

Gli script all'interno di una skill sono deterministici — producono sempre lo stesso output dato lo stesso input. Questo è in contrasto con le risposte dell'LLM, che sono non deterministiche (possono variare anche con lo stesso prompt). Comprendere questa distinzione è fondamentale per costruire skill affidabili.

Spiegazione Approfondita

La guida originale introduce questo concetto quando parla degli hook e lo rafforza parlando delle skill:

"Questi hook possono essere anche il momento in cui attiviamo un sub-workflow. Sono distaccati dal funzionamento dell'LLM. Non sono più legati alla token consumption. Partono ad evento e sono codice, quindi non sono qualcosa di non deterministico."

Lo stesso principio si applica agli script delle skill. Quando Claude esegue uno script Python per caricare un video su YouTube, quel codice:

- Non "interpreta" cosa fare — segue istruzioni precise
- Non "allucina" — esegue operazioni definite
- Non "varia" il risultato — dato lo stesso input, produce lo stesso output
- Non consuma token dell'LLM — è codice tradizionale

Questo è il motivo per cui le skill sono così potenti: combinano la flessibilità dell'LLM (per capire cosa l'utente vuole e orchestrare il processo) con la affidabilità del codice tradizionale (per eseguire le operazioni effettive).

text

CONFRONTO: APPROCCIO LLM-ONLY vs SKILL

APPROCCIO LLM-ONLY (senza skill):

Utente: "Pubblica questo video su YouTube"
Claude LLM:
→ Interpreta la richiesta (non determin.)
→ Cerca di capire come fare (non determin.)
→ Prova a scrivere codice (non determin.)
→ Esegue il codice (deterministico)
→ Verifica il risultato (non determin.)
Risultato: VARIABILE ogni volta
Token consumati: MOLTI
Tempo: LUNGO

APPROCCIO CON SKILL:

Utente: "Usa la skill publish per YouTube"
Claude + Skill:
→ Legge skill.md (checklist fissa)
→ Valida gli input (checklist fissa)
→ Esegue upload_youtube.py (determin.)
→ Verifica il risultato (checklist fissa)
Risultato: CONSISTENTE ogni volta
Token consumati: POCHI
Tempo: BREVE

L'Implicazione per la Costruzione di Skill

Questa comprensione della dualità LLM/codice ha implicazioni dirette per come costruite le vostre skill:

- Tutto ciò che può essere codificato come script, DEVE essere codificato come script: upload di file, chiamate API, trasformazioni di dati, manipolazione di file — tutto questo deve essere codice deterministico
- Solo ciò che richiede ragionamento resta nell'LLM: interpretazione dell'input utente, decisioni creative, gestione degli errori imprevisi, personalizzazione del tono di voce
- La skill.md fa da ponte: coordina quando usare l'LLM e quando usare gli script

CAPITOLO 28

Creare Skill Personalizzate

28.1 — Perché Creare Skill Personalizzate

Definizione del Concetto

Le skill personalizzate sono skill create da voi specificamente per i vostri processi, il vostro stile e le vostre esigenze. Sono l'opposto delle skill generiche importate da marketplace. L'autore della guida è categorico: le skill personalizzate sono la cosa a maggiore ROI che potete fare con Claude Code.

Spiegazione Approfondita

L'autore dimostra la differenza tra skill personalizzate e generiche con un esperimento diretto: genera un LinkedIn Post con la sua skill personalizzata e con una skill importata dal marketplace, usando lo stesso identico input.

Risultato con skill personalizzata:

- Post con la sua formattazione specifica
- Emoji posizionate secondo il suo stile
- Call to action personalizzata per il suo business
- Tono di voce che rispecchia la sua personalità
- Struttura di skimmability (leggibilità rapida) coerente con i suoi post precedenti
- Riferimenti alla sua community e ai suoi programmi

Risultato con skill importata:

- Post generico con M-dashes ("—") tipici dell'AI
- Struttura standard senza personalità
- Nessun riferimento al brand personale
- Tono "robotico" immediatamente riconoscibile come AI
- Formattazione diversa da qualsiasi post precedente dell'autore

"Stesso input, output estremamente diverso."

La differenza non sta nella qualità del codice o nella struttura della skill. La differenza sta nel reference data — i dati di riferimento che alimentano la skill. Ma di questo parleremo in dettaglio nel Capitolo 30.

Il Processo Mentale per Identificare Skill da Creare

Prima di costruire una skill, dovete identificare quali processi nel vostro lavoro sono:

text

CHECKLIST PER IDENTIFICARE SKILL UTILI

☐ RIPETITIVO: Faccio questa cosa più di 1 volta a settimana?
→ SÌ: Candidata ideale per una skill

☐ STRUTTURATO: Il processo segue sempre gli stessi passi?
→ SÌ: Facile da codificare in una checklist

☐ MISURABILE: Posso definire cosa è un "buon risultato"?
→ SÌ: Posso creare verifiche nella skill

☐ DELEGABILE: Potrei spiegare il processo a qualcuno?
→ SÌ: Posso spiegarlo anche a Claude

☐ TIME-CONSUMING: Mi prende più di 10 minuti ogni volta?
→ SÌ: Il risparmio di tempo giustifica la creazione

Se rispondete SÌ a 3 o più domande, quella task

dovrebbe diventare una skill.

Esempi concreti di skill che l'autore ha creato:

- LinkedIn Post Generator: genera post nello stile personale dell'autore
- Audit Skill: lancia automaticamente i tre sub-agenti (Researcher, Reviewer, QA)
- Publish Skill: pubblica contenuti su YouTube, Instagram e Facebook
- Meta Push: gestisce la pubblicazione su piattaforme Meta
- Shorts Creator: crea contenuti short-form per social media

28.2 — Il Processo di Creazione Passo per Passo

Definizione del Concetto

La creazione di una skill personalizzata è un processo in quattro fasi: definire, strutturare, codificare e testare. Non è necessario essere programmatori per iniziare — Claude Code può assistere in ogni fase.

Fase 1: Definire la Skill

Il primo passo è definire chiaramente cosa deve fare la skill. Questo significa rispondere a tre domande:

1. COSA: Quale risultato deve produrre?
2. COME: Quali passi deve seguire per arrivarci?
3. QUANDO: In quali circostanze viene chiamata?

Esempio (LinkedIn Post Generator):

text

```
COSA: Generare un post LinkedIn nel mio stile personale
COME:
1. Ricevere un topic o un'idea dall'utente
2. Analizzare i reference post per estrarre stile e tono
3. Generare una bozza del post
4. Formattare secondo le convenzioni personali
5. Aggiungere CTA appropriata
6. Presentare il risultato per approvazione
```

QUANDO: Ogni volta che voglio creare un nuovo post LinkedIn

Fase 2: Strutturare la Skill

Questa fase consiste nel creare la struttura della cartella e del file skill.md:

text

```
STRUTTURA DA CREARE
=====

.claude/skills/
└─ linkedin-post/
   │ skill.md           ← La checklist/orchestratore
   │ scripts/          ← Gli script eseguibili
   │   └─ generate.py
   │   └─ references/   ← I dati di riferimento
   └─ my_posts.md       ← Post precedenti come esempio
```

Il file skill.md dovrebbe seguire questa struttura:

Markdown

```
# LinkedIn Post Generator

Generate a LinkedIn post matching your writing style.

## Input Validation
- Topic or idea: REQUIRED
- Target audience: OPTIONAL (default: entrepreneurs)
- Post style: OPTIONAL (default: storytelling)
- Include CTA: OPTIONAL (default: yes)

## Process
1. Read reference posts from references/my_posts.md
2. Analyze writing patterns:
  - Sentence length
  - Emoji usage
  - Line spacing
  - Hook structure
  - CTA format
3. Generate draft post
4. Format according to patterns
5. Add appropriate CTA
6. Present to user

## Output Format
- Present the post in a code block
- Show character count
- Indicate target platform format compliance

## Error Handling
- If reference file not found → warn user, generate generic
- If topic too vague → ask for clarification
- If post exceeds LinkedIn character limit → offer to shorten

## Self-Healing
- If output doesn't match reference style → re-analyze
  references and regenerate
- If CTA is missing → add default CTA

- Log any corrections for future improvement
```

Fase 3: Codificare gli Script

Gli script trasformano la parte ripetitiva e deterministica del processo in codice eseguibile. Per il LinkedIn Post Generator, lo script potrebbe gestire:

- Lettura e parsing dei file di riferimento
- Analisi statistica del tono (lunghezza frasi, frequenza emoji, etc.)
- Formattazione del post secondo regole predefinite
- Conteggio caratteri e validazione limiti piattaforma

L'aspetto fondamentale è che non dovete necessariamente scrivere questi script voi stessi. Potete chiedere a Claude:

text

```
"Per favore, guarda la skill.md che ho creato nella
cartella linkedin-post e crea gli script necessari
per implementare il processo descritto. Segui le
best practice della documentazione ufficiale Claude."
```

Claude creerà gli script basandosi sulla checklist nel skill.md.

Fase 4: Testare e Iterare

Una volta creata la skill, testatela con diversi input:

text

Test 1: "Usa la skill LinkedIn Post per creare un post sull'importanza delle skill in Claude Code"

Test 2: "Usa la skill LinkedIn Post per creare un post su come l'AI sta cambiando il business"

Test 3: "Usa la skill LinkedIn Post per creare un post controverso sull'overuse degli Agent Teams"

Per ogni test, valutate:

- Il risultato rispecchia il vostro stile? Se no, migliorate i reference data
- Il processo ha avuto errori? Se sì, migliorate la checklist
- Il formato è corretto? Se no, aggiornate le regole di formattazione
- Il self-healing ha funzionato? Se no, migliorate la sezione Error Handling

Il Prompt Completo per Chiedere a Claude di Creare una Skill

Basandosi sull'approccio mostrato nella guida, ecco un template di prompt efficace:

text

"Per favore, crea una skill completa per [DESCRIZIONE].

La skill deve:

1. Essere posizionata in `.claude/skills/[NOME]/`
2. Avere un file `skill.md` con checklist completa
3. Avere una cartella `scripts/` con gli script necessari
4. Seguire le best practice della documentazione ufficiale Claude
5. Includere gestione errori e self-healing
6. Includere validazione degli input

Per il file `skill.md`, segui la struttura:

- Descrizione
- Input Validation
- Process (step by step)
- Output Format
- Error Handling
- Self-Healing

Per gli script, usa Python e assicurati che siano deterministici e ben documentati.

Se hai bisogno di reference data, chiedimi e te li fornirò."

28.3 — L'Esempio del Social Media Manager

Definizione del Concetto

L'autore della guida mostra il suo Social Media Manager come esempio completo di un progetto basato interamente su skill. Questo progetto controlla tutte le sue piattaforme social (YouTube italiano, YouTube inglese, LinkedIn e altri) tramite un insieme di skill coordinate.

Struttura Reale del Progetto

Dalla guida, la struttura del Social Media Manager dell'autore include le seguenti skill:

text

```
social-media-manager/
├── CLAUDE.md           ← Conciso, essenziale
├── .claude/
│   ├── agents/
│   │   ├── researcher.md
│   │   ├── reviewer.md
│   │   └── qa.md
│   ├── rules/
│   │   └── [regole modulari]
│   └── skills/
│       ├── linkedin-post/    ← Generatore LinkedIn post
│       │   ├── skill.md
│       │   └── scripts/
│       │       ├── references/ ← 50+ post scritti manualmente
│       │       └── publish/    ← Pubblicazione su piattaforme
│       │           ├── skill.md
│       │           └── scripts/
│       │               ├── build_schedule.py
│       │               ├── check_meta.py
│       │               ├── check_youtube.py
│       │               └── upload_youtube.py
│       ├── audit/           ← Lancia i 3 sub-agenti automaticamente
│       │   ├── skill.md
│       │   └── scripts/
│       ├── meta-push/       ← Pubblicazione su Meta
│       │   ├── skill.md
│       │   └── scripts/
│       ├── shorts/         ← Creazione contenuti short
│       │   └── skill.md
│       └── scripts/
```

L'Interazione tra Skill e Sub-agenti

Un dettaglio particolarmente potente emerso dalla guida è la skill Audit:

"Quando gli chiedo 'fammi un audit', devi chiamarmi tre agenti che sono esattamente quelli che vi ho presentato."

Questo significa che la skill Audit è un orchestratore di sub-agenti. Quando l'utente dice "fammi un audit", la skill:

- 1. Chiama automaticamente il Researcher sub-agent
- 2. Chiama automaticamente il Reviewer sub-agent
- 3. Chiama automaticamente il QA sub-agent
- 4. Raccoglie i risultati
- 5. Produce un report consolidato

L'utente non deve sapere che ci sono tre sub-agenti coinvolti. Dice solo "audit" e riceve un report completo. Questa è l'astrazione: la complessità è nascosta dietro un'interfaccia semplice.

Il Tempo di Creazione

L'autore condivide un dato importante sul tempo necessario per creare il suo Social Media Manager:

"C'ho messo 2 ore e mezza a fare planning e dopo ci ha messo 3 ore a costruirlo. Era un one-shot."

Quindi:

- 2,5 ore di planning mode (creazione del piano dettagliato)
- 3 ore di esecuzione in bypass permission (costruzione automatica)
- Totale: 5,5 ore per un sistema completo di gestione social media

Questo sistema, una volta costruito, viene usato ogni giorno per generare contenuti, pubblicarli e gestire le piattaforme. Il ROI è enorme: 5,5 ore di investimento per un sistema che risparmia ore ogni giorno.

L'Interfaccia UI Personalizzata

L'autore menziona brevemente un dettaglio avanzato:

"Questo mio ha poi un'interfaccia UI dove se premo un tasto e non mi piace, il feedback che do va, torna dentro Claude, migliora pian piano anche la generazione di post."

Questo significa che l'autore ha costruito un'interfaccia grafica personalizzata che:

1. Mostra il post generato
2. Permette di dare feedback con un singolo tasto
3. Il feedback viene inviato a Claude Code
4. Claude migliora la skill basandosi sul feedback
5. Le generazioni successive sono progressivamente migliori

Questo è il livello esperto dell'uso delle skill: un sistema che non solo funziona, ma migliora autonomamente con l'uso.

28.4 — Skill che Chiamano Skill

Definizione del Concetto

Le skill possono essere composte — una skill può chiamare altre skill come parte del suo processo. Questo permette di creare catene complesse di operazioni a partire da un singolo comando.

Spiegazione Approfondita

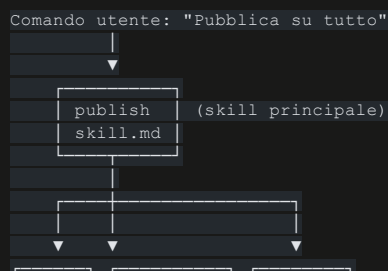
Nell'esempio del Social Media Manager, la skill "publish" potrebbe internamente:

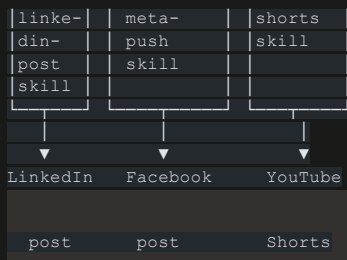
1. Chiamare la skill "linkedin-post" per generare il contenuto
2. Chiamare la skill "meta-push" per pubblicare su Facebook
3. Chiamare la skill "shorts" per creare una versione short del contenuto

Tutto questo da un singolo comando: *"Pubblica il mio ultimo contenuto su tutte le piattaforme."*

text

COMPOSIZIONE DI SKILL





Ogni skill ha i suoi script, le sue verifiche e il suo self-healing. La skill principale coordina tutto, gestisce gli errori propagati e produce un report finale consolidato.

CAPITOLO 29

Il Marketplace delle Skill

29.1 — Dove Trovare Skill Esterne

Definizione del Concetto

Il marketplace delle skill è un ecosistema di skill create dalla community che possono essere importate e utilizzate nei propri progetti. La guida menziona specificamente SkillM come marketplace principale.

Spiegazione Approfondita

L'autore fornisce informazioni concrete sul marketplace:

"SkillM è uno che è fenomenale. Ci sono 351.349 skill."

Il marketplace è organizzato per categorie:

text

CATEGORIE DEL MARKETPLACE SKILLM

└─ Tool Development
└─ Business
└─ Data
└─ AI
└─ Content
└─ Social Media
└─ Documentation

└─ [altre categorie]

Come Trovare Skill nel Marketplace

La procedura pratica mostrata nella guida:

1. Andate su skillm.com (o il sito del marketplace)
2. Navigate per categoria o cercate per keyword
3. Trovate una skill che vi interessa
4. Premete il pulsante "Copy" per copiare il comando di installazione
5. Incollate il comando in Claude Code
6. La skill viene importata nel vostro progetto

Come Importare una Skill

L'importazione è semplice come un prompt:

text

```
"Per favore importa questa skill: [incolla comando copiato dal marketplace] e creami un post di LinkedIn sull'argomento
```

```
[topic]"
```

Claude scarica la skill, la posiziona nella cartella corretta e la esegue immediatamente se richiesto.

L'Avvertimento sulla Sicurezza

La guida menziona un punto importante sulla sicurezza:

"Dovrete stare semplicemente attenti perché qui abbiamo MCP e strumenti sviluppati da terzi che possono contenere malware."

Lo stesso principio si applica alle skill importate dal marketplace:

Rischio	Come mitigarlo
Skill con codice malevolo	Leggete il codice degli script prima di eseguirli
Skill che accedono a dati sensibili	Verificate quali permessi richiede
Skill di bassa qualità	Controllate le valutazioni e i download

29.2 — Quando Importare vs Quando Creare

Definizione del Concetto

Non tutte le skill devono essere create da zero. Alcune volte è più efficiente importare una skill esistente e personalizzarla. Altre volte è indispensabile crearla da zero. La scelta dipende dalla specificità della vostra esigenza.

Framework Decisionale

text

ALBERO DECISIONALE: IMPORTARE O CREARE?

```
La skill richiede il MIO tono di voce personale?
|
|-- Sì → CREA da zero
|   (Nessuna skill esterna avrà il vostro stile)
|
|-- NO → La skill è tecnica/procedurale?
|
|   |-- Sì → IMPORTA dal marketplace
|   |
|   |   |-- Skill di deploy
|   |   |-- Skill di formattazione codice
|   |   |-- Skill di design/CSS
|   |   |-- Skill di test automatici
|   |
|   |-- NO → La skill richiede dati proprietari?
|   |
|   |   |-- Sì → CREA da zero con i tuoi dati
|   |
|   |   |-- NO → IMPORTA e personalizza
```

Perché le Skill di Design Funzionano Meglio come Import

La guida contiene un'osservazione sottile ma cruciale:

"Le skill che funzionano meglio sono generalmente quelle di design, dove abbiamo dei CSS o degli elementi all'interno dei website. Funzionano perché per fare design non abbiamo bisogno di un database molto ricco per allenare l'AI, perché non c'è il nostro tono di voce."

Questo spiega perché certi tipi di skill si possono importare tranquillamente:

- Skill di design/CSS: il "tono di voce" non esiste nel design visivo. Un bottone è un bottone, un layout è un layout. Le skill generiche funzionano bene.
- Skill di codice: il codice funzionale non ha "personalità". Una funzione di autenticazione è la stessa per tutti.
- Skill di deploy: il processo di deployment è standardizzato e non richiede personalizzazione.

Mentre altri tipi di skill devono essere personalizzate:

- Skill di scrittura (post, email, copywriting): il tono di voce è tutto
- Skill di comunicazione (cold outreach, onboarding email): la personalità fa la differenza
- Skill di branding (contenuti brand-specific): ogni brand è unico

CAPITOLO 30

La Qualità dei Dati di Riferimento

30.1 — Perché i Reference Data Sono Tutto

Definizione del Concetto

I reference data (dati di riferimento) sono gli esempi, i campioni e le informazioni che fornite a una skill per insegnarle il vostro stile, le vostre preferenze e i vostri standard di qualità. Sono il "materiale di addestramento" della skill.

Spiegazione Approfondita

Questo è forse il concetto più importante dell'intera sezione skill. L'autore della guida lo afferma con chiarezza:

"Il motivo per cui quando fate cold outreach le email sembrano degli automi, quando scrivete LinkedIn Post fate 87 prompt perché non vi viene l'output che volete, è semplicemente perché il database che avete come riferimento non è sufficientemente ricco."

La qualità del reference data determina la qualità dell'output. Non la skill, non il prompt, non il modello — i dati di riferimento.

L'Esempio Concreto dalla Guida

L'autore mostra il suo progetto di Social Media Manager e rivela:

"Ho una serie infinita di post di LinkedIn che ho scritto manualmente perché lui capisse quali sono le mie idee, qual è la mia voce. Come vedete sono un mezzo cazziliardo di robe."

"Per il mio LinkedIn, la mia skill mette in pasto tutti quelli che vi ho già mostrato, che sono qualche migliaio di righe."

Questo significa che la skill LinkedIn Post Generator dell'autore ha accesso a:

- Decine di post scritti manualmente dall'autore
- Migliaia di righe di testo di riferimento
- Pattern completi di formattazione, emoji, struttura, CTA

Confrontate questo con una skill importata dal marketplace che non ha NESSUN reference data personalizzato. La differenza di output è prevedibilmente abissale.

La Catena della Qualità

text

CATENA DELLA QUALITÀ DELLE SKILL

Reference Data di alta qualità

↓

Skill capisce il VOSTRO stile

↓

Output che sembra scritto da VOI

↓

Minimo editing manuale necessario

↓

Alta produttività, alto ROI

Reference Data scadente o assente

↓

Skill produce output GENERICO

↓

Output che sembra scritto da un robot

↓

Editing manuale estensivo necessario

↓

Bassa produttività, basso ROI

30.2 — Come Costruire Reference Data di Qualità

Definizione del Concetto

La costruzione di reference data di qualità è un processo che richiede tempo e impegno iniziale, ma produce dividendi enormi nel lungo periodo. Non è un processo che potete delegare completamente all'AI — richiede la vostra partecipazione attiva.

La Regola d'Oro

"La capacità di far scrivere l'AI in modo umano è strapagata ed è estremamente difficile."

Questa frase dell'autore contiene un'intuizione business fondamentale: la capacità di produrre contenuti AI che suonano umani è un servizio di alto valore nel mercato attuale. E la chiave per arrivarci sono i reference data.

Processo di Costruzione

text

PROCESSO DI COSTRUZIONE DEI REFERENCE DATA

FASE 1: RACCOLTA (Settimana 1-2)

- ☐ Raccogliete tutto il contenuto che avete già prodotto manualmente:
 - Post LinkedIn
 - Email inviate

- Messaggi di vendita
- Articoli scritti
- Script video
- Presentazioni

- ☐ Organizzate per tipologia e qualità
- ☐ Selezionate solo i migliori (top 20%)

FASE 2: CATEGORIZZAZIONE (Settimana 2-3)

- ☐ Dividete per tipo di contenuto
- ☐ Identificate pattern ricorrenti:
 - Come iniziate (hook)
 - Come strutturate il corpo
 - Come chiudete (CTA)
 - Che emoji usate e dove
 - Che lunghezza hanno i paragrafi
 - Che tono usate (formale, informale, tecnico)

FASE 3: FORMATTAZIONE (Settimana 3)

- ☐ Convertite tutto in formato Markdown
- ☐ Aggiungete annotazioni dove utile:
 - "Questo post ha convertito molto bene perché..."
 - "Questo formato funziona per il pubblico X..."
- ☐ Salvate nella cartella references/ della skill

FASE 4: ITERAZIONE CONTINUA (Ongoing)

- ☐ Ogni volta che producite contenuto di qualità, aggiungetelo ai reference data
- ☐ Ogni volta che un output AI è particolarmente buono, aggiungetelo come esempio
- ☐ Rimuovete periodicamente esempi obsoleti o di bassa qualità

Il Numero Minimo di Riferimenti

Non c'è un numero magico, ma basandosi sull'esperienza dell'autore:

Tipo di Contenuto	Minimo Consigliato	Ideale
Post LinkedIn	20 post	50+ post
Email commerciali	15 email	30+ email
Script video	10 script	20+ script
Articoli blog	10 articoli	25+ articoli

Più reference data avete, più Claude capirà le sfumature del vostro stile. La differenza tra 10 e 50 riferimenti è enorme in termini di qualità dell'output.

30.3 — L'Errore degli M-Dashes e il Tono Robotico

Definizione del Concetto

L'autore identifica specifici marker che rivelano quando un contenuto è stato generato da AI senza reference data adeguati. Questi marker includono l'uso eccessivo di M-dashes ("—"), strutture di frase ripetitive e un tono generalmente "robotico".

Spiegazione Approfondita

"Avete M-dashes quindi trattini molto AI e via dicendo."

Gli M-dashes sono il segnale più riconoscibile di testo generato da AI. Quando vedete frasi come:

- "L'intelligenza artificiale — strumento rivoluzionario del nostro tempo — sta trasformando..."
- "Le skill — se utilizzate correttamente — possono..."
- "Il business moderno — in continua evoluzione — richiede..."

Questi trattini lunghi sono usati dall'AI con una frequenza molto superiore a quella di un essere umano. Un lettore attento li riconosce immediatamente come "scritto da AI".

Come Eliminare il Tono Robotico

La soluzione è duplice:

1. Reference Data Ricchi:

Se la skill ha accesso a 50+ post scritti da voi manualmente e voi non usate mai M-dashes, Claude imparerà a non usarli.

2. Regole Esplicite nella Skill:

Potete aggiungere regole specifiche nella skill.md:

Markdown

Regole di Stile

- NON usare MAI M-dashes ("—")
- NON usare frasi che iniziano con "In un mondo..."
- NON usare "Immagina di..." come hook
- NON usare più di 2 emoji per paragrafo
- Usa frasi brevi (max 15 parole)

- Vai a capo spesso (ogni 1-2 frasi)

La combinazione di reference data ricchi e regole esplicite produce output che sono indistinguibili da contenuto scritto manualmente.

30.4 — Il Valore Commerciale delle Skill Personalizzate

Definizione del Concetto

Le skill personalizzate con reference data di alta qualità rappresentano un asset commerciale significativo. Possono essere vendute come servizio, utilizzate per fornire consulenza o implementate nelle aziende dei clienti.

Spiegazione Approfondita

L'autore della guida fa riferimento specifico alla monetizzazione:

"Quando andrete a vendere questa tipologia di servizio per social media agency, content creation agencies, ecco è estremamente importante la qualità del reference data."

E aggiunge:

"La capacità di far scrivere l'AI in modo umano è strapagata."

Questo posiziona le skill personalizzate come un servizio premium nel mercato dell'AI:

text

MODELLO DI BUSINESS BASATO SU SKILL

SERVIZIO BASE (Basso valore):

"Ti installo Claude Code e ti faccio vedere come usarlo"

→ Prezzo: €200-500

SERVIZIO INTERMEDIO (Medio valore):

"Ti creo un sistema con sub-agenti e skill generiche"

→ Prezzo: €1.000-3.000

SERVIZIO PREMIUM (Alto valore):

"Ti creo skill personalizzate con il TUO tono di voce,
i TUOI reference data, il TUO brand identity.

L'output è indistinguibile da quello che scrivi tu."

→ Prezzo: €5.000-15.000+

SERVIZIO ENTERPRISE:

"Implemento un sistema completo con skill personalizzate,
sub-agenti, agent teams, deployment e monitoring
per la tua azienda da 70 milioni di fatturato."

→ Prezzo: negoziato individualmente

La Lezione Commerciale

L'autore condivide un'intuizione importante per chi vuole monetizzare le skill:

"Non basta solo il database, non basta solo sapere cos'è una skill. Servono una serie di piccole altre accortezze che vi permettono poi di rispecchiare il vostro tono di voce."

Queste "piccole accortezze" — che si apprendono solo con l'esperienza pratica — sono ciò che distingue un'implementazione mediocre da una eccellente. Sono il motivo per cui l'autore può vendere servizi di consulenza ad alto prezzo: non vende la conoscenza di Claude Code (che è accessibile a tutti), ma vende l'esperienza di implementazione maturata in centinaia di progetti.

Riepilogo della Parte 8

In questa Parte avete appreso:

1. Cosa sono le skill: processi codificati e ripetibili composti da skill.md (orchestratore) e scripts/ (codice eseguibile)
2. L'architettura interna: il file skill.md è la checklist/direttore d'orchestra, gli script sono i musicisti che eseguono il lavoro deterministico
3. Il processo di self-healing: quando qualcosa va storto, Claude usa il CLAUDE.md e la skill.md per auto-correggersi e aggiornare la checklist
4. La differenza tra LLM e codice deterministico: gli script sono prevedibili e affidabili, l'LLM gestisce il ragionamento e l'orchestrazione
5. Come creare skill personalizzate: definire, strutturare, codificare, testare — un processo in quattro fasi
6. L'esempio del Social Media Manager: un sistema completo costruito in 5,5 ore che gestisce tutti i social dell'autore
7. Il marketplace delle skill: SkillM con 351.349+ skill disponibili, organizzate per categoria
8. Quando importare vs creare: skill tecniche → importa; skill con tono di voce → crea da zero
9. L'importanza critica dei reference data: la qualità dell'output dipende dalla quantità e qualità dei dati di riferimento, non dalla skill stessa
10. Il valore commerciale: le skill personalizzate con reference data di alta qualità sono un asset vendibile ad alto prezzo
11. Il pattern di conversione MCP → Skill: usare MCP per prototipare, poi convertire in skill per risparmiare contesto

PARTE 9 — MCP (MODEL CONTEXT PROTOCOL)

"L'MCP non è altro che un insieme di skill che sono state scritte da qualcun altro. Potete vederlo semplicemente come una chiavetta USB universale che se connettete ad un altro sistema vi permette di ereditare tutte le funzioni di questa applicazione."

— Dalla guida originale

Introduzione alla Parte 9

Se le skill sono ricette che voi create per il vostro chef (Claude), gli MCP sono interi ricettari scritti da qualcun altro che potete collegare al vostro progetto con un solo comando. Sembrano una soluzione magica — e in parte lo sono — ma come ogni strumento potente, hanno un costo nascosto che bisogna comprendere prima di usarli.

Questa Parte vi insegnerà non solo cosa sono gli MCP e come installarli, ma soprattutto quando usarli e quando non usarli. La guida originale contiene una delle lezioni più importanti dell'intero corso proprio in questa sezione: l'impatto devastante che un MCP mal scelto può avere sul contesto e, di conseguenza, sulla qualità del vostro lavoro.

Questa Parte è composta da quattro capitoli:

Capitolo	Titolo	Focus Principale
31	Comprendere l'MCP	Cos'è, come funziona, l'analogia della chiavetta USB
32	Installare e Gestire gli MCP	Procedura pratica e formati di configurazione
33	MCP vs Skill — Impatto sul Contesto	Il confronto critico e la strategia ottimale
34	Chrome Dev Tool MCP	L'unico MCP raccomandato per uso pratico

CAPITOLO 31

Comprendere l'MCP

31.1 — Cos'è il Model Context Protocol

Definizione del Concetto

L'MCP (Model Context Protocol) è un protocollo standardizzato che permette di collegare servizi e applicazioni di terze parti direttamente a Claude Code. Quando installate un MCP, Claude acquisisce automaticamente la capacità di interagire con quel servizio esterno — leggere dati, scrivere dati, eseguire operazioni — senza che dobbiate scrivere codice o creare skill manualmente.

Spiegazione Approfondita

L'analogia usata nella guida originale è perfetta per comprendere il concetto:

"Potete vederlo semplicemente come una chiavetta USB universale che se voi avete e connettete ad un altro sistema di terze parti, sviluppato da qualcun altro, vi permette di ereditare tutte le funzioni di questa applicazione."

Espandiamo questa analogia per renderla completamente chiara:

text

ANALOGIA: MCP COME CHIAVETTA USB UNIVERSALE

SENZA MCP:

CLAUDE CODE	X	CLICKUP
	←	
"Non so come interagire con ClickUp"	Nessuna connessione	"Ho tutte le funzionalità ma Claude non può usarli"

CON MCP:

CLAUDE CODE		CLICKUP
	←	
"Ora so fare tutto quello che ClickUp sa fare!"	MCP chiavetta USB	"Claude ora può usare tutte le mie funzionalità"

Quando collegate la "chiavetta USB" (l'MCP), Claude eredita automaticamente tutte le capacità dell'applicazione collegata. Se collegate l'MCP di ClickUp, Claude sa:

- Creare task in ClickUp
- Leggere le board
- Assegnare compiti
- Aggiornare stati
- Cercare nel workspace
- E tutte le altre funzionalità che ClickUp espone tramite il suo MCP

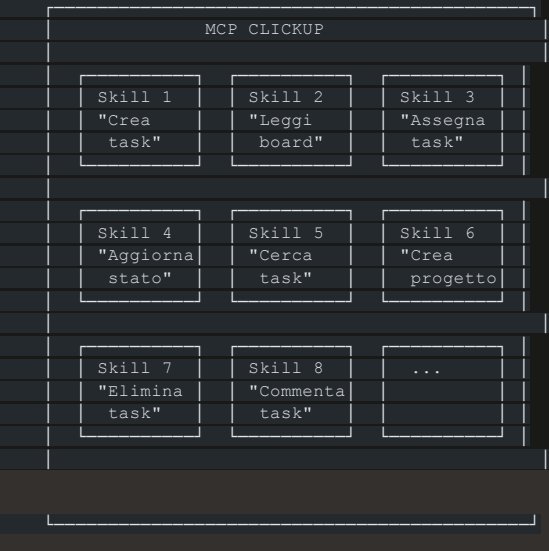
Il Meccanismo Sottostante

Per comprendere perché gli MCP funzionano, bisogna capire che un MCP è essenzialmente un pacchetto di skill pre-costruite dal fornitore del servizio:

text

STRUTTURA CONCETTUALE DI UN MCP

MCP di ClickUp = insieme di skill pre-costruite:



Ognuna di queste "skill interne" all'MCP ha:

- Una descrizione di cosa fa
- I parametri che accetta
- Il formato della risposta
- Le istruzioni per Claude su come utilizzarla

Tutte queste descrizioni vengono caricate nel contesto di Claude quando l'MCP è installato. Ed è esattamente qui che nasce il problema principale degli MCP, che vedremo in dettaglio nel Capitolo 33.

Perché l'MCP è Diventato Comune Solo Recentemente

La guida menziona un dettaglio temporale importante:

"Solo ultimamente è diventato molto più comune e molto più utilizzato."

Questo accade perché il protocollo MCP è relativamente nuovo e la sua adozione è stata graduale. All'inizio, pochi servizi offrivano MCP compatibili con Claude Code. Man mano che Claude Code è cresciuto in popolarità e che Anthropic ha standardizzato il protocollo, sempre più aziende hanno iniziato a creare i propri MCP.

L'effetto è stato una rapida espansione dell'ecosistema: oggi esistono centinaia di MCP per servizi diversi — da ClickUp a GitHub, da Slack a database, da servizi email a strumenti di analytics.

31.2 — Tipologie di MCP

Definizione del Concetto

Non tutti gli MCP sono uguali. Esistono differenze significative tra gli MCP in termini di peso (consumo di contesto), modalità di funzionamento e origine (chi li ha sviluppati). Comprendere queste differenze è essenziale per prendere decisioni informate su quali installare.

MCP Leggeri vs MCP Pesanti

La guida originale dimostra questa distinzione con dati concreti:

text

CONFRONTO: MCP LEGGERO vs MCP PESANTE

MCP LEGGERO – Chrome Dev Tool:

Consumo contesto: ~0,1%
Funzionalità: navigazione web, screenshot, scraping
Numero di "skill interne": poche
Descrizioni: concise
IMPATTO: trascurabile

MCP PESANTE – ClickUp:

Consumo contesto: ~27%
Funzionalità: gestione completa dei progetti e task
Numero di "skill interne": molte
Descrizioni: dettagliate
IMPATTO: devastante

DIFFERENZA: 270 volte più pesante!

Un MCP leggero come Chrome Dev Tool occupa lo 0,1% del contesto. Un MCP pesante come ClickUp occupa il 27%. Questo significa che ClickUp è 270 volte più pesante di Chrome Dev Tool in termini di impatto sul contesto.

MCP Built-in vs MCP di Terze Parti

La guida menziona che Claude Code ha già degli MCP built-in (integrati):

text

TIPOLOGIE DI MCP PER ORIGINE

1. BUILT-IN (integrati in Claude Code):
 - └─ Già presenti, non richiedono installazione
 - └─ Fanno parte del system prompt
 - └─ Consumo incluso nel 10% base del system prompt
 2. DI TERZE PARTI (installati dall'utente):
 - └─ Richiedono installazione manuale
 - └─ Aggiungono consumo di contesto AGGIUNTIVO
 - └─ Qualità e sicurezza variabili
 - └─ Possono contenere malware → ATTENZIONE
 3. CUSTOM (creati dall'utente):
 - └─ Configurati nel file .mcp.json del progetto
 - └─ Completamente sotto il vostro controllo
- └─ Consumo dipende dalla complessità

MCP "On-Demand" vs MCP "Always-On"

Un concetto importante che emerge dalla guida è la distinzione tra MCP che vengono caricati sempre e MCP che vengono chiamati solo quando necessario:

"Non tutti gli MCP sono i cosiddetti MCP di third, che vengono chiamati solamente a chiamata o a bisogno."

Questo è un punto tecnico cruciale:

Tipo	Comportamento	Impatto Contesto
Always-On	Le descrizioni di tutte le funzionalità sono caricate SEMPRE nel contesto	PERMANENTE — occupa contesto anche quando non lo usate
On-Demand	Le descrizioni vengono caricate solo quando l'MCP viene effettivamente chiamato	TEMPORANEO — occupa contesto solo durante l'uso

Il problema principale è che la maggior parte degli MCP di terze parti sono Always-On: le loro descrizioni vengono caricate nel contesto all'avvio della sessione e ci restano per tutta la durata, consumando spazio anche quando non state usando quel servizio.

Implicazione Pratica

Questa distinzione ha un'implicazione enorme per la strategia di utilizzo:

- Se installate 3 MCP pesanti (tutti Always-On), potreste trovarvi con il 60-80% del contesto occupato prima ancora di scrivere il primo messaggio
- È come salire in macchina con il bagagliaio pieno di attrezzi che non userete mai durante quel viaggio: occupano spazio e rendono la macchina più lenta

31.3 — Dove Trovare gli MCP

Definizione del Concetto

Gli MCP sono distribuiti attraverso vari canali: repository GitHub, marketplace dedicati e documentazione ufficiale dei servizi che li offrono.

Il Marketplace Principale

La guida indica Awesome MCP Servers come risorsa principale:

"MCP Server find — troveremo Awesome MCP Servers. Questa è un'ottima piattaforma."

Questo repository contiene un elenco curato di MCP organizzati per categoria e verificati dalla community.

Come Navigare e Scegliere

Quando cercate un MCP, valutate:

text

CHECKLIST DI VALUTAZIONE MCP

- ☐ FUNZIONALITÀ: Fa quello che mi serve?
 - Leggete la lista delle funzionalità esposte
- ☐ PESO: Quanto contesto consuma?
 - Se possibile, verificate prima di installare
 - Installate, fate /context, verificate il consumo
- ☐ SICUREZZA: È sviluppato da una fonte affidabile?
 - Preferite MCP ufficiali (sviluppati dal servizio stesso)
 - Es: MCP di ClickUp sviluppato da ClickUp = più sicuro
 - MCP di ClickUp sviluppato da "random_user_123" = rischio
- ☐ MANUTENZIONE: È aggiornato regolarmente?
 - Controllate la data dell'ultimo commit su GitHub
 - MCP non aggiornati possono avere bug o incompatibilità
- ☐ NECESSITÀ: Mi serve DAVVERO come MCP?
 - Posso ottenere lo stesso risultato con una skill?
 - Lo userò abbastanza spesso da giustificare

il consumo permanente di contesto?

CAPITOLO 32

Installare e Gestire gli MCP

32.1 — Il Formato JSON di Configurazione

Definizione del Concetto

Ogni MCP ha un file di configurazione in formato JSON che contiene le informazioni necessarie per il collegamento. Questo JSON definisce come Claude Code deve comunicare con il servizio esterno.

Spiegazione Approfondita

La guida spiega il formato JSON:

"Questo non è altro che un formato JSON che io ora ho copiato. Il formato JSON lo vediamo perché è contenuto in parentesi graffe, ha la prima parte che si chiamano key (chiavi), la seconda parte che si chiamano value (valori), e sono divisi da questi due punti."

Un file di configurazione MCP ha tipicamente questa struttura:

JSON

```
{
  "mcpServers": {
    "nome-del-servizio": {
      "command": "npx",
      "args": [
        "-y",
        "@nome-pacchetto/mcp-server"
      ],
      "env": {
        "API_KEY": "la-vostra-chiave-api",
        "WORKSPACE_ID": "il-vostro-workspace"
      }
    }
  }
}
```

Analisi della struttura:

Elemento	Significato
mcpServers	Contenitore di tutti gli MCP configurati
nome-del-servizio	Identificativo dell'MCP (es: "clickup", "chrome-devtools")
command	Il comando per avviare il server MCP
args	Gli argomenti passati al comando
env	Le variabili d'ambiente (API key, credenziali, etc.)

Dove Va il File di Configurazione

Il file di configurazione MCP si chiama `.mcp.json` e può essere posizionato a diversi livelli:

text

POSIZIONAMENTO DEL FILE .mcp.json

LIVELLO LOCAL (dentro il progetto):

```
progetto/  
└── .mcp.json      ← MCP disponibili solo in questo progetto
```

LIVELLO GLOBAL (nel computer dell'utente):

```
~/.claude/  
  
└── .mcp.json      ← MCP disponibili in TUTTI i progetti
```

La scelta del posizionamento segue la stessa logica delle regole e dei sub-agenti:

- Local: quando l'MCP serve solo per un progetto specifico
- Global: quando l'MCP è utile in tutti i progetti (come Chrome Dev Tool)

32.2 — Procedura di Installazione

Definizione del Concetto

L'installazione di un MCP può avvenire in diversi modi, dal più semplice (prompt a Claude) al più manuale (editing diretto del file JSON).

Metodo 1 — Installazione Tramite Prompt (Raccomandato)

Il metodo più semplice, mostrato nella guida:

text

"Per favore installa il [nome] MCP da questo link:

[link alla pagina del MCP o al JSON di configurazione]"

Claude:

1. Legge il link o il JSON fornito
2. Identifica il tipo di MCP
3. Scarica e configura automaticamente
4. Aggiorna il file .mcp.json
5. Conferma l'installazione

Esempio concreto dalla guida per Chrome Dev Tool:

text

"Per favore installa Dev Tool MCP"

Claude cerca la documentazione, trova il JSON di configurazione e lo installa automaticamente.

Metodo 2 — Installazione Tramite Dev Tool MCP

Un metodo particolarmente elegante mostrato nella guida: usare il Chrome Dev Tool MCP (già installato) per navigare alla pagina di un altro MCP e installarlo:

text

```
"Per favore, usando il Dev Tool MCP, guarda questo
link [pagina del MCP su GitHub] e collegami l'MCP
di ClickUp a questo progetto."
```

In questo caso, Claude:

1. Usa il Dev Tool MCP per navigare alla pagina GitHub del MCP
2. Legge le istruzioni di installazione dalla pagina
3. Identifica il JSON di configurazione
4. Installa l'MCP seguendo le istruzioni trovate

Metodo 3 — Installazione con Comando Specifico

Alcuni MCP hanno comandi di installazione specifici. Dalla guida, l'esempio di ClickUp:

"Cloud Code, use the following command. Once you open Claude Code session, run [comando] to go through the authentication flow."

In questo caso:

1. Copiate il comando dalla documentazione del MCP
2. Incollatelo nel terminal di Claude Code
3. Seguite il processo di autenticazione (se richiesto)
4. Riavviate la sessione di Claude Code
5. Verificate con `/mcp` che l'MCP sia attivo

Il Processo di Autenticazione

Molti MCP richiedono un processo di autenticazione per accedere al servizio esterno. La guida mostra questo processo con l'esempio di ClickUp:

PROCESSO DI AUTENTICAZIONE MCP

Passo 1: Installazione dell'MCP
→ Claude configura il server MCP locale

Passo 2: Riavvio della sessione
→ "Chiudi e riavvia Claude Code"

Passo 3: Verifica connessione
→ Eseguire `/mcp` per verificare lo stato

Passo 4: Autenticazione
→ "ClickUp ha bisogno di un'autenticazione"
→ "Enter to confirm"
→ Si apre il browser
→ Login nel servizio (es: ClickUp)
→ Autorizzazione dell'accesso

Passo 5: Conferma
→ Il terminale conferma la connessione

```
→ L'MCP è ora operativo
```

32.3 — Verificare e Gestire gli MCP Installati

Definizione del Concetto

Dopo l'installazione, è fondamentale verificare che l'MCP funzioni correttamente e monitorare il suo impatto sul contesto.

Il Comando `/mcp`

Per verificare quali MCP sono installati e il loro stato:

```
/mcp
```

Questo comando mostra:

- L'elenco degli MCP installati
- Lo stato di ciascuno (connesso, disconnesso, errore)
- Eventuali problemi di autenticazione

Verificare l'Impatto sul Contesto

Immediatamente dopo l'installazione di un nuovo MCP, eseguite:

```
/context
```

E confrontate i numeri con quelli precedenti all'installazione. Se l'impatto è eccessivo (più del 5-10% per un singolo MCP), valutate se ne vale la pena.

Come Rimuovere un MCP

La rimozione è altrettanto semplice dell'installazione:

```
"Per favore rimuovi l'MCP [nome]"
```

L'autore della guida lo dimostra con l'esempio di Canva:

"Per favore puoi rimuovere il mio Cloud AI Canva MCP? Non mi serve."

Claude rimuove la configurazione dal file `.mcp.json` e libera il contesto corrispondente.

La Gestione come Routine

L'installazione e la rimozione degli MCP dovrebbe essere una routine consapevole, non un'azione casuale:

ROUTINE DI GESTIONE MCP

- PRIMA DI INIZIARE UN PROGETTO:
- ☐ Verificare quali MCP sono installati (/mcp)
 - ☐ Verificare il loro impatto sul contesto (/context)
 - ☐ Rimuovere MCP non necessari per questo progetto
 - ☐ Installare MCP specifici necessari per questo progetto

- DURANTE IL LAVORO:
- ☐ Monitorare il contesto regolarmente
 - ☐ Se un MCP non viene più usato → rimuoverlo

- ALLA FINE DEL PROGETTO:
- ☐ Rimuovere MCP project-specific

- ☐ Mantenere solo MCP globali (es: Chrome Dev Tool)

CAPITOLO 33

MCP vs Skill — Impatto sul Contesto

33.1 — L'Esperimento Cruciale della Guida

Definizione del Concetto

La guida originale contiene quello che è forse l'esperimento più illuminante dell'intero corso: il confronto diretto tra l'impatto sul contesto degli MCP e delle skill. Questo confronto rivela una verità fondamentale che cambia radicalmente il modo in cui si dovrebbero usare gli MCP.

Spiegazione Approfondita — I Numeri

L'autore esegue un test sistematico. Prima verifica il contesto con solo Chrome Dev Tool installato:

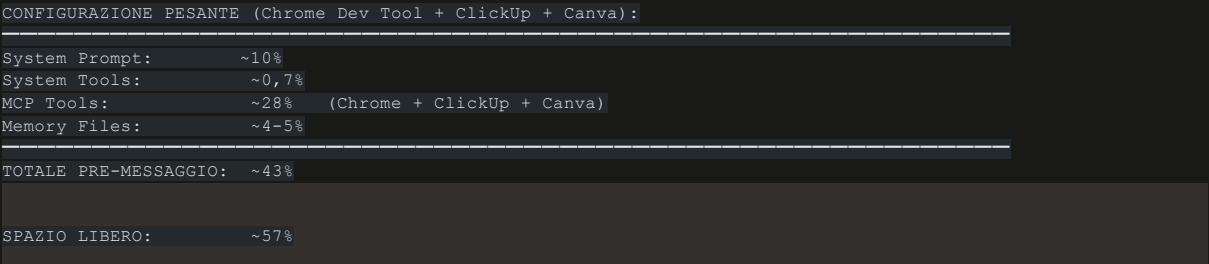
CONFIGURAZIONE LEGGERA (solo Chrome Dev Tool):

System Prompt:	~10%
System Tools:	~0,7%
MCP Tools:	~0,1% (Chrome Dev Tool)
Memory Files:	~4-5%

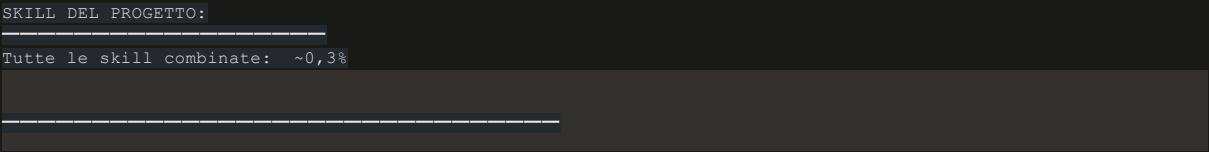
TOTALE PRE-MESSAGGIO: ~15-16%

SPAZIO LIBERO: ~84-85%

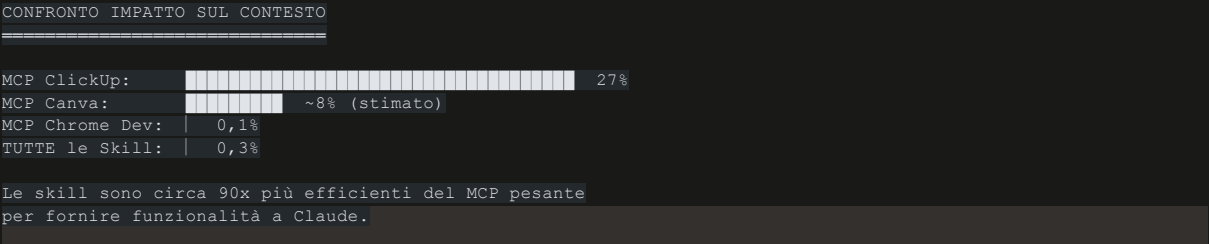
Poi installa ClickUp e Canva e verifica di nuovo:



E poi confronta con il consumo delle skill:



Il Confronto Visivo



La Conclusione dell'Autore

"Quando e come usate gli MCP? Beh sostanzialmente gli MCP sono una chiavetta universale che ti permette di avere tutte le varie skill e che ti permette di chiamare tutti i servizi. Vi consiglio di utilizzarle soprattutto per fare quello che noi definiremo un MVP, quindi per assicurarci che la cosa che vogliamo fare funzioni. Poi quello che faremo è chiedere a Claude: 'Ora che sai cercarmi le cose dentro al workspace, creami una skill adatta a quello che vuoi fare.' E dopo una volta fatta la skill, toglieremo l'MCP."

Questa è la strategia ottimale. Ripetiamola perché è fondamentale.

33.2 — La Strategia MCP → Skill Conversion

Definizione del Concetto

La MCP → Skill Conversion è il processo strategico di usare un MCP come strumento di prototipazione rapida per poi convertire le funzionalità necessarie in skill native, rimuovendo l'MCP e liberando il contesto.

Il Processo in Dettaglio

PROCESSO MCP → SKILL CONVERSION

FASE 1: INSTALLAZIONE MCP (temporanea)

- Installate l'MCP del servizio che vi interessa
- Verificate l'impatto sul contesto (/context)
- Annotate mentalmente: "Questo è temporaneo"

FASE 2: PROTOTIPAZIONE CON MCP

- Usate l'MCP per fare quello che vi serve
- Testate tutte le funzionalità necessarie
- Verificate che il collegamento funzioni
- Identificate QUALI funzionalità vi servono davvero (probabilmente solo 2-3 su 20+ disponibili)

FASE 3: CONVERSIONE IN SKILL

- Chiedete a Claude:
"Ora che sai come funziona [servizio],
creami una skill che faccia [operazione specifica].
La skill deve usare l'API di [servizio] direttamente
senza bisogno dell'MCP."
□ Claude crea la skill con gli script necessari
- La skill fa chiamate API dirette al servizio

FASE 4: RIMOZIONE MCP

- Testate che la skill funzioni indipendentemente
- Rimuovete l'MCP: "Rimuovi l'MCP [nome]"
- Verificate con /context che il contesto sia libero
- Verificate che la skill continui a funzionare

RISULTATO:

PRIMA: MCP ClickUp = 27% del contesto per TUTTE le funzionalità
DOPO: Skill custom = 0,1% del contesto per LE funzionalità che servono
RISPARMIO: 26,9% di contesto → enorme miglioramento

Perché Funziona

La ragione per cui questa strategia è così efficace è che un MCP carica le descrizioni di TUTTE le funzionalità del servizio nel contesto, anche quelle che non userete mai. Se ClickUp ha 50 funzionalità e voi ne usate solo 3, state pagando il "peso" di 47 funzionalità inutili.

Con una skill personalizzata, caricate nel contesto solo le istruzioni per le 3 funzionalità che vi servono. Il risparmio è proporzionale al rapporto tra funzionalità totali e funzionalità necessarie.

ESEMPIO NUMERICO

MCP ClickUp: 50 funzionalità × ~5.400 token ciascuna = 270.000 token
Voi usate: 3 funzionalità

CON MCP: 270.000 token nel contesto (27% su 1M)

CON SKILL: ~3.000 token nel contesto (0,3% su 1M)

EFFICIENZA: 90x migliore con la skill

Quando NON Convertire (Eccezioni)

Ci sono situazioni in cui ha senso mantenere l'MCP invece di convertire in skill:

Situazione	Motivo per Mantenere l'MCP
Usate molte funzionalità del servizio (>10)	Creare 10+ skill separate sarebbe più dispendioso
Il servizio cambia API frequentemente	Le skill diventerebbero obsolete rapidamente, l'MCP viene aggiornato dal fornitore
Siete in fase esplorativa	Non sapete ancora quali funzionalità vi servono
L'MCP è leggero (<1% contesto)	Il costo di mantenimento è trascurabile

33.3 — Il Collegamento con il Lost in the Middle

Definizione del Concetto

L'impatto degli MCP sul contesto non è solo quantitativo (occupano spazio) ma anche qualitativo: gli MCP pesanti spostano le vostre informazioni utili nella zona "Lost in the Middle", degradando la qualità delle risposte.

Spiegazione Approfondita

La guida collega esplicitamente l'impatto degli MCP al fenomeno del Lost in the Middle:

"Vi ricordate il Lost in the Middle di cui abbiamo discusso all'inizio? Vuol dire che noi siamo arrivati già qua. Abbiamo fatto una cosa orribile per caricare queste tipologie di MCP."

Ecco cosa succede visivamente:

IMPATTO MCP SUL LOST IN THE MIDDLE

SENZA MCP PESANTI:

System Prompt [INIZIO - alta attenzione]	10%
CLAUDE.md [INIZIO - alta attenzione]	5%
Messaggi [MEZZO → FINE]	20%
Ultimo prompt [FINE - alta attenzione]	1%
SPAZIO LIBERO	64%

→ Le vostre informazioni sono nelle zone di ALTA attenzione
→ Risultato: risposte eccellenti

CON MCP PESANTI (27% occupato):

System Prompt [INIZIO - alta attenzione]	10%
MCP PESANTE [INIZIO-MEZZO]	27%
CLAUDE.md [MEZZO - BASSA attenzione]	5% ← PROBLEMA!
Messaggi [MEZZO - BASSA attenzione]	20% ← PROBLEMA!
Ultimo prompt [FINE - alta attenzione]	1%
SPAZIO LIBERO	37%

→ Il CLAUDE.md e i messaggi sono nella zona di BASSA attenzione
→ Risultato: Claude "dimentica" le vostre regole e istruzioni

→ Risposte degradate significativamente

L'MCP pesante non solo occupa spazio, ma spinge le vostre informazioni importanti (CLAUDE.md, regole, messaggi) nella zona cieca del Lost in the Middle. Questo è il doppio danno degli MCP pesanti.

La Regola Pratica

"Quello che noi vogliamo fare è andare a costruire qualcosa che ci permetta di essere efficienti. Ci permetta di agire nelle fasi qui [inizio e fine] con le cose più importanti. Non vogliamo mai che il nostro prompt iniziale sia nell'intervallo in cui sostanzialmente l'LLM perde qualsiasi cosa."

Tradotto in regola pratica:

Se un MCP occupa più del 5% del contesto, valutate seriamente la conversione in skill.

Sopra il 5%, l'MCP inizia a spingere le vostre informazioni nella zona Lost in the Middle. Sopra il 15%, l'impatto è significativo. Sopra il 25%, è devastante.

CAPITOLO 34

Chrome Dev Tool MCP

34.1 — L'Unico MCP Raccomandato

Definizione del Concetto

Il Chrome Dev Tool MCP è l'unica integrazione MCP che l'autore della guida raccomanda esplicitamente per uso pratico e aziendale. È un MCP leggero che permette a Claude di navigare il web, fare screenshot, leggere contenuti di pagine e interagire con i browser.

Spiegazione Approfondita

"La Chrome Dev Tool Integration è l'unica integrazione di Claude Code che io raccomando a livello pratico e aziendale. Non ho trovato nessun'altra che abbia valore per quanto mi riguarda."

Questa affermazione è forte e deliberata. L'autore, che implementa Claude Code in aziende che generano fino a 70 milioni di euro di fatturato, dice che tra tutti gli MCP disponibili, solo uno vale la pena di mantenere installato permanentemente.

Le ragioni sono:

- 1. Peso trascurabile: solo 0,1% del contesto
- 2. Utilità universale: la navigazione web è utile in praticamente ogni progetto
- 3. Complementarità con le skill: permette di navigare a siti e fare screenshot che le skill da sole non potrebbero fare
- 4. Nessun equivalente in skill: la navigazione web interattiva è difficile da replicare come skill
- 5. Strumento di verifica: essenziale per il ciclo Task-Do-Verify nella costruzione di siti web

34.2 — Funzionalità del Chrome Dev Tool MCP

Definizione del Concetto

Il Chrome Dev Tool MCP fornisce a Claude la capacità di controllare un browser Chrome come se fosse un utente umano. Può navigare pagine, leggere contenuti, fare screenshot, cliccare elementi e interagire con interfacce web.

Le Capacità Specifiche

FUNZIONALITÀ DEL CHROME DEV TOOL MCP

- 1. NAVIGAZIONE WEB
 - └─ Aprire URL specifici
 - └─ Navigare tra pagine
 - └─ Seguire link
 - └─ Es: "Vai alla documentazione ufficiale di Anthropic"
- 2. SCREENSHOT
 - └─ Catturare screenshot di pagine web
 - └─ Screenshot full-page
 - └─ Screenshot di elementi specifici
 - └─ Es: "Fai uno screenshot del sito e confrontalo con l'immagine di riferimento"
- 3. LETTURA CONTENUTI
 - └─ Leggere il testo di una pagina web
 - └─ Estrarre informazioni strutturate
 - └─ Fare summary di pagine
 - └─ Es: "Leggi questa pagina e fammene un riassunto"
- 4. INTERAZIONE
 - └─ Cliccare bottoni
 - └─ Compilare form
 - └─ Scrollare pagine
 - └─ Es: "Vai su Google e cerca [query]"
- 5. SCRAPING
 - └─ Estrarre dati da pagine web
 - └─ Raccogliere informazioni strutturate

Es: "Raccogli tutti i prezzi da questa pagina"

Applicazioni Pratiche dalla Guida

1. Verifica di siti web costruiti:

Il Chrome Dev Tool MCP è fondamentale per il ciclo di verifica screenshot nella costruzione di siti. Claude può:

- Fare screenshot del sito in costruzione
- Confrontarlo con l'immagine di riferimento
- Identificare le differenze
- Correggerle
- Ripetere il ciclo

2. Ricerca senza API:

"Quando non ci sono API, potrete usare questo tool qui e vi permette di girare nel web."

Questo è un caso d'uso cruciale: molti servizi non offrono API pubbliche. Con il Chrome Dev Tool MCP, Claude può comunque interagire con questi servizi navigando la loro interfaccia web come farebbe un umano.

3. Pubblicazione su piattaforme:

Nella skill "publish" dell'autore, Instagram viene gestito tramite Chrome Dev Tool MCP perché non è pratico usare l'API di Instagram:

"YouTube: Python API. Instagram: tramite Chrome Dev Tool MCP."

Questo mostra come l'MCP può essere integrato nelle skill come strumento complementare.

4. Installazione di altri MCP:

Come visto nel Capitolo 32, il Chrome Dev Tool MCP può essere usato per navigare alle pagine GitHub di altri MCP, leggere le istruzioni di installazione e aiutare a installarli.

34.3 — Come Installare Chrome Dev Tool MCP

Procedura di Installazione

La guida mostra il processo:

Passo 1: Trovare la pagina dell'MCP

- Cercate "Chrome Dev Tool MCP" su Google
- Trovate la repository GitHub ufficiale

Passo 2: Copiare il JSON di configurazione

- Sulla pagina GitHub, trovate il JSON
- Copiate il blocco di configurazione

Passo 3: Installare tramite Claude Code

- "Per favore installa Dev Tool MCP"
- Oppure: incollate il JSON e chiedete di installarlo

Passo 4: Autenticazione

- Al primo uso, Claude vi chiederà di autenticare
- Si aprirà Chrome
- Confermate l'accesso

Passo 5: Verifica

- "Per favore accedi a google.com usando il Dev Tool MCP"

- Se funziona, l'installazione è completa

Consumo di Token

"Consumando una quantità anche ragionevole di token perché non è eccessivamente costoso."

Il Chrome Dev Tool MCP è progettato per essere efficiente:

- Le navigazioni semplici consumano pochi token
- Gli screenshot sono compressi automaticamente
- La lettura di pagine è ottimizzata per estrarre solo il testo rilevante
- L'impatto permanente sul contesto è solo dello 0,1%

Questo lo rende ideale per un uso frequente senza preoccupazioni di costo.

34.4 — Il Pattern Strategico Completo per gli MCP

Riepilogo della Strategia Ottimale

Combinando tutti i concetti appresi in questa Parte, ecco la strategia completa per la gestione degli MCP:

STRATEGIA COMPLETA DI GESTIONE MCP

LIVELLO BASE (per tutti):

- ✓ Installate Chrome Dev Tool MCP → tenetelo SEMPRE
- ✓ Non installate nient'altro a meno che non serva
- ✓ Consumo contesto base: ~0,1%

QUANDO SERVE UN SERVIZIO ESTERNO:

1. Installate l'MCP del servizio
2. Verificate il consumo con /context
3. Usate l'MCP per capire COME funziona il servizio
4. Identificate le 2-3 funzionalità che vi servono
5. Chiedete a Claude di creare skill per quelle funzionalità
6. Testate le skill indipendentemente
7. Rimuovete l'MCP
8. Verificate con /context che il contesto sia libero

ECCEZIONI (mantenete l'MCP):

- MCP con consumo < 1% del contesto
- MCP che usate quotidianamente con molte funzionalità
- MCP in fase di esplorazione attiva

MAI:

- Non installate più di 2-3 MCP contemporaneamente
- Non lasciate MCP installati che non state usando
- Non ignorate l'impatto sul contesto

- Non assumete che "più MCP = migliore"

Riepilogo della Parte 9

In questa Parte avete appreso:

1. Cos'è un MCP: un protocollo che collega servizi esterni a Claude Code, come una "chiavetta USB universale" che eredita tutte le funzionalità del servizio
2. Le tipologie di MCP: leggeri vs pesanti, built-in vs terze parti, always-on vs on-demand
3. Il formato JSON di configurazione: struttura con key-value che definisce come Claude comunica con il servizio
4. Come installare un MCP: tramite prompt, tramite Dev Tool MCP o tramite comando specifico
5. L'impatto devastante degli MCP pesanti sul contesto: ClickUp occupa il 27% del contesto, 270 volte più del Chrome Dev Tool
6. La strategia MCP → Skill Conversion: usare l'MCP per prototipare, poi convertire in skill e rimuovere l'MCP
7. Il collegamento con il Lost in the Middle: gli MCP pesanti spingono le vostre informazioni nella zona cieca del contesto
8. Il Chrome Dev Tool MCP: l'unico MCP raccomandato, con 0,1% di consumo e funzionalità universali
9. La regola del 5%: se un MCP occupa più del 5% del contesto, valutate la conversione in skill
10. La strategia completa: Chrome Dev Tool sempre, tutto il resto temporaneo e convertito in skill

PARTE 10 — FUNZIONALITÀ AVANZATE E DEPLOYMENT

"Non mi resta altro che ringraziarvi per avere seguito il corso di Cloud per Business. Ovviamente c'è un mondo addizionale a tutto questo, ma questo vi darà le basi per costruire qualsiasi cosa voi vogliate."

— Dalla guida originale

Introduzione alla Parte 10

Questa è la Parte finale del manuale. Avete percorso un cammino che vi ha portato dalle fondamenta (cos'è Claude Code, come si installa) attraverso l'architettura del progetto (CLAUDE.md, cartella .claude, livelli local/global/enterprise), il context management, i sub-agenti, le skill e gli MCP. Ora è il momento di completare il quadro con le funzionalità avanzate che trasformano i vostri progetti da esperimenti locali a sistemi produttivi e monetizzabili.

Questa Parte copre quattro argomenti che, sebbene possano sembrare indipendenti, sono in realtà profondamente collegati: gli hook automatizzano i vostri workflow, la memoria li rende persistenti, il version control li rende sicuri e il deployment li rende accessibili al mondo.

Capitolo	Titolo	Focus Principale
35	Hooks — Automazione Basata su Eventi	Script automatici che si attivano a eventi specifici
36	Auto Memory e Persistenza tra Sessioni	Come Claude ricorda informazioni tra sessioni diverse
37	Git Worktrees e Version Control	Lavorare in parallelo con branch isolati e sicuri
38	Deployment e Monetizzazione	Portare skill e applicazioni nel cloud per generare valore

CAPITOLO 35

Hooks — Automazione Basata su Eventi

35.1 — Cosa Sono gli Hooks

Definizione del Concetto

Un hook è uno script personalizzato che si avvia automaticamente prima o dopo ogni chiamata di strumenti da parte di Claude. In parole semplici, un hook è un'azione automatica che scatta ogni volta che un certo evento si verifica nel vostro workflow di Claude Code. Immaginate un hook come una trappola a molla: ogni volta che qualcosa passa sopra (l'evento), la molla scatta (lo script si esegue).

Spiegazione Approfondita

La guida originale introduce gli hook con una definizione precisa:

"Che cosa sono gli hook? Ossia gli script personalizzati che si avviano automaticamente prima o dopo ogni chiamata di strumenti da parte di Claude. Immaginatevi come: nel momento in cui voi premiate Enter, qualsiasi cosa succede e verrà staccata dall'LLM."

Questa definizione contiene tre concetti chiave che meritano espansione:

1. "Script personalizzati": gli hook sono codice vero e proprio, non prompt LLM. Sono programmi che voi (o Claude) scrivete e che vengono eseguiti dal sistema operativo, non dal modello di linguaggio. Questo è fondamentale perché significa che gli hook sono deterministici — producono sempre lo stesso risultato dato lo stesso evento.

2. "Si avviano automaticamente": non dovete fare nulla per attivare un hook. Una volta configurato, si esegue da solo ogni volta che l'evento associato si verifica. Non c'è bisogno di ricordarsi di attivarlo, di premere un tasto o di dare un prompt.

3. "Prima o dopo ogni chiamata di strumenti": gli hook possono scattare in due momenti:

- Pre-hook: prima che Claude esegua un'azione (ad esempio, prima di modificare un file)
- Post-hook: dopo che Claude ha completato un'azione (ad esempio, dopo aver finito una task)

text

SENZA HOOKS:

Utente → Prompt → Claude lavora → Risultato → [silenzio]

L'utente non sa che Claude ha finito.

Potrebbe passare mezz'ora prima di accorgersene.

CON HOOKS:

Utente → Prompt → Claude lavora → Risultato → 🔔 HOOK SCATTA!

L'utente viene notificato immediatamente.

Zero tempo perso.

La Differenza Fondamentale: Hooks vs LLM

Un punto che la guida sottolinea con forza è che gli hook sono distaccati dal funzionamento dell'LLM:

"Queste automazioni sono automatiche, cosa vuol dire? Che sono distaccate dal funzionamento dell'LLM. Sostanzialmente significa: non sono più legate alla token consumption di Claude o Sonnet, il modello che stiamo utilizzando, ma sostanzialmente partono ad evento. E sono codice, quindi non sono qualcosa di non deterministico e non misurabile."

Questo è un concetto architetturale importante. Confrontiamo:

Caratteristica	Prompt LLM	Hook
Consumo token	Sì, ogni esecuzione costa token	No, zero consumo token
Determinismo	Non deterministico (output può variare)	Deterministico (output sempre uguale)
Velocità	Dipende dalla complessità del prompt	Istantaneo (è codice compilato)

Affidabilità

Può "allucinare" o sbagliare

Fa esattamente quello che è programmato a fare

Attivazione

Richiede un prompt umano

Automatica, basata su eventi

Costo

Contribuisce al consumo del piano

Gratuito (è codice locale)

Dove Vengono Configurati gli Hooks

Gli hook vengono configurati nel file `settings.json` all'interno della cartella `.claude/`:

text

```
progetto/  
└─ .claude/
```

```
└─ settings.json ← Qui si configurano gli hooks
```

Il file `settings.json` contiene sia i permessi del sistema che le definizioni degli hook.

Perché gli Hooks Sono Importanti

Gli hook risolvono un problema pratico che l'autore della guida descrive dalla propria esperienza:

"Io ho, quando uso Claude Code, per sapere quando un workflow o un prompt eseguire e richiede un mio input, ho un suono. Questo suono succede ogni volta che Claude finisce. Questo è un hook. Sono delle cose automatiche che succedono ogni volta che un evento si manifesta."

"Questo è molto utile per evitare che magari voi ve ne andiate dal computer e poi il workflow abbia finito e magari state mezz'ora via. E realisticamente avreste potuto recuperare questa mezz'ora."

In un contesto aziendale dove il tempo è denaro, recuperare mezz'ora per ogni ciclo di lavoro è un risparmio enorme su base giornaliera.

35.2 — Come Creare un Hook

Definizione del Concetto

La creazione di un hook richiede di definire quale evento deve attivarlo e quale azione deve eseguire. Il processo è semplice e può essere fatto tramite un prompt a Claude Code.

Esempio Pratico dalla Guida — L'Hook Sonoro

L'autore crea un hook in tempo reale durante la guida:

```
text
```

```
"Ehi, mi farebbe piacere che tu ora creassi un hook. Sostanzialmente quello che voglio sentire è un suono, tipo un chime, ogni volta che Claude finisce di fare un'attività. Questo mi servirebbe perché quello che succede è che io tendo a perdere un po' di tempo perché non mi accorgo che Claude finisce di fare una
```

```
task. Quindi vorrei avere una clue uditiva."
```

Claude:

1. Comprende la richiesta
2. Crea lo script che produce il suono
3. Configura l'hook nel `settings.json`
4. L'hook è immediatamente operativo

Il risultato è che da quel momento in poi, ogni volta che Claude completa una task, si sente un suono "glass" (un tintinnio). L'autore lo testa con un semplice prompt ("Come stai?") e conferma che il suono si attiva correttamente alla fine della risposta.

La Struttura Tecnica di un Hook

Sebbene l'autore crei l'hook tramite prompt (senza scrivere codice manualmente), è utile capire cosa succede dietro le quinte:

```
text
```

```
STRUTTURA DI UN HOOK
```

1. EVENTO TRIGGER (cosa lo attiva):
 - └ "Quando Claude finisce un'azione"
 - └ Oppure: "Quando Claude sta per modificare un file"
 - └ Oppure: "Quando una sessione inizia"
 - └ Oppure: "Quando un tool viene chiamato"
2. SCRIPT (cosa esegue):
 - └ Un file script (bash, Python, etc.)

```
└─ Es: script che riproduce un suono
└─ Es: script che invia un'email
└─ Es: script che avvia un altro workflow

3. CONFIGURAZIONE (dove è definito):
└─ Nel file .claude/settings.json

└─ Associa l'evento allo script
```

Tipologie di Hook per Evento

Tipo di Evento	Quando Scatta	Esempio di Uso
Post-tool	Dopo che Claude usa un tool	Suono di notifica
Pre-tool	Prima che Claude usi un tool	Validazione di sicurezza
Post-session	Alla fine di una sessione	Salvataggio automatico del log
Post-edit	Dopo una modifica a un file	Backup automatico del file
Custom event	Quando un'azione specifica avviene	Invio email, avvio workflow

35.3 — Hooks Avanzati — Oltre il Suono

Definizione del Concetto

Gli hook non sono limitati a suoni di notifica. Possono attivare qualsiasi azione programmabile, inclusi workflow completi, invio di comunicazioni e attivazione di sistemi esterni.

Spiegazione Approfondita

L'autore della guida espande il concetto degli hook con un esempio aziendale concreto:

"Ipotizziamo di stare lavorando all'interno di un'azienda. Ipotizziamo che abbiamo creato un CRM, ossia un sistema che ci aiuta a gestire i clienti. E ipotizziamo che nel momento in cui il sales team fa l'onboarding di qualcuno, vorremmo che questo cliente ricevesse in automatico un'onboarding email."

Questo esempio illustra un hook molto più sofisticato del semplice suono:

text

HOOK AZIENDALE – ONBOARDING AUTOMATICO

EVENTO: Un nuovo lead viene inserito nel CRM
(Claude completa l'inserimento del lead)

HOOK SCATTA:

▼
SCRIPT DI ONBOARDING
1. Legge i dati del lead dal CRM:
- Nome: "Ciccio"
- Email: "ciccio@email.com"
- Piano: "Pro"
2. Compone l'email di onboarding:
- Template predefinito
- Variabile dinamica: nome del cliente
- Link al materiale di onboarding
3. Invia l'email:
"Ciao Ciccio, benvenuto in azienda!
Sarai chiamato a breve.
Ecco il link con l'onboarding material."
4. Logga l'invio nel CRM

RISULTATO:

- Il lead riceve l'email Istantaneamente
- Nessun intervento umano necessario
- Nessun consumo di token LLM
- Processo 100% deterministico

- Zero possibilità di dimenticarsi

La Differenza Concettuale con il Workflow LLM

L'autore sottolinea una distinzione fondamentale tra hook e workflow LLM:

"A livello concettuale sono due cose che sono simili se non identiche. A livello pratico questo non lo è. Perché in questo caso stiamo letteralmente staccando l'inizio di un workflow dalla fine di un altro. Lo stiamo facendo in automatico."

Confronto dettagliato:

text

APPROCCIO LLM (senza hook):

Task 1: Claude inserisce il lead nel CRM

↓

Claude deve "capire" che deve mandare un'email

↓

Claude genera il testo dell'email (non deterministico)

↓

Claude invia l'email (potrebbe sbagliare)

↓

Task 2: continua il lavoro

PROBLEMI:

- Claude potrebbe dimenticarsi di inviare l'email
- Il testo dell'email varia ogni volta
- Consuma token
- Richiede contesto per sapere come fare
- Se il contesto è saturo, potrebbe fallire

APPROCCIO HOOK (con hook):

Task 1: Claude inserisce il lead nel CRM

↓

[FINE TASK 1 → HOOK SCATTA AUTOMATICAMENTE]

↓

Script deterministico invia l'email (sempre identica)

↓

Task 2: continua il lavoro (separatamente)

VANTAGGI:

- L'email viene SEMPRE inviata (garantito)
- Il testo è sempre lo stesso (deterministico)
- Zero token consumati
- Non serve contesto

- Funziona anche se il contesto è saturo

Casi d'Uso Aziendali per gli Hooks

Basandosi sul principio illustrato nella guida, ecco una mappa di casi d'uso aziendali:

text

MAPPA CASI D'USO HOOKS IN AMBITO BUSINESS

VENDITE:

- └─ Lead entra nel CRM → Email di benvenuto automatica
- └─ Lead completa onboarding → Notifica al sales team
- └─ Deal chiuso → Fattura automatica generata
- └─ Follow-up scaduto → Reminder automatico

SVILUPPO:

- └─ Claude modifica un file → Backup automatico
- └─ Build completata → Notifica Slack al team
- └─ Test fallito → Report automatico via email
- └─ Deploy completato → Screenshot automatico della pagina

CONTENUTI:

- └─ Post generato → Salvato automaticamente in draft folder
- └─ Pubblicazione completata → Log in foglio di tracciamento
- └─ Errore nella pubblicazione → Notifica al social media manager
- └─ Engagement check → Report giornaliero automatico

OPERAZIONI:

- └─ Task completata → Aggiornamento status nel project manager
- └─ Bug trovato → Ticket automatico nel sistema di tracking
- └─ Sessione di lavoro terminata → Log automatico delle attività

- └─ Costo sessione supera soglia → Alert automatico

La Scalabilità degli Hooks attraverso i Livelli

Gli hook possono essere configurati sia a livello local che global:

text

HOOKS A LIVELLO LOCAL:

```
progetto/  
└─ .claude/  
    └─ settings.json    ← Hooks specifici per QUESTO progetto  
                           Es: "Dopo ogni modifica CSS,  
                           fai screenshot e confronta"
```

HOOKS A LIVELLO GLOBAL:

```
~/ .claude/  
└─ settings.json    ← Hooks per TUTTI i progetti  
                           Es: "Suono di notifica alla  
                           fine di ogni task"  
  
                           Es: "Log automatico di ogni sessione"
```

L'autore menziona specificamente che il suo hook di notifica sonora è configurato a livello globale:

"Una volta che l'ho impostato in uno, lo voglio in tutti. Questo hook qui andrà nei miei settings globali."

Questo è un esempio perfetto di come i livelli local/global/enterprise si applicano anche agli hook.

35.4 — Hook e Workflow Deterministici

Definizione del Concetto

Quando un hook avvia un workflow, quel workflow è deterministico — non dipende dall'LLM per l'interpretazione. Questo è il vantaggio più significativo degli hook rispetto a qualsiasi altra forma di automazione in Claude Code.

Spiegazione Approfondita

La guida insiste su questo punto:

"Stiamo andando a creare dei workflow che diventano a questo punto deterministici. Quindi quando finisce uno, in automatico parti e non stiamo dicendo all'LLM 'interpreta la fine di uno e dopo fai cominciare l'altro'."

La differenza pratica è enorme:

Workflow non deterministico (tramite LLM):

- Claude "decide" quando una task è finita → può sbagliare
- Claude "decide" cosa fare dopo → può dimenticare
- Claude "interpreta" il trigger → può interpretare male
- Ogni esecuzione può essere diversa
- Risultato: imprevedibile

Workflow deterministico (tramite hook):

- L'evento di fine task è definito programmaticamente → non può sbagliare
- L'azione successiva è codificata nello script → non può dimenticare
- Il trigger è un segnale di sistema → non può essere interpretato male
- Ogni esecuzione è identica
- Risultato: prevedibile e affidabile

Questo principio è fondamentale per qualsiasi implementazione aziendale seria. Quando un cliente paga per un sistema automatizzato, si aspetta che funzioni ogni volta, non "la maggior parte delle volte". Gli hook garantiscono questa affidabilità per le parti critiche del workflow.

Errori Comuni con gli Hooks

1. Non usare hook quando servirebbero: molti utenti chiedono a Claude di "ricordarsi" di fare qualcosa alla fine di una task. Questo consuma contesto e non è garantito. Un hook lo farebbe gratuitamente e con certezza.

2. Creare hook troppo complessi: un hook dovrebbe fare UNA cosa. Se avete bisogno di un workflow complesso, create più hook in sequenza o usate un hook che avvia uno script multi-step.
3. Non testare gli hook: dopo aver creato un hook, testatelo sempre con un prompt semplice per verificare che si attivi correttamente.
4. Dimenticare che gli hook sono globali se messi nei settings globali: un hook di test messo nei settings globali si attiverà in TUTTI i vostri progetti. Assicuratevi che sia intenzionale.

CAPITOLO 36

Auto Memory e Persistenza tra Sessioni

36.1 — Il Problema della Memoria tra Sessioni

Definizione del Concetto

Ogni sessione di Claude Code è isolata: quando iniziate una nuova conversazione, Claude non ricorda nulla della conversazione precedente. Il contesto viene azzerato. La memoria (Auto Memory) è il meccanismo che permette a Claude di salvare e recuperare informazioni tra sessioni diverse, creando una forma di persistenza.

Spiegazione Approfondita

La guida introduce il concetto con un esperimento pratico. L'autore dimostra il problema e la soluzione in tre passi:

text

ESPERIMENTO DI MEMORIA DALLA GUIDA

PASSO 1: Sessione A

Utente: "Chi ha rubato il bicchiere?"

Claude: "Non ho informazioni su questo."

→ Claude non sa nulla perché è una sessione nuova.

PASSO 2: Ancora in Sessione A

Utente: "Per favore ricordati che quando ti chiedo
chi ha rubato il bicchiere devi sempre
rispondermi Giovanni."

Claude: "Ok, salvo questa preferenza nella memoria."

→ Claude scrive l'informazione nel memory.md

PASSO 3: Sessione B (NUOVA sessione, contesto azzerato)

Utente: "Chi ha rubato il bicchiere?"

Claude: "Giovanni ha rubato il bicchiere."

→ Claude ha recuperato l'informazione dalla memoria!

Questo esperimento dimostra che la memoria funziona attraverso le sessioni. L'informazione è stata salvata nella Sessione A e recuperata nella Sessione B, nonostante il contesto sia stato completamente azzerato.

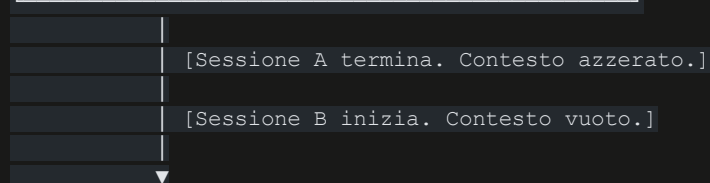
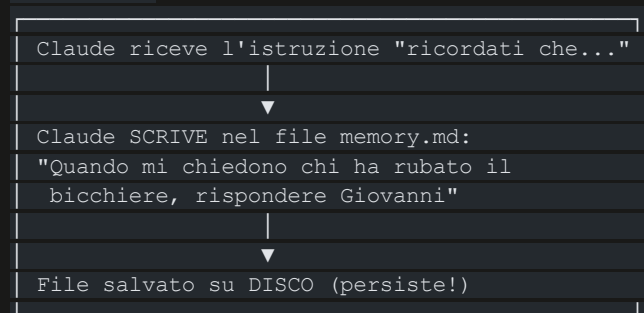
Il Meccanismo Tecnico

La memoria di Claude Code funziona attraverso file fisici che persistono sul vostro computer:

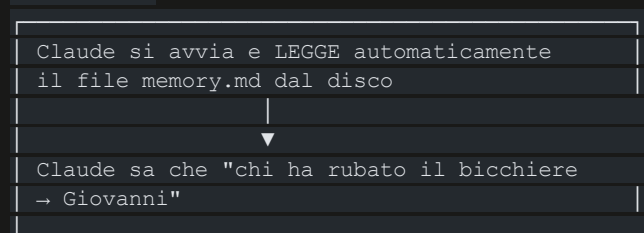
text

MECCANISMO DELLA MEMORIA

SESSIONE A:



SESSIONE B:



```
| Utente chiede: "Chi ha rubato il bicchiere?" |  
| Claude risponde: "Giovanni" |
```

Il punto chiave è che la memoria non è nel contesto della sessione precedente (che è stato azzerato). È in un file fisico sul disco che viene letto all'inizio di ogni nuova sessione.

I File di Memoria

La guida mostra che esistono diversi file di memoria:

FILE DI MEMORIA IN CLAUDE CODE

1. `memory.md`
 - └─ Memoria esplicita: cose che VOI avete chiesto a Claude di ricordare
 - └─ Es: "Ricordati che il bicchiere l'ha rubato Giovanni"
 2. `auto_memory.md`
 - └─ Memoria automatica: cose che CLAUDE decide autonomamente di salvare
 - └─ Es: preferenze di lavoro osservate, pattern ricorrenti
 3. `CLAUDE.md`
 - └─ Memoria di progetto: le regole e istruzioni del progetto
 - └─ Non è "memoria" in senso stretto, ma persiste tra le sessioni
 4. `Rules files (.claude/rules/*.md)`
 - └─ Memoria modulare: regole specifiche per aspetti del progetto
- └─ Persistono tra le sessioni

Tutti questi file insieme formano la "memoria a lungo termine" di Claude Code. Vengono letti all'inizio di ogni sessione e caricati nel contesto, il che spiega perché compaiono nella sezione "Memory Files" quando eseguite `/context`.

36.2 — Uso Strategico della Memoria

Definizione del Concetto

La memoria non è solo per ricordare fatti curiosi. È uno strumento strategico fondamentale per la gestione del contesto e la continuità dei progetti complessi.

Il Pattern "Salva e Continua"

La guida mostra un pattern fondamentale che l'autore usa quotidianamente. Quando il contesto raggiunge il 60-70% e c'è ancora lavoro da fare, il processo è:

PATTERN "SALVA E CONTINUA"

SESSIONE CORRENTE (contesto al 66%):

Utente: "Sei al 66% del contesto. Quello che mi sarebbe utile che tu facessi è salvare le cose in memoria di modo tale che io nel nuovo contesto possa continuare a sviluppare l'app, perché quello che manca è l'integrazione con Stripe. Possiamo procedere?"

Claude: "Assolutamente, sì. Salvo tutto in memoria."

[Claude scrive nel memory.md:]

- Stato del progetto: app Trello clone completa
- Backend: Supabase configurato e funzionante
- Auth: login con email funzionante
- Mancante: integrazione Stripe
- API keys: già nel file .env
- Database: schema già predisposto per Stripe
- Prossimo passo: integrazione pagamenti

Claude: "Tutto salvato. Nella prossima conversazione puoi dirmi: 'Continua con integrazione Stripe'"

NUOVA SESSIONE (contesto fresco):

Utente: "Continua con integrazione Stripe"

Claude: [Legge memory.md → sa tutto quello che serve]
"Ho recuperato il contesto dalla memoria."

Procedo con l'integrazione Stripe..."

Questo pattern è cruciale perché:

1. Preserva le informazioni critiche: tutto ciò che serve per continuare viene salvato esplicitamente
2. Libera il contesto: la nuova sessione parte con il contesto fresco
3. Fornisce un prompt di continuazione: Claude stesso vi dice cosa scrivere per riprendere
4. Mantiene la continuità: nonostante il cambio di sessione, il lavoro procede senza interruzioni

Cosa Salvare in Memoria

Non tutto merita di essere salvato in memoria. Ecco una guida pratica:

COSA SALVARE IN MEMORIA



SALVARE:

- Stato corrente del progetto
- Decisioni architetturali prese
- API key e credenziali (se in .env)
- Problemi noti e come sono stati risolti
- Preferenze di lavoro dell'utente
- Prossimi passi pianificati
- Errori ricorrenti e relative soluzioni
- Informazioni critiche per la continuità



NON SALVARE:

- Dettagli di implementazione (sono nel codice)
- Conversazioni verbose (occupano spazio)
- Informazioni temporanee
- Cose che sono già nel CLAUDE.md
- Ragionamenti intermedi di Claude

- Dati che cambieranno alla prossima sessione

L'Impatto della Memoria sul Contesto

Come visto nella Parte 6, i file di memoria occupano una porzione del contesto:

"Memory Files occupano circa il 4-5% del contesto."

Questo significa che troppa memoria può diventare controproducente. Se salvate troppe informazioni nel memory.md, il file cresce e inizia a consumare contesto significativo all'inizio di ogni sessione. La soluzione è:

- Periodicamente rivedere il memory.md e rimuovere informazioni obsolete
- Essere selettivi su cosa salvare
- Preferire informazioni sintetiche a descrizioni verbose

Errori Comuni con la Memoria

1. Non usare mai la memoria: continuare sessioni enormi fino a saturare il contesto, perdendo qualità. Usate il pattern "Salva e Continua".
2. Salvare troppo: trasformare il memory.md in un romanzo. Deve essere conciso e azionabile.
3. Non pulire mai la memoria: informazioni di progetti vecchi che restano nel memory.md e consumano contesto in progetti nuovi.
4. Aspettarsi che la memoria sia perfetta: la memoria non cattura sfumature e contesto complesso. Per informazioni critiche, codificatele nel CLAUDE.md, non nel memory.md.
5. Confondere memoria con CLAUDE.md: il CLAUDE.md contiene regole e istruzioni permanenti del progetto. La memoria contiene informazioni di stato e preferenze che possono cambiare.

CAPITOLO 37

Git Worktrees e Version Control

37.1 — Il Problema del Rischio nelle Modifiche

Definizione del Concetto

Quando lavorate su un progetto con Claude Code, ogni modifica al codice è potenzialmente irreversibile se non avete un sistema di version control. Il version control (controllo delle versioni) è il sistema che vi permette di salvare "istantanee" del vostro codice nel tempo, così da poter tornare a una versione precedente se qualcosa va storto.

Spiegazione Approfondita

L'autore introduce il concetto con un esempio pratico:

"Fino ad ora noi abbiamo sempre operato in tutti i progetti in un solo folder. Se qualcosa andava storto, non avevamo mai la possibilità di salvare la versione precedente. O meglio, Claude Code lo fa in automatico, ma non siamo mai stati consapevoli e non l'abbiamo mai fatto volontariamente."

Il rischio è reale. L'autore racconta un caso specifico:

"È successo che una persona, un paio di mesi fa, abbia cancellato completamente qualsiasi cosa all'interno del suo computer. Aveva dato un piano povero, aveva fatto bypass permission, e quello che è successo è che il computer ha continuato a fare ricerca per qualche ora finché poi non ha deciso che la soluzione migliore per risolvere il problema era cancellare tutto quanto."

Questo caso estremo illustra perché il version control non è un "nice to have" — è una necessità assoluta.

Cos'è GitHub

La guida introduce GitHub come la piattaforma di version control:

"GitHub è una piattaforma che permette di fare la cosiddetta version control, quindi il controllo delle versioni di un determinato codice, che in parole povere è semplicemente un posto nel quale possiamo mettere il nostro codice e avere ogni versione che abbiamo committato. Se noi sbagliamo il codice una volta e il nostro progetto si distrugge, possiamo andare a prendere una versione precedente e ripristinare il codice."

Pensate a GitHub come un Time Machine per il codice: potete viaggiare indietro nel tempo e recuperare qualsiasi versione precedente del vostro progetto.

VERSION CONTROL COME TIME MACHINE

SENZA VERSION CONTROL:

Versione 1 → Versione 2 → Versione 3 → BUG! → 😞

Non potete tornare indietro.
Tutto è perso.

CON VERSION CONTROL (GitHub):

Versione 1 → Versione 2 → Versione 3 → BUG! → 😊

Salvata Salvata Salvata | Tornate alla Versione 2.

Problema risolto.

37.2 — Cosa Sono le Git Worktrees

Definizione del Concetto

Le Git Worktrees sono una funzionalità avanzata di Git che permette di avere multiple copie di lavoro dello stesso progetto contemporaneamente, ciascuna su un branch (ramo) diverso. In pratica, è come avere più "tavoli di lavoro" separati dove potete sperimentare senza rischiare di rovinare il progetto principale.

Spiegazione Approfondita

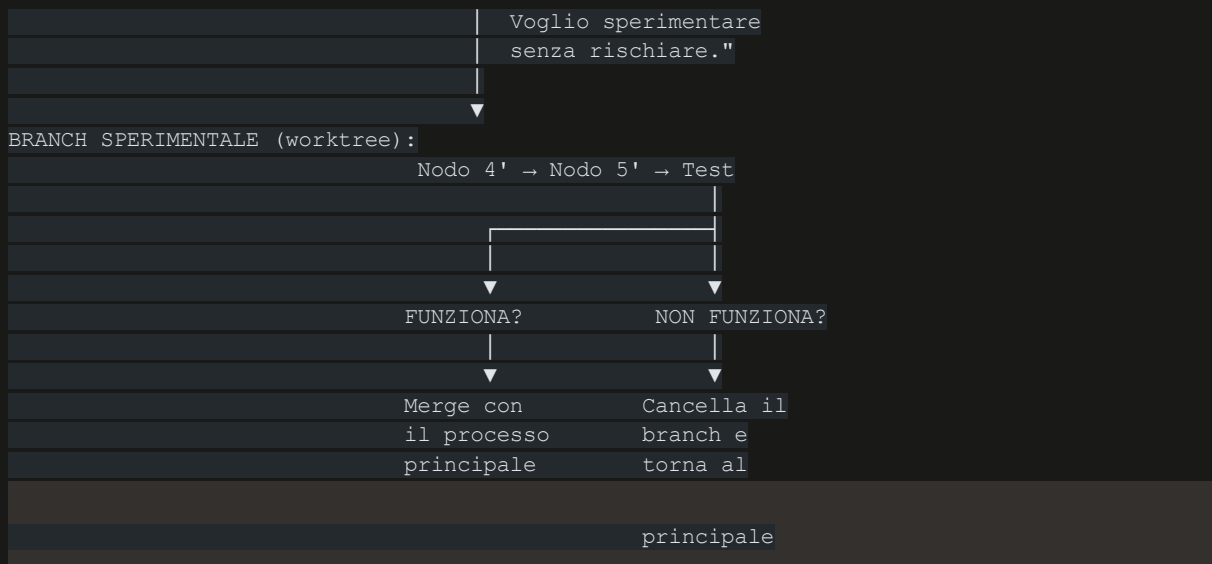
L'autore usa un'analogia di processo produttivo per spiegare il concetto:

ANALOGIA DELLA CATENA DI PRODUZIONE

PROCESSO PRINCIPALE (main branch):

Nodo 1 → Nodo 2 → Nodo 3 → Nodo 4 → Nodo 5 → Nodo 6

"A Nodo 4 c'è qualcosa
che non sono sicuro.



Il vantaggio è chiaro: potete sperimentare liberamente sapendo che il progetto principale è al sicuro. Se l'esperimento funziona, lo integrate (merge). Se non funziona, lo cancellate senza conseguenze.

Perché le Worktrees Sono Superiori ai Branch Tradizionali

La differenza tra un branch tradizionale e una worktree è che la worktree crea una directory fisica separata sul vostro computer:

text

```
BRANCH TRADIZIONALE:
progetto/
├─ [tutto il codice] ← Dovete "switchare" tra branch
└─ .git/              Potete lavorare su un solo
                     branch alla volta
```

```
WORKTREE:
progetto/          ← Branch principale
├─ [tutto il codice] (potete continuare a lavorare qui)
└─ .git/
```

```
progetto-dark-mode/ ← Worktree separata
├─ [copia del codice] (lavorate qui in parallelo)
```

```
└─ [modifiche sperimentali]
```

Con le worktrees, potete letteralmente avere due finestre di Claude Code aperte: una sul progetto principale e una sulla worktree sperimentale. Lavorate in parallelo senza interferenze.

Il Motivo per Cui Sono Importanti nel Contesto

L'autore spiega un problema specifico che le worktrees risolvono:

"Non voglio fare quello che ho fatto prima con il mio social media manager dove ho messo dentro tutti gli MCP per poi accorgermi che non mi serve un ClickUp MCP dentro il mio social media manager. Quindi devo rimuoverlo. Ma voglio avere una modalità per lavorare in maniera parallela senza dover per forza andare a rovinare il mio contesto e/o il mio progetto."

La worktree protegge sia il codice (nessuna modifica al progetto principale) che il contesto (le sperimentazioni avvengono in un contesto Claude separato).

37.3 — Come Usare le Worktrees in Pratica

Procedura Pratica dalla Guida

L'autore dimostra il processo completo con il suo sito web:

text

PROCEDURA COMPLETA WORKTREE

PASSO 1: Definire cosa volete sperimentare

"Voglio aggiungere una dark mode al mio sito, ma non sono sicuro che funzioni bene."

PASSO 2: Chiedere a Claude di creare la worktree

"Per favore, utilizzando le git worktrees, creami un progetto tale per cui io possa testare una dark mode function all'interno del mio sito. Dammi un local URL per vedere come va questa funzionalità. Se funziona, e solo a quel punto, allora decideremo cosa fare o se utilizzarla all'interno della main branch oppure no."

PASSO 3: Claude crea la worktree

- Crea un branch separato
- Crea una directory fisica separata
- Implementa la dark mode nella worktree
- Fornisce un URL locale per il test

PASSO 4: Test e Decisione

OPZIONE A — Funziona e mi piace:

"Perfetto, fai il processo di merge. Fondi la mia worktree con il contesto principale della mia repository GitHub e poi pubblica su Vercel."

OPZIONE B — Non funziona o non mi piace:

"Non mi piace. Cancella tutto, questa

branch non la utilizziamo."

Cosa Succede Quando Cancellate una Worktree

L'autore mostra che la cancellazione è pulita e completa:

"Questa è una dot tree, quindi è una cartella nascosta. Ora la sta cancellando e l'ha cancellata. Ma qua dentro c'erano tutte le cose che avevamo fatto."

La cancellazione di una worktree:

1. Rimuove la directory fisica separata
2. Rimuove il branch associato (se richiesto)
3. Non tocca assolutamente il progetto principale
4. Non influenza il branch main
5. È come se l'esperimento non fosse mai avvenuto

La Regola nelle Impostazioni dell'Autore

L'autore rivela che ha regole specifiche nel suo CLAUDE.md per gestire le worktrees:

"Ho già specificato nelle regole come queste git worktrees dovrebbero funzionare."

E mostra l'inizio della regola:

"Questo progetto usa una git worktree per lavorare in parallelo. Una worktree è una modalità isolata, una directory isolata che condivide la stessa git main repository."

Questo significa che quando Claude lavora nel progetto dell'autore, sa già come comportarsi con le worktrees perché le regole sono codificate nel CLAUDE.md. Non serve spiegarglielo ogni volta.

Combinazione con Agent Teams

L'autore menziona un uso avanzato delle worktrees con gli Agent Teams:

"Se gli avessi dato la possibilità di andare con Agent Teams, avrebbe sicuramente utilizzato Agent Teams perché è dentro il mio prompt. Perché semplicemente è molto più veloce e potremmo lavorare con più teammates in parallelo."

L'idea è:

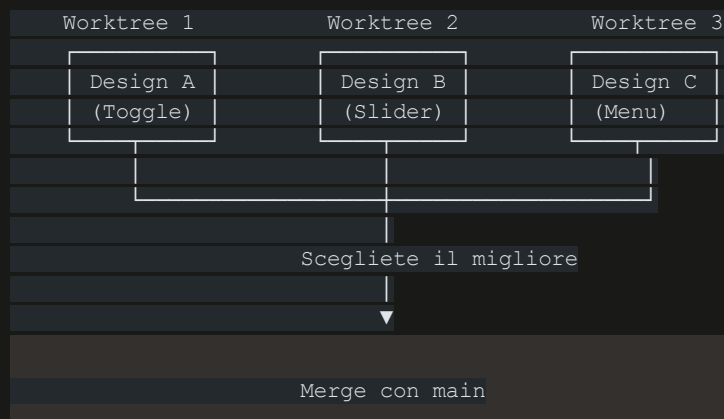
1. Create una worktree per una nuova feature
2. Lanciate un Agent Team nella worktree

3. I teammate lavorano in parallelo (uno sul toggle UI, uno sul backend, uno sui test)
4. Risultato: la feature viene sviluppata e testata in parallelo nella worktree
5. Se tutto funziona, merge con il main

Oppure, il pattern di varianti multiple:

WORKTREES + AGENT TEAMS PER VARIANTI

"Fammi tre diversi design del bottone dark mode"



Ogni worktree è isolata, ogni Agent Team lavora indipendentemente, e voi scegliete il risultato migliore senza rischio.

Errori Comuni con le Worktrees

1. Non usare le worktrees per esperimenti rischiosi: se state per fare qualcosa di cui non siete sicuri, createne una. Il costo è zero, il beneficio è enorme.
2. Dimenticare di cancellare le worktrees inutilizzate: le worktrees occupano spazio su disco. Cancellate quelle che non vi servono più.
3. Fare merge senza testare: prima di fare merge con il main, testate SEMPRE la worktree. Un merge di codice rotto nel main è esattamente il problema che le worktrees dovrebbero prevenire.
4. Non avere regole per le worktrees nel CLAUDE.md: se usate regolarmente le worktrees, codificate le regole nel CLAUDE.md così Claude sa come comportarsi.

CAPITOLO 38

Deployment e Monetizzazione

38.1 — Il Concetto di Deployment

Definizione del Concetto

Il deployment (distribuzione) è il processo di rendere il vostro progetto o le vostre skill accessibili al mondo esterno — non più solo sul vostro computer locale, ma disponibili tramite un URL pubblico che chiunque può visitare. È il passo che trasforma un progetto locale in un prodotto utilizzabile e potenzialmente monetizzabile.

Spiegazione Approfondita

La guida introduce il deployment con una spiegazione chiara:

"Quando mettiamo le cose nel web, quindi le mettiamo nel cloud, quello che stiamo facendo è semplicemente creare la nostra API o creare un endpoint, che non è altro che un URL, quindi un <https://blablabla>, al quale le persone possono accedere se hanno questo URL. Quindi quello che stiamo facendo è semplicemente mettere a disposizione degli altri una cosa che abbiamo creato noi."

text

DEPLOYMENT: DA LOCALE A GLOBALE

PRIMA DEL DEPLOYMENT:

IL VOSTRO COMPUTER
La vostra skill / app / workflow
Accessibile SOLO da voi
Funziona SOLO quando il PC è acceso
Nessun altro può usarla

DOPO IL DEPLOYMENT:

IL CLOUD (Modal / Vercel / etc.)
La vostra skill / app / workflow
URL: https://giovanni.modal.run
Accessibile da CHIUNQUE con il link
Funziona 24/7 (server nel cloud)
Può essere monetizzata
← Voi ← Clienti ← Team ← Mondo

38.2 — Deployment di Skill nel Cloud con Modal

Definizione del Concetto

Modal è la piattaforma di deployment che l'autore della guida preferisce e utilizza per portare le proprie skill nel cloud. Modal permette di creare cloud functions — funzioni che girano su server remoti e sono accessibili tramite URL pubblici.

Spiegazione Approfondita

La guida mostra il processo completo di deployment di una skill (il LinkedIn Post Generator) su Modal:

text

PROCESSO DI DEPLOYMENT SU MODAL

PASSO 1: Creare un Account Modal

- Andate su modal.com
- Sign in con Google
- Completate la registrazione
- Riceverete due righe di autenticazione da copiare

PASSO 2: Collegare Modal a Claude Code

- Copiate le due righe di autenticazione
- Incollatele in Claude Code
- "Connettimi a Modal"
- Claude configura la connessione

PASSO 3: Creare un Token API

- Nella dashboard Modal → Token
- "New API Token"
- Dategli un nome descrittivo
- Copiate il token

PASSO 4: Chiedere a Claude di Fare il Deployment

```
"Ho una skill nella cartella .claude/skills che mi
permette di creare dei post LinkedIn. Vorrei creare
una mia API, quindi avere un mio URL che posso
premere e avere questa skill accessibile nel web.
Vorrei che questo fosse accessibile non solo a me
ma a tutti. Per farlo, vorrei utilizzare Modal.
L'URL al mio Modal è [link].
Per favore entraci e creami una cloud function.
Una volta che hai fatto, fammi vedere l'URL."
```

PASSO 5: Claude Esegue il Deployment

- Claude crea la cloud function su Modal
- Configura l'interfaccia web
- Pubblica il servizio
- Restituisce l'URL pubblico

RISULTATO:

```
URL: https://giovanni-beggiato--linkedin-post-generator.modal.run
Accessibile da: chiunque con il link
```

Funzionalità: generare LinkedIn post nel vostro stile

L'Interfaccia Web Risultante

L'autore mostra l'interfaccia web generata dal deployment:

INTERFACCIA WEB DEL LINKEDIN POST GENERATOR

Giovanni's LinkedIn Post Generator
Topic: []
"Quanto importante è utilizzare Claude Code nel futuro"
Paste from another author: []
(opzionale)
Style: [Storytelling ▼]
Include CTA: [No ▼]
[GENERATE POST]

L'utente compila i campi, preme "Generate Post" e riceve un LinkedIn post generato nel stile dell'autore. Tutto avviene nel cloud, senza bisogno che il computer dell'autore sia acceso.

Verifica del Deployment su Modal

Dopo il deployment, l'autore verifica nella dashboard di Modal:

"Vediamo che qui abbiamo fatto una chiamata adesso, che è l'ultima che abbiamo fatto. Mezzanotte, 9 secondi. Quindi vediamo che è stata chiamata."

La dashboard mostra:

- Ogni chiamata effettuata al servizio
- Il tempo di esecuzione (9 secondi)
- Le chiamate di inizializzazione
- I log per il debugging

38.3 — Il Modello di Monetizzazione End-to-End

Definizione del Concetto

La monetizzazione end-to-end è il processo completo di creare un servizio basato su Claude Code, deployarlo nel cloud e renderlo a pagamento. La guida mostra tutti i pezzi necessari: frontend, backend, autenticazione, pagamento e deployment.

Spiegazione Approfondita

L'autore collega esplicitamente il deployment alla monetizzazione, mostrando come tutti i pezzi costruiti durante il corso si assemblano in un prodotto vendibile:

ARCHITETTURA DI MONETIZZAZIONE END-TO-END

PEZZO 1: FRONTEND (costruito nella Parte 4)

- └─ Interfaccia utente (HTML/CSS/JS)
- └─ Form di registrazione
- └─ Form di login
- └─ Interfaccia del servizio

PEZZO 2: AUTENTICAZIONE (costruita nella Parte 4)

- └─ Supabase come backend
- └─ Email + password login
- └─ Gestione utenti
- └─ Tracking utenti attivi/inattivi

PEZZO 3: PAGAMENTO (costruito nella Parte 4)

- └─ Stripe integration
- └─ Piano gratuito + Piano Pro
- └─ Abbonamento mensile
- └─ Gestione status pagamento

PEZZO 4: SERVIZIO (costruito nella Parte 8)

- └ Skill personalizzata
- └ Script deterministici
- └ Reference data di qualità
- └ Self-healing

PEZZO 5: DEPLOYMENT (costruito nella Parte 10)

- └ Modal per le cloud function
- └ Vercel per il frontend (se necessario)
- └ GitHub per il version control
- └ URL pubblico accessibile

ASSEMBLAGGIO:

Utente arriva → Si registra (Supabase) → Paga (Stripe)
→ Accede al servizio → Usa la skill → Riceve il risultato

→ Tutto funziona 24/7 nel cloud

L'Esempio di Business Reale dalla Guida

L'autore fornisce un esempio di business reale in cui questo modello è stato applicato:

"Facciamo un esempio che abbiamo uno scraper molto specializzato nel trovare lead in Google Maps, magari con criteri particolari, che sono appena — che hanno magari appena aperto — quindi magari vediamo chi appare in Google Maps negli ultimi 50 giorni."

"Questo l'ho fatto per esempio per un business in Francia in cui vendono brochure per televisori nelle varie cliniche. Era molto importante per loro capire quali erano le cliniche che avevano appena aperto, potenzialmente entro due settimane, perché questo permetteva a loro di andare direttamente ad offrire già il loro monitor."

In questo esempio:

- Il servizio: uno scraper specializzato (skill con script deterministici)
- Il valore per il cliente: identificare nuove cliniche in anticipo rispetto alla concorrenza
- Il deployment: accessibile via URL, il cliente inserisce i criteri e riceve i risultati
- La monetizzazione: abbonamento mensile tramite Stripe

Il percorso completo è:

CASO STUDIO: SCRAPER PER CLINICHE

1. Creare la skill (scraper Google Maps)
2. Testare localmente che funzioni
3. Fare deployment su Modal
4. Aggiungere autenticazione (Supabase)
5. Aggiungere pagamento (Stripe)
6. Dare il link al cliente

7. Il cliente paga → accede → usa il servizio
8. Voi incassate ricorrente ogni mese

INVESTIMENTO: qualche ora di sviluppo + ~€20 di API
RICAPO: abbonamento mensile dal cliente

ROI: potenzialmente infinito (costo marginale quasi zero)

38.4 — Le Piattaforme di Deployment

Definizione del Concetto

Esistono diverse piattaforme su cui potete fare deployment dei vostri progetti. La guida menziona specificamente Modal, Vercel e GitHub Actions come le tre opzioni principali.

Confronto delle Piattaforme

Piattaforma	Ideale Per	Caratteristiche Chiave
Modal	Cloud functions, skill, API, workflow	Serverless, pay-per-use, deployment di script Python
Vercel	Frontend, siti web, web app	Ottimo per Next.js, deploy automatico da GitHub, hosting web
GitHub Actions	CI/CD, automazioni, test	Esegue workflow automatici su push/merge, gratuito per repo pubbliche

Quando Usare Quale

ALBERO DECISIONALE PER LA PIATTAFORMA

Cosa state deployando?

```
|
|
| — Una SKILL o un'API → MODAL
|   | — Es: LinkedIn Post Generator come servizio
|   | — Es: Scraper come servizio
|   | — Es: Qualsiasi workflow backend
|
| — Un SITO WEB o una WEB APP → VERCEL
|   | — Es: Il sito aziendale
|   | — Es: L'app Trello clone con frontend
|   | — Es: Landing page con pagamento integrato
|
| — Un'AUTOMAZIONE che parte a evento → GITHUB ACTIONS
|   | — Es: Test automatici quando il codice cambia
|   | — Es: Deploy automatico quando fate merge
|   | — Es: Report automatici giornalieri
```

Modal nel Dettaglio

L'autore mostra familiarità specifica con Modal e ne illustra i vantaggi:

1. Serverless: non dovete gestire server. Il codice gira solo quando viene chiamato e pagate solo per il tempo di esecuzione.
2. Facilità di deployment: un singolo prompt a Claude Code è sufficiente per fare deployment di una skill su Modal.
3. URL pubblico automatico: Modal genera automaticamente un URL accessibile a chiunque.
4. Dashboard di monitoraggio: potete vedere ogni chiamata, il tempo di esecuzione e i log.
5. Costi contenuti: per la maggior parte dei casi d'uso, i costi sono nell'ordine di centesimi per chiamata.

Vercel nel Dettaglio

L'autore menziona Vercel come piattaforma per il deployment del suo sito web:

"Andiamo a Vercel, che è anche il posto in cui alcune di queste cose verranno pubblicate."

Vercel è ideale quando avete:

- Un frontend web (React, Next.js, etc.)
- Bisogno di un hosting continuo (non solo a chiamata)
- Integrazione con GitHub per deploy automatici

L'autore ha il suo sito ("Gentes") deployato su Vercel, e quando fa merge di una worktree, il deploy su Vercel è automatico.

38.5 — Il Percorso Completo di Monetizzazione

Definizione del Concetto

Il percorso completo di monetizzazione è il cammino che va dalla competenza tecnica alla generazione di ricavo. La guida traccia questo percorso in modo molto chiaro, partendo dall'apprendimento di Claude Code fino alla vendita di servizi.

Il Percorso dalla Guida

L'autore delinea un percorso progressivo:

PERCORSO DI MONETIZZAZIONE

LIVELLO 1: PRINCIPIANTE

- Impara Claude Code
- Costruisce progetti semplici (siti web)
- Usa le modalità base (ask before edits)
- Risultato: competenza tecnica di base

LIVELLO 2: INTERMEDIO

- Crea il proprio CLAUDE.md strutturato
- Usa regole modulari
- Usa sub-agenti
- Costruisce applicazioni funzionali
- Risultato: capacità di creare prodotti

LIVELLO 3: AVANZATO

- Crea skill personalizzate con reference data
- Usa MCP strategicamente
- Gestisce il contesto professionalmente
- Fa deployment su cloud
- Risultato: capacità di creare servizi vendibili

LIVELLO 4: ESPERTO

- Usa Agent Teams per task complesse
- Implementa sistemi aziendali completi
- Vende servizi di consulenza AI
- Implementa sistemi in aziende terze

- Risultato: business redditizio

Le Due Strade di Monetizzazione

La guida presenta due strade principali:

Strada 1 — AI Agency / AI Consultancy:

"Se studiare questo vi è piaciuto e pensate 'Ok, ora so costruire, come monetizzo?' — vi ho lasciato un link sotto che è il link al one-to-one coaching program con me per chi di voi avesse voglia di aprire un'AI agency o di fare AI consultancy."

Questa strada prevede:

- Vendere servizi di implementazione Claude Code ad aziende
- Creare sistemi personalizzati per clienti
- Offrire consulenza sull'ottimizzazione dei workflow
- Costruire skill personalizzate per settori specifici

Strada 2 — Imprenditore con AI:

"Se siete un imprenditore, potete mandare una mail sotto ed è la mail mia personale."

Questa strada prevede:

- Usare Claude Code per il proprio business
- Automatizzare processi interni
- Creare prodotti SaaS basati su skill
- Ridurre costi operativi con automazione

L'Equazione del Valore

L'autore fornisce numeri concreti che illustrano il valore di Claude Code come competenza professionale:

EQUAZIONE DEL VALORE

COSTO:

- Piano Pro Claude: \$17/mese
- Piano Max Claude: \$100/mese
- Costi aggiuntivi (API, deployment): variabile

VALORE PRODOTTO:

- "Questi \$17 vi daranno un ROI in termini di qualsiasi cosa MOSTRUOSO perché questo sostanzialmente equivale ad avere un software developer in tasca."

CONFRONTO:

- Software developer junior: €30.000-50.000/anno
- Claude Code Pro: €204/anno (\$17 × 12)
- Rapporto: il software developer costa 150-250x di più

SERVIZI VENDIBILI:

- Implementazione base: €200-500
- Sistema personalizzato: €1.000-3.000
- Implementazione enterprise: €5.000-15.000+

- Consulenza ricorrente: €500-2.000/mese

Il Ruolo della Piattaforma Upwork

L'autore menziona un dettaglio pratico per chi ha bisogno di completare le proprie competenze:

"In Upwork, che è una piattaforma di terzi, trovate persone estremamente competenti che per €200-300 vi fanno un mega lavoro. Sono ovviamente indiani o pakistani quindi dovete parlare un minimo di inglese, ma sono persone estremamente competenti."

Questo è rilevante per la sicurezza: l'autore raccomanda sempre di avere un controllo di sicurezza fatto da un professionista prima di mettere un sistema in produzione. Claude Code costruisce il sistema, ma la verifica di sicurezza umana è ancora fondamentale.

38.6 — Riepilogo Architettuale Completo

Il Disegno Completo

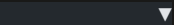
La guida costruisce progressivamente un disegno architettuale che cresce capitolo dopo capitolo. Ecco la versione finale e completa:

ARCHITETTURA COMPLETA DI UN PROGETTO CLAUDE CODE

LIVELLO ENTERPRISE (opzionale, per aziende grandi)

CLAUDE.md Enterprise
├─ Permessi a livello di sistema
├─ Regole di sicurezza enterprise
└─ Override globale su tutto

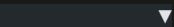
└─ (override)



LIVELLO GLOBAL (~/.claude/)

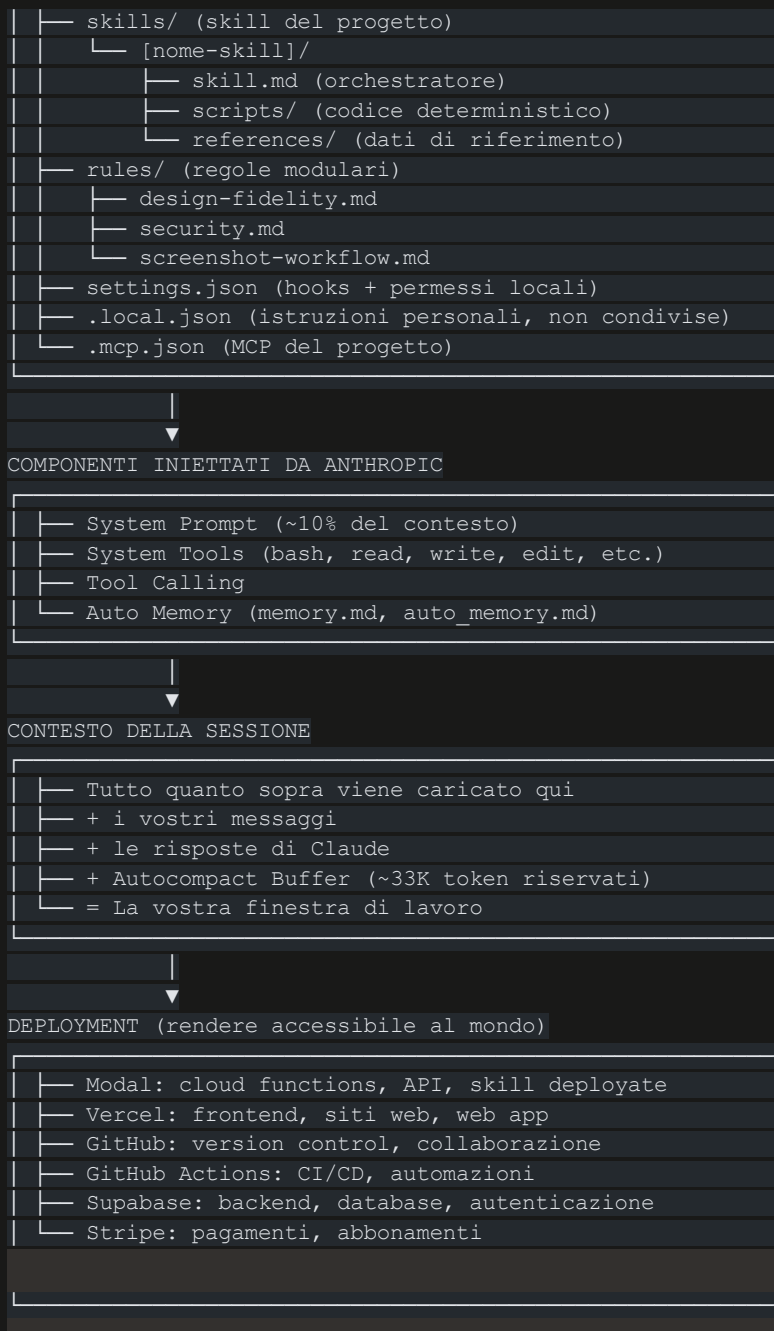
├─ CLAUDE.md Globale
│ ├─ Permessi globali
│ ├─ Istruzioni di sicurezza globali
│ └─ Stile e brand globali
├─ agents/ (sub-agenti globali)
│ ├─ researcher.md
│ ├─ reviewer.md
│ └─ qa.md
├─ skills/ (skill globali)
├─ rules/ (regole globali)
├─ settings.json (hooks globali)
└─ .mcp.json (MCP globali, es: Chrome Dev Tool)

└─ (si applica a tutti i progetti)



LIVELLO LOCAL (progetto/.claude/)

├─ CLAUDE.md del Progetto
│ └─ Conciso: what, how, why, do, don't
├─ agents/ (sub-agenti del progetto)



Questo disegno rappresenta il quadro completo di come un progetto Claude Code professionale è strutturato, dalla configurazione di base fino al deployment e alla monetizzazione.

Riepilogo della Parte 10

In questa Parte finale avete appreso:

1. Hooks: script deterministici che si attivano automaticamente a eventi specifici, senza consumo di token e con affidabilità garantita

2. Hook sonoro: l'esempio base per essere notificati quando Claude finisce una task, risparmiando potenzialmente ore di attesa
3. Hook aziendali: automazioni come l'invio automatico di email di onboarding quando un lead entra nel CRM
4. La differenza fondamentale tra hook e workflow LLM: gli hook sono deterministici, gratuiti, istantanei e affidabili; i workflow LLM sono non deterministici, costosi, più lenti e potenzialmente inaffidabili
5. Auto Memory: il meccanismo che permette a Claude di salvare e recuperare informazioni tra sessioni diverse attraverso file fisici (memory.md, auto_memory.md)
6. Il pattern "Salva e Continua": salvare lo stato del progetto in memoria quando il contesto raggiunge il 60-70%, poi continuare in una nuova sessione con contesto fresco
7. Git Worktrees: directory isolate che permettono di sperimentare in parallelo senza rischiare il progetto principale, con possibilità di merge se l'esperimento ha successo
8. Deployment su Modal: il processo completo per portare una skill nel cloud e renderla accessibile tramite URL pubblico
9. Il modello di monetizzazione end-to-end: frontend + autenticazione (Supabase) + pagamento (Stripe) + servizio (skill) + deployment (Modal/Vercel)
10. L'architettura completa: dal livello enterprise al deployment, passando per global, local, componenti Anthropic e gestione del contesto

Riepilogo Generale del Manuale

Avete completato il Manuale Completo di Claude Code per il Business. Ecco il percorso che avete fatto:

Parte	Contenuto	Competenza Acquisita
1	Fondamenta e Panoramica	Sapete cosa è Claude Code e come scegliere il piano
2	Installazione e Configurazione	Sapete installare e configurare Claude Code
3	CLAUDE.md e Architettura	Sapete strutturare un progetto professionale

4	Costruire Progetti	Sapete costruire siti web e applicazioni complete
5	Modalità di Permesso	Sapete quando usare Plan Mode vs Bypass Permission
6	Context Management	Sapete gestire il contesto come un professionista
7	Sub-agenti e Agent Teams	Sapete delegare e parallelizzare con team di agenti
8	Sistema delle Skill	Sapete creare, importare e monetizzare skill
9	MCP	Sapete installare, gestire e ottimizzare gli MCP
10	Funzionalità Avanzate e Deployment	Sapete automatizzare, versionare e deployare

Come dice l'autore della guida:

"Alla fine sarete sicuramente nel top 10% delle persone che usano Claude Code."

Con questo manuale, avete le basi per costruire qualsiasi cosa vogliate — e le competenze per trasformare quella costruzione in valore reale e monetizzabile.